

Herong's Notes on JDK

Version 4.30

Dr. Herong Yang

Copyright © 1997-2005 by Dr. Herong Yang. All rights reserved.

Table of Contents

About This Book	8
Installing J2SE 1.3.1 on Windows 2000	9
• Download	
• First Java Program	
Installing J2SE 1.4.1 on Windows 2000	10
• Download	
• First Java Program	
• Upgrade to J2SDK 1.4.1_01	
Time and Calendar	12
• Time or Date	
• Calendar and Time Zone	
• Calendar Manipulations	
Formatting and Parsing Dates	18
• Formatting Dates to Strings	
• Parsing Strings for Dates	
Formatting and Parsing Numbers	22
• Formatting Numbers to Strings	
• Parsing Strings for Numbers	
Internationalization	25
• Locale	
• Internationalization on Predefined Classes	
• Internationalization with Resource Bundles	
Unnamed Packages	37
• What is Unnamed Package	
• One Class in an Unnamed Package - Hello.java	
• Two Classes in Unnamed Packages - Hello.java and CallingHello.java	
• Importing Classes Defined in Unnamed Packages	
• Importing Classes Defined in Unnamed Packages Using JDK 1.4.1	
• Importing Classes Defined in Unnamed Packages to Named Packages	
• Importing Classes Defined in Unnamed Packages to Named Packages Using JDK	

1.4.1

Collections	46
• Types of Collections	
• Data Structures of Collection Implementations	
• Collection Implementations in J2SDK	
• Search Operation Performance	
• Data Elements with Natural Order	
Character Set and Encoding	54
• What is a Character Encoding	
• Supported Character Encodings	
• Methods to Encode Characters	
• Methods to Decode Byte Sequences	
Encoding Maps	63
• Encoding Map Analyzer	
• US-ASCII	
• ISO-8859-1 - Latin 1	
• CP1252 - Windows-1252	
• UTF-8	
• UTF-16	
• UTF-16LE	
• UTF-16BE	
Encoding Map Counts	70
• Encoding Map Counter	
• Comparison of Encoding Maps	
Encoding Conversion	73
• Unicode Data Entry	
• Unicode Encoding Conversion	
• Viewing Unicode Text	
• Unicode Signs in Different Encodings	
Socket Communication	81
• What Is a Socket	
• Establishing a Socket Communication	
• ReverseEchoer - A Simple Server Socket Application	
• SocketClient - A Simple Client Socket Application	

• ReverseEchoServer - A Simple Multi-Connection Socket Server	
• Binding Sockets to Local Ports	
Datagram Communication	89
• What Is a Datagram	
• Establishing a Datagram Communication	
• Datagram Socket Client And Server Programs	
Document Object Model (DOM)	95
• What is Document Object Model (DOM)	
• Parsing XML Files with DOM	
• The DOM Tree Structure	
• Building a New DOM Document	
• Converting DOM Documents to XML Files	
Simple API for XML (SAX)	102
• What is Simple API for XML (SAX)	
• SAX Implementations in J2SDK	
• SAX Content Handler Interface	
• My SAX Based XML Browser	
DTD Validation in Java	107
• Validation with DOM	
• Validation with SAX	
XSD Validation in Java	112
• Validation with XMLReader	
• Using SAXParserFactory to Load Parsers	
XSL - Transformer in Java	117
• XSL Implementations in J2SDK	
• A Simple XSL Transformer Program	
JCA - Message Digest	
• What Is Message Digest	
• The MessageDigest Class	
• Message Digest Sample Program - JcaMessageDigest.java	
JCA - Private and Public Key Pairs	124
• What Are Private and Public Key Pairs?	

• The Key Interfaces	
• The KeyPair Class	
• The KeyPairGenerator Class	
• Key Pair Sample Program - JcaKeyPair.java	
JCA - Key Factory - Encoding Tool	132
• What is Key Encoding?	
• java.security.spec.PKCS8EncodedKeySpec Class	
• java.security.spec.X509EncodedKeySpec Class	
• java.security.KeyFactory Class	
• Key Factory Sample Program - JcaKeyFactoryTest.java	
JCA - Digital Signature	138
• What is Digital Signature?	
• The Signing Process and the Verification Process	
• The Signature Class	
• Signature Sample Program - JcaSignatureTest.java	
• Signature Generation Program - JcaSign.java	
• Signature Verification Program - JcaVerify.java	
JCA - Certificates, 'keytool' and 'keystore'	149
• Certificates and Certificate Chains	
• What is "keystore"?	
• "keytool" - Key and Certificate Management Tool	
• "keytool" Example - Generating Key Pairs and Self-Signed Certificates	
• "keytool" Example - Exporting and Import Certificates	
• "keytool" Example - Cloning Certificates with New Identities	
JCA - KeyStore and Related Classes	154
• Certificate Class	
• CertificateFactory Class	
• JcaCertificateTest.java - Sample Program	
• KeyStore Class	
• JcaKeyStoreTest.java - Sample Program	
JCE - Secret Key	161
• What Is a Secret Key?	
• The SecretKey Interface	
• The KeyGenerator Class	

• Converting Secret Keys to and from Byte Arrays	
• JceSecretKeyTest.java - Sample Program	
JCE - Cipher for Encryption and Decryption	166
• The Cipher Class	
• JceSecretCipher.java - Cipher with Secret Key	
• JcePublicCipher.java - Cipher with Private and Public Key Pair	
Installing J2SE 1.5.0 on Windows 2000	172
• Download	
• First Java Program	
SSL (Secure Socket Layer)	173
• What is SSL?	
• SSL Specification Overview	
• How Can SSL Ensure Privacy?	
• JSSE - Java Implementation of SSL and TLS	
• Default SSL Socket Factories - SslSocketTest.java	
• SSL Socket Factories with SSLContext - SslContextTest.java	
SSL - Socket Communication	180
• SSL Server Socket Example - SslReverseEchoer.java	
• SSL Client Socket Example - SslSocketClient.java	
• JSSE Debug Option	
SSL - HTTPS	188
• What is HTTPS?	
• My First HTTPS Server - HttpsHello.java	
• Connecting HttpsHello.java with Internet Explorer	
• A Better HTTPS Server - HttpsEchoer.java	
SSL - Client Authentication	197
• What is SSL Client Authentication	
• Server Program for Client Authentication - SslReverseEchoerRevised	
• Client Program for Client Authentication - SslSocketClientRevised	
• Running Client Authentication Programs	
References	205

Key Words: Blowfish, book, Calendar, certificate, character set, cipher, client authentication, collections, datagram, Date, decryption, DES, digital signature,

encryption, DOM, DSA, DTD, example, HTTP, HTTPS, J2SDK, Java, JCA, JDK, JDK 1.3.1, JDK 1.4.1, JDK 1.5.0, key store, keytool, Locale, notes, message digests, MD5, online, private key, public key, RSA, sample, SAX, secret key, SHA, socket, SSL, TLS, tutorial, unnamed package, Unicode, UTF-8, X.509, XML, XSD, XSL

About This Book

This book is a collection of notes and sample codes I wrote while I was learning the JDK (Java Development Kit), starting in 1997. This collection only includes topics related to JDK packages other than `java.lang.*`. Topics related to the basics of Java language and the `java.lang.*` package will be included in another book: "Herong's Notes on Java".

Revision history:

- Version 4.30, 2005. Added notes on SSL.
- Version 4.00, 2003. Updated and formatted in PDF format.
- Version 3.00, 2002. Updated and formatted in HTML format.
- Version 2.00, 1999. Updated and formatted in Word format.
- Version 1.00, 1997. Collected in text format.

Viewing statistics:

This book has been viewed a total of:

- 26141 times as of March 2005.
- 18517 times as of October 2004.

Copyright:

- The notes and example codes are Copyright © 1997 - 2005 by Dr. Herong Yang. All rights reserved.
- Using and distributing the notes and example codes are permitted, as long as the above copyright notice is retained.
- The example codes is provided as-is, with no warranty of any kind.

Installing J2SE 1.3.1 on Windows 2000

Download

- Go to <http://java.sun.com/j2se/1.3/download.html>
- Click the download button in the SDK column and next to "Windows (all languages)" in the "Download J2SETM v 1.3.1_05" section.
- Follow the instructions on the Web pages to download j2sdk-1_3_1_05-windows-i586.exe to a working directory on your hard disk. This file is about 32 MB.
- Double click on the file name: j2sdk-1_3_1_05-windows-i586.exe in the working directory in the File Explorer.
- Follow the instruction on the installation window to finish the installation.
- Open a command window to try the java command. If you are getting the following output, your installation was ok: returns with message:

```
\jdk1.3.1_05\bin\java -version
java version "1.3.1_05"
Java(TM) 2 Runtime Environment, Standard Edition (build 1.3.1_05)
Java HotSpot(TM) Client VM (build 1.3.1_05, mixed mode)
```

First Java Program

Use NotePad to enter the following Java program into a file called Hello.java:

```
class Hello {
    public static void main(String[] a) {
        System.out.println("Hello world!");
    }
}
```

Then compile this program in a command window with the javac command:

```
\jdk1.3.1_05\bin\javac Hello.java
```

To execute the program, use the java command:

```
\jdk1.3.1_05\bin\java -cp . Hello
Hello world!
```

Congratulations, you have successfully entered, compiled and executed your first Java program.

Installing J2SE 1.4.1 on Windows 2000

Download

- Go to <http://java.sun.com/j2se/1.4.1/download.html>
- Click the download button in the SDK column and next to "Windows (all languages, including English)" in the "Download J2SE(TM) v 1.4.1" section.
- Follow the instructions on the Web pages to download j2sdk-1_4_1-windows-i586.exe to a working directory on your hard disk. This file is about 35 MB.
- Double click on the file name: j2sdk-1_4_1-windows-i586.exe in the working directory in the File Explorer.
- Follow the instruction on the installation window to finish the installation.
- Open a command window to try the java command. If you are getting the following output, your installation was ok: returns with message:

```
\j2sdk1.4.1\bin\java -version
java version "1.4.1"
Java(TM) 2 Runtime Environment, Standard Edition (build 1.4.1-b21)
Java HotSpot(TM) Client VM (build 1.4.1-b21, mixed mode)
```

First Java Program

Use NotePad to enter the following Java program into a file called Hello.java:

```
class Hello {
    public static void main(String[] a) {
        System.out.println("Hello world!");
    }
}
```

Then compile this program in a command window with the javac command:

```
\j2sdk1.4.1\bin\javac Hello.java
```

To execute the program, use the java command:

```
\j2sdk1.4.1\bin\java -cp . Hello
Hello world!
```

Congratulations, you have successfully entered, compiled and executed your first Java program.

Upgrade to J2SDK 1.4.1_01

When a new release of J2SDK is available, you can always go back to the same download Web site, download j2sdk-1_4_1_01-windows-i586.exe, and run it to install. You may use the same command to verify the installation:

```
\j2sdk1.4.1_01\bin\java -version  
java version "1.4.1_01"  
Java(TM) 2 Runtime Environment, Standard Edition (build 1.4.1_01-b01)  
Java HotSpot(TM) Client VM (build 1.4.1_01-b01, mixed mode)
```

Time and Calendar

Sample programs listed in this chapter have been tested with JDK 1.3.1, 1.4.1 and 1.5.0.

Time or Date

Time usually refers to the hour, the minute and may be the second of a specific moment within a day. For example, if someone asks you: "What time is it?", you would answer: "It's nine thirty.", meaning that 9 hours and 30 minutes passed away since the beginning of the day.

Date usually refers to the calendar date of a specific day within a year. For example, if someone ask you: "What date is it today?", you would answer: "Today is October 25.", meaning that today is 21st day of the 10th month of this year.

So both terms: "time" and "date" are measurements of a specific moment. "Time" measures a specific moment with a unit of minute or second, using the beginning of the day as a reference point. "Date" measures a specific moment with a unit of day, using the beginning of the year as a reference point. Notice that these types of measurements are calendar and time zone dependent. At exactly the same moment, the values of "time" and "date" are different from one country to another.

In JDK, we decided to use only one class, the **Date** class (java.util.Date), to measure a specific moment with a unit of millisecond, using a fixed moment of January 1, 1970, 00:00:00 GMT as a reference point.

The Date class measurement is calendar and time zone independent. So if you run the following code at exactly the same moment any where in the world, you will get exactly the same value:

```
Date now = new Date(); // what time is it?
long t = now.getTime();
System.out.println("Time since 01-Jan-1970 00:00:00 GMT: " +
    t + " milliseconds.");
```

Output:

```
Time since 01-Jan-1970 00:00:00 GMT: 1035236526708 milliseconds.
```

The Date class is very convenient for measuring a period of time:

```
long t1 = (new Date()).getTime();
// performing task XYZ
long t2 = (new Date()).getTime();
System.out.println("It took about " + ((t2-t1)/1000) + " seconds to"
    + " finish XYZ.");
```

However, the Date class does not provide any calendar and time zone related information. It requires the help of the Calendar class, see the next section.

The following is what you should remember about date and time in JDK:

- The Date class actually measures time.
- The Date class does not measure calendar date and hour.
- There is no Time class.

Calendar and Time Zone

The Calendar class, `java.util.Calendar`, is an abstract base class, providing a foundation for subclasses to represent a specific instance in time, in a specific calendar system.

The `GregorianCalendar` class, `java.util.GregorianCalendar`, is a concrete subclass of the Calendar class, representing a specific instance in time in Gregorian Calendar system with time zone and daylight saving adjustments.

This abstract base class and concrete subclass structure is very useful for future implementations of other calendar systems, like the Chinese lunar calendar system.

The `TimeZone` class, `java.util.TimeZone`, is an abstract base class, providing a foundation for subclasses to represent a particular time zone system.

The `SimpleTimeZone` class, `java.util.SimpleTimeZone`, is a concrete subclass of the `TimeZone` class, representing a time zone for use with the `GregorianCalendar` class.

I don't know why we need this base class and subclass structure to implement the time zone system. How many time zone systems are there? I only know one.

The following program shows some features of the time related classes:

```
/**
 * DateTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 *
 * This program shows you some basic features of date related classes.
 */
import java.util.*;
class DateTest {
    public static void main(String[] a) {
        showDate();
        showCalendar();
        showTimeZone();
    }
    public static void showDate() {
        Date now = new Date(); // the current time
        long t = now.getTime();
        System.out.println("Time since 01-Jan-1970 00:00:00 GMT: " +
            t + " milliseconds.");
    }
    public static void showCalendar() {
        GregorianCalendar c = new GregorianCalendar();
        // the current time represented in the Gregorian calendar
        // in local time zone and daylight saving adjustments
    }
}
```

```
System.out.println("Year: "+c.get(Calendar.YEAR));
System.out.println("Month: "+c.get(Calendar.MONTH));
System.out.println("Date: "+c.get(Calendar.DATE));
System.out.println("Day of year: "+c.get(Calendar.DAY_OF_YEAR));
System.out.println("Day of week: "+c.get(Calendar.DAY_OF_WEEK));
System.out.println("AM or PM: "+c.get(Calendar.AM_PM));
System.out.println("Hour: "+c.get(Calendar.HOUR));
System.out.println("Hour of day: "+c.get(Calendar.HOUR_OF_DAY));
System.out.println("Minute: "+c.get(Calendar.MINUTE));
System.out.println("Second: "+c.get(Calendar.SECOND));
System.out.println("Millisecond: "+c.get(Calendar.MILLISECOND));
System.out.println("Zone offset: "+
    c.get(Calendar.ZONE_OFFSET)/(60*60*1000));
System.out.println("Daylight saving offset: "+
    c.get(Calendar.DST_OFFSET)/(60*60*1000));
}
public static void showTimeZone() {
    GregorianCalendar c = new GregorianCalendar();
    TimeZone tz = c.getTimeZone();
    System.out.println("My time: "+c.get(Calendar.HOUR_OF_DAY)+
        ":"+c.get(Calendar.MINUTE));
    System.out.println("My time zone ID: "+tz.getID());
    tz = TimeZone.getTimeZone("America/Los_Angeles");
    c.setTimeZone(tz); // changing the time zone
    System.out.println("Los Angeles time: "+
        c.get(Calendar.HOUR_OF_DAY)+":"+c.get(Calendar.MINUTE));
}
}
```

Output:

```
Time since 01-Jan-1970 00:00:00 GMT: 1035236526708 milliseconds.
Year: 2002
Month: 9
Date: 21
Day of year: 294
Day of week: 2
AM or PM: 1
Hour: 5
Hour of day: 17
Minute: 42
Second: 6
Millisecond: 738
Zone offset: -5
Daylight saving offset: 1
My time: 17:42
My time zone ID: America/New_York
Los Angeles time: 14:42
```

The `GregorianCalendar` is a tool that takes three inputs: an instance of time, a time zone system, and daylight saving adjustment, and converts them into calendar related information, such as the month, the date, the day of week, and the hour of day. But in this program, I am using the default constructor, `GregorianCalendar()`, to take the current time, and the time zone and daylight saving adjustment settings on the operating system, where this program was executed.

From the output of the program, you can see that, my computer is set to east coast time zone, with one hour daylight saving offset.

Calendar Manipulations

Objects of Date class are easy to be manipulated, since each of them has only one numerical field in unit of millisecond. For example, to calculate the number of full days between two Date objects, you can use the following expression:

```
(date1.getTime() - date2.getTime()) / (1000*60*60*24)
```

However, objects of Calendar are not so easy to be manipulated, since each of them has many related numerical fields in different calendar units. For example, incrementing the day-in-month field by 1, may affect the month field, if the Calendar object is at the end of a month; then the change in the month field may affect the year field, if the Calendar object is at the end of a year.

To help us manipulate Calendar objects safely, JDK offers the following method:

```
add(int field, int delta)
```

with two rules:

- The new value of the specified field is the old value plus the delta, and modulo any overflow. If overflow occurs, the next higher field will be incremented or decremented.
- If the new value of the specified field is causing changes on the value range of the next lower field, that field needs to be adjusted.

The following program demonstrates how the add() method works:

```
import java.util.*;
class DateOperationTest {
    public static void main(String[] a) {
        GregorianCalendar dt = new GregorianCalendar();
        dt.set(2002,9,31); //31-Oct-2002
        System.out.println("Current time: "+dt.getTime().toString());
        dt.add(Calendar.DATE,1);
        System.out.println("Add 1 to date: "+dt.getTime().toString());
        dt.set(2002,9,31); //31-Oct-2002
        System.out.println("Current time: "+dt.getTime().toString());
        dt.roll(Calendar.DATE,1);
        System.out.println("Roll 1 to date: "+dt.getTime().toString());
        dt.set(2002,9,31); //31-Oct-2002
        System.out.println("Current time: "+dt.getTime().toString());
        dt.add(Calendar.MONTH,1);
        System.out.println("Add 1 to month: "+dt.getTime().toString());

        GregorianCalendar dt1 = new GregorianCalendar();
        GregorianCalendar dt2 = new GregorianCalendar();
        dt1.set(2002,9,31,0,0,0);
        dt2.set(2002,10,1,0,0,0);
        System.out.println("    From: "+dt1.getTime().toString());
        System.out.println("    To: "+dt2.getTime().toString());
        long days = fullDayDiff(dt1,dt2);
        System.out.println("Full days: "+days);

        dt1.set(2002,9,31,0,0,0);
```

```

        dt2.set(2002,10,30,0,0,0);
        System.out.println("        From: "+dt1.getTime().toString());
        System.out.println("        To: "+dt2.getTime().toString());
        long months = fullMonthDiff(dt1,dt2);
        System.out.println("Full months: "+months);

        dt1.set(2002,9,30,0,0,0);
        dt2.set(2003,10,29,0,0,0);
        System.out.println("        From: "+dt1.getTime().toString());
        System.out.println("        To: "+dt2.getTime().toString());
        months = fullMonthDiff(dt1,dt2);
        System.out.println("Full months: "+months);
    }
    public static long fullDayDiff(Calendar dt1, Calendar dt2) {
        long tm1 = dt1.getTime().getTime();
        long tm2 = dt2.getTime().getTime();
        return (tm2-tm1)/(1000*60*60*24);
    }
    public static long fullMonthDiff(Calendar dt1, Calendar dt2) {
        // Both dates are truncated to the beginning of the day
        // dt1 and dt2 are damaged
        // 30-Oct-2002 to 30-Nov-2002: 1 month
        // 31-Oct-2002 to 30-Nov-2002: 1 month
        // 30-Oct-2002 to 29-Nov-2002: < 1 month
        Calendar a, b;
        if (dt1.before(dt2)) {
            a = dt1;
            b = dt2;
        } else {
            a = dt2;
            b = dt1;
        }

        // work on the years
        int y1 = a.get(Calendar.YEAR);
        int y2 = b.get(Calendar.YEAR);
        int m1 = a.get(Calendar.MONTH);
        int m2 = b.get(Calendar.MONTH);
        int l = y2 - y1;
        if (m2 < m1) l--;
        int diff = 0;
        if (l > 0) {
            diff += l*12;
            a.add(Calendar.YEAR, l);
        }

        // work on the months
        y1 = a.get(Calendar.YEAR);
        y2 = b.get(Calendar.YEAR);
        m1 = a.get(Calendar.MONTH);
        m2 = b.get(Calendar.MONTH);
        if (m2 < m1) {
            l = m2 + 12 - m1;
        } else {
            l = m2 - m1;
        }
        l = m2 - m1;
        if (l > 0) {
            diff += l;
            a.add(Calendar.MONTH, 1);
        }
    }

```



```
// date adjustment
int d1 = a.get(Calendar.DATE);
int d2 = b.get(Calendar.DATE);
if (d2 < d1) diff--;

return diff;
}
```

Output:

```
Current time: Thu Oct 31 19:02:19 EST 2002
Add 1 to date: Fri Nov 01 19:02:19 EST 2002
Current time: Thu Oct 31 19:02:19 EST 2002
Roll 1 to date: Tue Oct 01 19:02:19 EDT 2002
Current time: Thu Oct 31 19:02:19 EST 2002
Add 1 to month: Sat Nov 30 19:02:19 EST 2002
    From: Thu Oct 31 00:00:00 EST 2002
    To: Fri Nov 01 00:00:00 EST 2002
Full days: 1
    From: Thu Oct 31 00:00:00 EST 2002
    To: Sat Nov 30 00:00:00 EST 2002
Full months: 1
    From: Wed Oct 30 00:00:00 EST 2002
    To: Sat Nov 29 00:00:00 EST 2003
Full months: 12
```

Note that I have only done few tests on the `fullMonthDiff()` method. So don't take it to your program directly. Do more tests yourself.

Exercise: Write a program to calculate the annually compound interest of an investment with the following method signature:

```
public static double getInterest(double capital,
    double interest_rate, Calendar start_date, Calendar end_date)
```

Formatting and Parsing Dates

Sample programs listed in this chapter have been tested with JDK 1.3.1, 1.4.1 and 1.5.0.

Formatting Dates to Strings

The date and calendar classes manage date, time and calendar related information in numeric values. JDK provides two classes to convert those values into text in different languages and styles.

The `DateFormat` class, `java.text.DateFormat`, is an abstract class providing a foundation for subclasses to present time in a specific calendar in text, and parsing text back to time.

The `SimpleDateFormat` class, `java.text.SimpleDateFormat`, is concrete subclass of the `DateFormat` class. It can be customized with different formatting patterns and in different locales.

Once a `DateFormat` object is created with a predefined pattern or a customized pattern, it can be used to convert a `Date` object into a string based on the pattern using the `format()` method. The following program shows how to create `DateFormat` objects and how to use them:

```
import java.util.*;
import java.text.*;
class DateFormatTest {
    public static void main(String[] a) {
        showDateFormat();
        showSimpleDateFormat();
    }
    public static void showDateFormat() {
        Date now = new Date(); // the current time
        System.out.println("Date toString() format: " + now.toString());
        DateFormat df = DateFormat.getInstance();
        System.out.println("Default DateFormat: " + df.format(now));
        df.setTimeZone(TimeZone.getTimeZone("America/Los_Angeles"));
        System.out.println("Default DateFormat in west coast: "
            + df.format(now));
        df = DateFormat.getDateInstance(DateFormat.SHORT,
            DateFormat.SHORT, Locale.US);
        System.out.println("Short date time in US: " + df.format(now));
        df = DateFormat.getDateInstance(DateFormat.MEDIUM,
            DateFormat.MEDIUM, Locale.US);
        System.out.println("Medium date time in US: " + df.format(now));
        df = DateFormat.getDateInstance(DateFormat.SHORT,
            DateFormat.SHORT, Locale.FRENCH);
        System.out.println("Short date time in French: "
            + df.format(now));
        df = DateFormat.getDateInstance(DateFormat.LONG,
            DateFormat.LONG, Locale.FRENCH);
        System.out.println("Long date time in French: "+df.format(now));
    }
    public static void showSimpleDateFormat() {
        Date now = new Date(); // the current time
```

```
System.out.println("Date toString() format: " + now.toString());
SimpleDateFormat sdf = new SimpleDateFormat();
System.out.println("Default SimpleDateFormat: " + sdf.format(now));
sdf = new SimpleDateFormat("dd-MMM-yyyy");
System.out.println("The FDA standard format: " + sdf.format(now));
sdf = new SimpleDateFormat("EEE MMM dd HH:mm:ss zzz yyyy");
System.out.println("Date.toString() format: " + sdf.format(now));
sdf = new SimpleDateFormat("EEE MMM dd HH:mm:ss zzz yyyy",
    Locale.FRENCH);
System.out.println("Date.toString() format in French: "
    + sdf.format(now));
}
```

Output:

```
Date toString() format: Mon Oct 21 20:54:05 EDT 2002
Default SimpleDateFormat: 10/21/02 8:54 PM
Default SimpleDateFormat in west coast: 10/21/02 5:54 PM
Short date time in US: 10/21/02 8:54 PM
Medium date time in US: Oct 21, 2002 8:54:05 PM
Short date time in French: 21/10/02 20:54
Long date time in French: 21 octobre 2002 20:54:05 EDT
Date toString() format: Mon Oct 21 20:54:05 EDT 2002
Default SimpleDateFormat: 10/21/02 8:54 PM
The FDA standard format: 21-Oct-2002
Date.toString() format: Mon Oct 21 20:54:05 EDT 2002
Date.toString() format in French: lun. oct. 21 20:54:05 EDT 2002
```

Note that the `DateFormat` class is providing predefined formats for a number of commonly used styles in different locales. The predefined formats are accessible by the `getInstance()` methods.

Note also that the `DateFormat` class uses `GregorianCalendar` as the default calendar.

Parsing Strings for Dates

`DateFormat` objects can also be used to convert string back to `Date` objects using the `parse()` method, see the following demonstration program:

```
import java.util.*;
import java.text.*;
class DateParsingTest {
    public static void main(String[] a) {
        parseDate();
        parseDateSpecial();
    }
    public static void parseDate() {
        SimpleDateFormat mf = new SimpleDateFormat("MMM-yyyy");
        SimpleDateFormat tf = new SimpleDateFormat("HH:mm:ss.SSS");
        SimpleDateFormat df = new SimpleDateFormat("dd-MMM-yyyy");
        SimpleDateFormat dtf
            = new SimpleDateFormat("dd-MMM-yyyy HH:mm:ss.SSS");

        String ms = "Oct-2002";
        String ts = "22:13:19.123";
        String ds = "21-Oct-2002";
        String dts = "21-Oct-2002 22:13:19.123";
```

```

    try {
        System.out.println("Parsing " + ms + " with " + mf.toPattern());
        System.out.println("    " + mf.parse(ms));
        System.out.println("Parsing " + ts + " with " + tf.toPattern());
        System.out.println("    " + tf.parse(ts));
        System.out.println("Parsing " + ds + " with " + df.toPattern());
        System.out.println("    " + df.parse(ds));
        System.out.println("Parsing " + dts + " with " +
            dtf.toPattern());
        System.out.println("    " + dtf.parse(dts));
        System.out.println("Parsing " + dts + " with " + df.toPattern());
        System.out.println("    " + df.parse(dts));
        System.out.println("Parsing " + ds + " with " + dtf.toPattern());
        System.out.println("    " + dtf.parse(ds));
    } catch (ParseException e) {
        System.out.println("Exception: " + e.toString());
    }
}

public static void parseDateSpecial() {
    SimpleDateFormat dtf
        = new SimpleDateFormat("EEE MMM dd HH:mm:ss zzz yyyy");
    String str0 = "Mon Oct 21 20:54:05 EDT 2002"; // correct
    String str1 = "Mon Oct 20 20:54:05 EDT 2002"; // wrong d.o.w.
    String str2 = "Mon Oct 21 25:54:05 EDT 2002"; // wrong hour
    String str3 = "Mon Oct 21 20:54:05 PDT 2002"; // time zone changed
    String str4 = "Mon Oct 1 20:54:05 EDT 2002"; // missing a 'd'
    try {
        System.out.println("Parsing " + str0 + " with "
            + dtf.toPattern());
        System.out.println("    " + dtf.parse(str0));
        System.out.println("Parsing " + str1 + " with "
            + dtf.toPattern());
        System.out.println("    " + dtf.parse(str1));
        System.out.println("Parsing " + str2 + " with "
            + dtf.toPattern());
        System.out.println("    " + dtf.parse(str2));
        System.out.println("Parsing " + str3 + " with "
            + dtf.toPattern());
        System.out.println("    " + dtf.parse(str3));
        System.out.println("Parsing " + str4 + " with "
            + dtf.toPattern());
        System.out.println("    " + dtf.parse(str4));
    } catch (ParseException e) {
        System.out.println("Exception: " + e.toString());
    }
}
}

```

Output:

```

Parsing Oct-2002 with MMM-yyyy
Tue Oct 01 00:00:00 EDT 2002
Parsing 22:13:19.123 with HH:mm:ss.SSS
Thu Jan 01 22:13:19 EST 1970
Parsing 21-Oct-2002 with dd-MMM-yyyy
Mon Oct 21 00:00:00 EDT 2002
Parsing 21-Oct-2002 22:13:19.123 with dd-MMM-yyyy HH:mm:ss.SSS
Mon Oct 21 22:13:19 EDT 2002
Parsing 21-Oct-2002 22:13:19.123 with dd-MMM-yyyy
Mon Oct 21 00:00:00 EDT 2002
Parsing 21-Oct-2002 with dd-MMM-yyyy HH:mm:ss.SSS

```

```
Exception: java.text.ParseException: Unparseable date: "21-Oct-2002"
Parsing Mon Oct 21 20:54:05 EDT 2002 with EEE MMM dd HH:mm:ss zzz yyyy
Mon Oct 21 20:54:05 EDT 2002
Parsing Mon Oct 20 20:54:05 EDT 2002 with EEE MMM dd HH:mm:ss zzz yyyy
Sun Oct 20 20:54:05 EDT 2002
Parsing Mon Oct 21 25:54:05 EDT 2002 with EEE MMM dd HH:mm:ss zzz yyyy
Tue Oct 22 01:54:05 EDT 2002
Parsing Mon Oct 21 20:54:05 PDT 2002 with EEE MMM dd HH:mm:ss zzz yyyy
Mon Oct 21 23:54:05 EDT 2002
Parsing Mon Oct 1 20:54:05 EDT 2002 with EEE MMM dd HH:mm:ss zzz yyyy
Tue Oct 01 20:54:05 EDT 2002
```

Examining the output carefully, you will see each parsing example shows a different feature of the `parse()` method:

- If a field is missing, it will be set to its lowermost value.
- If the year field is missing, it will be set to 1970.
- If time fields are missing, they will be set to 0.
- If time zone is missing, the local time zone will be used.
- If the input string is longer than the pattern, the remaining part of the string will be ignored.
- If the input string is not matching the pattern, you will get an exception.
- Long pattern is not a problem.
- Day of month overrides day of week.
- An overflow lower field will change the higher field. Hour 25 gives hour 1 plus 1 day.
- Specified time zone overrides local time zone.
- Single digit day of month matches double digits pattern.

Formatting and Parsing Numbers

Sample programs listed in this chapter have been tested with JDK 1.3.1, 1.4.1 and 1.5.0.

Formatting Numbers to Strings

The **Number** class, `java.lang.Number`, is an abstract class representing a numerical value. It has several concrete subclasses: `Byte`, `Short`, `Integer`, `Long`, `Float`, and `Double`.

The **NumberFormat** class, `java.text.NumberFormat`, is an abstract class providing a foundation for the derived subclasses to format numbers to strings and parse strings back to numbers.

The **DecimalFormat** class, `java.text.DecimalFormat`, is a concrete subclass of `NumberFormat`. An object of `DecimalFormat` contains a formatting pattern and locale information. It can be used to format numbers into string representations; or parsing strings for numbers.

There are some factory methods in `NumberFormat` class that return predefined default `DecimalFormat` objects with commonly used formatting patterns. For example, `NumberFormat.getCurrencyInstance()` returns a `DecimalFormat` object with a pattern of `"\u00A4#,###0.00;\u00A4#,###0.00"` good for formatting a number into a currency representation.

The following programs shows some examples of how to format numbers to strings:

```
import java.util.*;
import java.text.*;
class NumberFormatTest {
    public static void main(String[] a) {
        testDefault();
        myFormats();
    }
    public static void testDefault() {
        NumberFormat nf;
        double x = -1234.5678;
        nf = NumberFormat.getInstance();
        System.out.println("Default format: "
            + ((DecimalFormat)nf).toPattern() + " -> " + nf.format(x));
        nf = NumberFormat.getNumberInstance();
        System.out.println("Default number format: "
            + ((DecimalFormat)nf).toPattern() + " -> " + nf.format(x));
        nf = NumberFormat.getCurrencyInstance();
        System.out.println("Default currency format: "
            + ((DecimalFormat)nf).toPattern() + " -> " + nf.format(x));
        nf = NumberFormat.getPercentInstance();
        System.out.println("Default percent format: "
            + ((DecimalFormat)nf).toPattern() + " -> " + nf.format(x));
    }
    public static void myFormats() {
        double myNumber = -1234.5678;
```

```
// getting a formatter for default locale
DecimalFormat df = new DecimalFormat("#,##0.00");
System.out.println("Pattern: " + df.toPattern()
    + " -> " + df.format(myNumber));

// locale sensitive currency symbole
df = new DecimalFormat("\u00A4#,##0.00");
System.out.println("Pattern: " + df.toPattern()
    + " -> " + df.format(myNumber));

// speical negative subpattern
df = new DecimalFormat("\u00A4#,##0.00;(\u00A4#,##0.00)");
System.out.println("Pattern: " + df.toPattern()
    + " -> " + df.format(myNumber));

// percentage
df = new DecimalFormat("#,##0.00%");
System.out.println("Pattern: " + df.toPattern()
    + " -> " + df.format(myNumber));

// locale sensitive currency symbole
Locale.setDefault(Locale.JAPAN);
df = new DecimalFormat("\u00A4#,##0.00");
System.out.println("Pattern (Japan): " + df.toPattern()
    + " -> " + df.format(myNumber));
Locale.setDefault(Locale.JAPAN);
}
}
```

Output:

```
Default format: #,##0.### -> -1,234.568
Default number format: #,##0.### -> -1,234.568
Default currency format: ¤#,##0.00;(\u00A4#,##0.00) -> ($1,234.57)
Default percent format: #,##0% -> -123,457%
Pattern: #,##0.00 -> -1,234.57
Pattern: ¤#,##0.00 -> -$1,234.57
Pattern: ¤#,##0.00;(\u00A4#,##0.00) -> ($1,234.57)
Pattern: #,##0.00% -> -123,456.78%
Pattern (Japan): ¤#,##0.00 -> -¥1,234.57
```

Note that:

- The factory methods `getInstance()` and `getNumberInstance()` return `DecimalFormat` objects with the same formatting pattern.
- Rounding is used to remove the extra digits in the fraction part.
- If leading extra digits in the integer part will show up in the output string. No error, or truncation, will be given.
- The percent sign in pattern will convert the number by multiplying with 100.
- The locale sensitive currency character will be replaced by the specific currency sign of country setting in the default locale.

Parsing Strings for Numbers

`DecimalFormat` objects can also be used to parse strings for numbers. Here is a demonstration program:

```
import java.util.*;
import java.text.*;
class NumberParsingTest {
    public static void main(String[] a) {
        parseNumbers();
    }
    public static void parseNumbers() {
        DecimalFormat nf = new DecimalFormat("#,###0.00");
        String ns = "-1234.5678";
        Number nv = null;
        DecimalFormat cf = new DecimalFormat("\u00A4#,###0.00");
        String cs = "-$1234.5678";
        Number cv = null;
        DecimalFormat xf
            = new DecimalFormat("\u00A4#,###0.00;(\u00A4#,###0.00)");
        String xs = "($1234.5678)";
        Number xv = null;
        DecimalFormat pf = new DecimalFormat("#,###0.00%");
        String ps = "-1234.5678%";
        Number pv = null;
        try {
            nv = nf.parse(ns);
            cv = cf.parse(cs);
            xv = xf.parse(xs);
            pv = pf.parse(ps);
        } catch (ParseException e) {
            System.out.println("Exception: " + e.toString());
        }
        System.out.println("Parsing " + ns + " with " + nf.toPattern());
        System.out.println("    " + nv.doubleValue());
        System.out.println("Parsing " + cs + " with " + cf.toPattern());
        System.out.println("    " + cv.doubleValue());
        System.out.println("Parsing " + xs + " with " + xf.toPattern());
        System.out.println("    " + xv.doubleValue());
        System.out.println("Parsing " + ps + " with " + pf.toPattern());
        System.out.println("    " + pv.doubleValue());
    }
}
```

Output:

```
Parsing -51,234.5678 with #,###0.00
-51234.5678
Parsing -$51234.5678 with ñ#,###0.00
-51234.5678
Parsing ($51234.5678) with ñ#,###0.00;(\ñ#,###0.00)
-51234.5678
Parsing -51234.5678% with #,###0.00%
-512.345678
```

The result is very interesting:

- The grouping separator "," is not required in the input string.
- The number digits in the fraction part is not truly respected.

Internationalization

Sample programs listed in this chapter have been tested with JDK 1.3.1, 1.4.1 and 1.5.0.

Locale

Internationalization: An process that tailors information for the user based on his/her specific geographical, political, or cultural preference. For example, each time when I insert my Fleet Bank client card into a Fleet Bank ATM machine, it will greet me and display instructions in Chinese. This process of check my language preference setting and tailoring the display information my preferred language is called internationalization.

The **Locale** class, `java.util.Locale`, is class presenting a specific geographical, political, or cultural region, where information is displayed in a way different than other regions. For example, French speaking areas in Canada is a locale, and English speaking areas in Canada is another locale.

Major methods offered on Locale class:

- `getDefault()`: Returns the default locale of the JVM.
- `getLanguage()`: Returns the language code of this locale. Language codes are the lower-case two-letter codes as defined by ISO-639.
- `getCountry()`: Returns the country code of this locale. Country codes are the upper-case two-letter codes as defined by ISO-3166.
- `Locale(l,c)`: Constructs a new locale with a given language and a given country.

The following program shows some interesting features of the Locale class:

```
/**
 * LocaleTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
class LocaleTest {
    public static void main(String[] a) {
        // get the default locale
        Locale l = Locale.getDefault();
        System.out.println("    Language, Country, Variant, Name");
        System.out.println("");
        System.out.println("Default locale: ");
        System.out.println("    "+l.getLanguage()+" "+l.getCountry()+" "+
            "+ "+l.getVariant()+" "+l.getDisplayName());
        // get a predefined locale
        l = Locale.CANADA_FRENCH;
        System.out.println("A predefined locale - Locale.CANADA_FRENCH:");
        System.out.println("    "+l.getLanguage()+" "+l.getCountry()+" "+
            "+ "+l.getVariant()+" "+l.getDisplayName());
        // define a new locale
        l = new Locale("en", "CN");
        System.out.println("User defined locale -Locale(\"en\",\"CN\"):");
        System.out.println("    "+l.getLanguage()+" "+l.getCountry()+" "+
```

```

        +", "+l.getVariant()+", "+l.getDisplayName());
// define another new locale
l = new Locale("ll", "CC");
System.out.println("User defined locale -Locale(\"ll\", \"CC\"):");
System.out.println("    "+l.getLanguage()+", "+l.getCountry()+", "
    +", "+l.getVariant()+", "+l.getDisplayName());
// get the supported locales
Locale[] s = Locale.getAvailableLocales();
System.out.println("Supported locales: ");
for (int i=0; i<s.length; i++) {
    System.out.println("    "+s[i].getLanguage()+", "
        +s[i].getCountry()+", "+s[i].getVariant()+", "
        +s[i].getDisplayName());
    }
}
}

```

Output:

```

    Language, Country, Variant, Name
Default locale:
    en, US, , , English (United States)
A predefined locale - Locale.CANADA_FRENCH:
    fr, CA, , , French (Canada)
User defined locale - Locale("en", "CN"):
    en, CN, , , English (China)
User defined locale - Locale("ll", "CC"):
    ll, CC, , , ll (CC)
Supported locales:
    ar, , , Arabic
    ar, AE, , , Arabic (United Arab Emirates)
    ar, BH, , , Arabic (Bahrain)
    ar, DZ, , , Arabic (Algeria)
    ar, EG, , , Arabic (Egypt)
    ar, IQ, , , Arabic (Iraq)
    ar, JO, , , Arabic (Jordan)
    ar, KW, , , Arabic (Kuwait)
    ar, LB, , , Arabic (Lebanon)
    ar, LY, , , Arabic (Libya)
    ar, MA, , , Arabic (Morocco)
    ar, OM, , , Arabic (Oman)
    ar, QA, , , Arabic (Qatar)
    ar, SA, , , Arabic (Saudi Arabia)
    ar, SD, , , Arabic (Sudan)
    ar, SY, , , Arabic (Syria)
    ar, TN, , , Arabic (Tunisia)
    ar, YE, , , Arabic (Yemen)
    be, , , Byelorussian
    be, BY, , , Byelorussian (Belarus)
    bg, , , Bulgarian
    bg, BG, , , Bulgarian (Bulgaria)
    ca, , , Catalan
    ca, ES, , , Catalan (Spain)
    cs, , , Czech
    cs, CZ, , , Czech (Czech Republic)
    da, , , Danish
    da, DK, , , Danish (Denmark)
    de, , , German
    de, AT, , , German (Austria)
    de, CH, , , German (Switzerland)

```

```

de, DE, , German (Germany)
de, LU, , German (Luxembourg)
el, , , Greek
el, GR, , Greek (Greece)
en, AU, , English (Australia)
en, CA, , English (Canada)
en, GB, , English (United Kingdom)
en, IE, , English (Ireland)
en, IN, , English (India)
en, NZ, , English (New Zealand)
en, ZA, , English (South Africa)
es, , , Spanish
es, AR, , Spanish (Argentina)
es, BO, , Spanish (Bolivia)
es, CL, , Spanish (Chile)
es, CO, , Spanish (Colombia)
es, CR, , Spanish (Costa Rica)
es, DO, , Spanish (Dominican Republic)
es, EC, , Spanish (Ecuador)
es, ES, , Spanish (Spain)
es, GT, , Spanish (Guatemala)
es, HN, , Spanish (Honduras)
es, MX, , Spanish (Mexico)
es, NI, , Spanish (Nicaragua)
es, PA, , Spanish (Panama)
es, PE, , Spanish (Peru)
es, PR, , Spanish (Puerto Rico)
es, PY, , Spanish (Paraguay)
es, SV, , Spanish (El Salvador)
es, UY, , Spanish (Uruguay)
es, VE, , Spanish (Venezuela)
et, , , Estonian
et, EE, , Estonian (Estonia)
fi, , , Finnish
fi, FI, , Finnish (Finland)
fr, , , French
fr, BE, , French (Belgium)
fr, CA, , French (Canada)
fr, CH, , French (Switzerland)
fr, FR, , French (France)
fr, LU, , French (Luxembourg)
hi, IN, , Hindi (India)
hr, , , Croatian
hr, HR, , Croatian (Croatia)
hu, , , Hungarian
hu, HU, , Hungarian (Hungary)
is, , , Icelandic
is, IS, , Icelandic (Iceland)
it, , , Italian
it, CH, , Italian (Switzerland)
it, IT, , Italian (Italy)
iw, , , Hebrew
iw, IL, , Hebrew (Israel)
ja, , , Japanese
ja, JP, , Japanese (Japan)
ko, , , Korean
ko, KR, , Korean (South Korea)
lt, , , Lithuanian
lt, LT, , Lithuanian (Lithuania)
lv, , , Latvian (Lettish)
lv, LV, , Latvian (Lettish) (Latvia)
mk, , , Macedonian

```

```

mk, MK, , Macedonian (Macedonia)
nl, , , Dutch
nl, BE, , Dutch (Belgium)
nl, NL, , Dutch (Netherlands)
no, , , Norwegian
no, NO, , Norwegian (Norway)
no, NO, NY, Norwegian (Norway, Nynorsk)
pl, , , Polish
pl, PL, , Polish (Poland)
pt, , , Portuguese
pt, BR, , Portuguese (Brazil)
pt, PT, , Portuguese (Portugal)
ro, , , Romanian
ro, RO, , Romanian (Romania)
ru, , , Russian
ru, RU, , Russian (Russia)
sh, , , Serbo-Croatian
sh, YU, , Serbo-Croatian (Yugoslavia)
sk, , , Slovak
sk, SK, , Slovak (Slovakia)
sl, , , Slovenian
sl, SI, , Slovenian (Slovenia)
sq, , , Albanian
sq, AL, , Albanian (Albania)
sr, , , Serbian
sr, YU, , Serbian (Yugoslavia)
sv, , , Swedish
sv, SE, , Swedish (Sweden)
th, , , Thai
th, TH, , Thai (Thailand)
th, TH, TH, Thai (Thailand, TH)
tr, , , Turkish
tr, TR, , Turkish (Turkey)
uk, , , Ukrainian
uk, UA, , Ukrainian (Ukraine)
zh, , , Chinese
zh, CN, , Chinese (China)
zh, HK, , Chinese (Hong Kong)
zh, TW, , Chinese (Taiwan)
en, , , English
en, US, , English (United States)

```

Sorry about the length of the output. But it might be useful as a reference source.

The output shows that I can make up a new locale with any combination of language and country, like "en, CN, , , English (China)", or "ll, CC, , , ll (CC)".

Internationalization on Predefined Classes

Locale Sensitive Operations: Operations that require to know a user's locale in order to tailor information for the user. For example, formatting a calendar date into a string is a locale sensitive operation, because for an English speaking user in US, a calendar date should be formatted as "Mon Oct 21 20:54:05 EDT 2002"; but for a French speaking user, the same date should be formatted as "lun. oct. 21 20:54:05 EDT 2002".

Separated formatter classes are provided to handle locale sensitive operations related to some predefined classes. Here is a list of formatter classes:

Formatter Class	Data Class	Operations
SimpleDateFormat	GregorianCalendar	Formatting and parsing
DecimalFormat	Number	Formatting and parsing

The following program shows how to internationalize the formatting operation of dates, numbers and currencies:

```
/**
 * FormatLocaleTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
import java.text.*;
class FormatLocaleTest {
    public static void main(String[] a) {
        showDateFormat();
        showNumberFormat();
    }
    public static void showDateFormat() {
        Date now = new Date(); // the current time
        DateFormat df;
        SimpleDateFormat sdf;
        System.out.println("Default date format:");
        df = DateFormat.getDateInstance(DateFormat.LONG,
            DateFormat.LONG);
        System.out.println("    Locale(Default): " + df.format(now));
        df = DateFormat.getDateInstance(DateFormat.LONG,
            DateFormat.LONG, Locale.FRANCE);
        System.out.println("    Locale(France): " + df.format(now));
        df = DateFormat.getDateInstance(DateFormat.LONG,
            DateFormat.LONG, Locale.FRENCH);
        System.out.println("    Locale(French): " + df.format(now));
        df = DateFormat.getDateInstance(DateFormat.LONG,
            DateFormat.LONG, Locale.GERMAN);
        System.out.println("    Locale(German): " + df.format(now));
        System.out.println("Customized date format:");
        sdf = new SimpleDateFormat("EEE MMM dd HH:mm:ss zzz yyyy");
        System.out.println("    Locale(Default): " + sdf.format(now));
        sdf = new SimpleDateFormat("EEE MMM dd HH:mm:ss zzz yyyy",
            Locale.FRENCH);
        System.out.println("    Locale(French): " + sdf.format(now));
        sdf = new SimpleDateFormat("EEE MMM dd HH:mm:ss zzz yyyy",
            Locale.GERMAN);
        System.out.println("    Locale(German): " + sdf.format(now));
        sdf = new SimpleDateFormat("EEE MMM dd HH:mm:ss zzz yyyy",
            Locale.CHINESE);
        System.out.println("    Locale(Chinese): " + sdf.format(now));
    }
    public static void showNumberFormat() {
        double num = -1234.5678;
        NumberFormat nf;
        System.out.println("Number format:");
        nf = NumberFormat.getNumberInstance();
        System.out.println("    Locale(Default): "+ nf.format(num));
        nf = NumberFormat.getNumberInstance(Locale.FRANCE);
        System.out.println("    Locale(France): "+ nf.format(num));
        nf = NumberFormat.getNumberInstance(Locale.FRENCH);
        System.out.println("    Locale(French): "+ nf.format(num));
    }
}
```

```

        nf = NumberFormat.getNumberInstance(Locale.JAPAN);
        System.out.println("    Locale(Japanese): "+ nf.format(num));

        System.out.println("Currency format:");
        nf = NumberFormat.getCurrencyInstance();
        System.out.println("    Locale(Default): "+ nf.format(num));
        nf = NumberFormat.getCurrencyInstance(Locale.FRANCE);
        System.out.println("    Locale(France): "+ nf.format(num));
        nf = NumberFormat.getCurrencyInstance(Locale.FRENCH);
        System.out.println("    Locale(French): "+ nf.format(num));
        nf = NumberFormat.getCurrencyInstance(Locale.JAPAN);
        System.out.println("    Locale(Japanese): "+ nf.format(num));
    }
}

```

Output:

```

Default date format:
  Locale(Default): November 1, 2002 9:45:44 AM EST
  Locale(France): 1 novembre 2002 09:45:44 EST
  Locale(French): 1 novembre 2002 09:45:44 EST
  Locale(German): 1. November 2002 09:45:44 EST
Customized date format:
  Locale(Default): Fri Nov 01 09:45:44 EST 2002
  Locale(French): ven. nov. 01 09:45:44 EST 2002
  Locale(German): Fr Nov 01 09:45:44 EST 2002
  Locale(Chinese): ??? ??? 01 09:45:44 EST 2002
Number format:
  Locale(Default): -1,234.568
  Locale(France): -1â234,568
  Locale(French): -1â234,568
  Locale(Japanese): -1,234.568
Currency format:
  Locale(Default): ($1,234.57)
  Locale(France): -1â234,57 F
  Locale(French): -1â234,57 ñ
  Locale(Japanese): -?1,235

```

Note that:

- When the locale is changed, the default formatter not only changes the language, but also changes the pattern.
- The "?" in the output of Chinese locale is caused by the console window not able to display the Chinese characters.
- The currency formatter is sensitive to not only the language, but also to the country.
- The precision is also adjusted in the default currency formatter for Japanese.

Internationalization with Resource Bundles

The **ResourceBundle** class, `java.util.ResourceBundle`, is an abstract class providing a foundation for subclasses to represent a specific map of resource keys and the associated localized resource objects.

Resource bundles are commonly used to internationalize the text messages in an application, where a key name is assigned to each individual text message. There are 3

ways to map the keys to the localized text messages:

- Implement a series of subclasses of ResourceBundle class
- Implement a series of subclasses of ListResourceBundle class
- Implement a series of property files, and let the system to automatically create objects of PropertyResourceBundle class

The following sample classes and demo program illustrate those different ways.

Creating Resource Bundle with ResourceBundle Classes: ErrorMessage

Mapping ErrorMessage keys to the default locale:

```
/**
 * ErrorMessage.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
public class ErrorMessage extends ResourceBundle {
    public Object handleGetObject(String key) {
        if (key.equals("NO_FILE")) return "File doesn't exist.";
        if (key.equals("FILE_EMPTY")) return "File is empty.";
        return null;
    }
    public Enumeration getKeys() {
        Vector keys = new Vector();
        keys.addElement("NO_FILE");
        keys.addElement("FILE_EMPTY");
        return keys.elements();
    }
}
```

Note that:

- Only the two required methods handleGetObject() and getKeys() are implemented. They are declared as abstract in the ResourceBundle class.

Mapping ErrorMessage keys to the fr locale:

```
/**
 * ErrorMessage_fr.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
public class ErrorMessage_fr extends ErrorMessage {
    public Object handleGetObject(String key) {
        if (key.equals("NO_FILE")) return "Document n'existe pas.";
        if (key.equals("FILE_EMPTY")) return "Rien.";
        return null;
    }
}
```

Note that:

- I am extending ErrorMessage_fr from ErrorMessage, so I don't have to repeat the

handleGetObject() method.

- The resource bundle name is the class name of the default locale. The class names for other locales must have a suffix of language code.

Mapping ErrorMessage keys to the fr and CA locale:

```
/**
 * ErrorMessage_fr_CA.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
public class ErrorMessage_fr_CA extends ErrorMessage {
    public Object handleGetObject(String key) {
        if (key.equals("NO_FILE")) return "Dossier n'existe pas.";
        return null;
    }
}
```

Note that:

- The class names have the same hierarchical structure as the locales. Default locale is at the top level; language specific locales are at the middle level; language and country specific locales are at the bottom level.
- For a given key and a given locale, the lookup mechanism searches in the class that matches the locale first. If failed, it will continue with the class at the parent level. In my example, for the key, "FILE_EMPTY", and the locale, "fr and CA", the lookup mechanism will search the ErrorMessage_fr_CA class first, then the ErrorMessage_fr class, then the ErrorMessage class. So the resulting object will be the String, "Rien." from the ErrorMessage_fr class.

Creating Resource Bundle with ListResourceBundle Classes: WarningMessage

Mapping WarningMessage keys to the default locale:

```
/**
 * WarningMessage.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
public class WarningMessage extends ListResourceBundle {
    public Object[][] getContents() {
        return contents;
    }
    static final Object[][] contents = {
        {"LAST_MINUTE", "Bus is leaving within one minute."},
        {"HIGH_LOAD", "System load is very high."}
    };
}
```

Note that:

- The two required methods handleGetObject() and getKeys() are already defined in the ListResourceBundle class. We only need implement a different required method getContents(), which is much easier to understand and to implement.

Mapping WarningMessage keys to the fr locale:

```
/**
 * WarningMessage_fr.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
public class WarningMessage_fr extends ListResourceBundle {
    public Object[][] getContents() {
        return contents;
    }
    static final Object[][] contents = {
        {"LAST_MINUTE", "L'autobus part d'une minute."},
        {"HIGH_LOAD", "La charge de système est très haute."}
    };
}
```

Mapping WarningMessage keys to the fr and CA locale:

```
/**
 * WarningMessage_fr_CA.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
public class WarningMessage_fr_CA extends ListResourceBundle {
    public Object[][] getContents() {
        return contents;
    }
    static final Object[][] contents = {
        {"HIGH_LOAD", "L'utilisation de système est très haute."}
    };
}
```

Creating Resource Bundle with Property Files: NoteMessage

Mapping NoteMessage keys to the default locale, file name: NoteMessage.properties

```
STARTING_SYSTEM=Starting the system...
LOADING_SETTINGS=Loading user's settings...
```

Note that:

- The property file names follow the same convention as the bundle class names.
- When the NoteMessage bundle is requested, the system will actually read in the property file, and convert it into a PropertyResourceBundle.

Mapping NoteMessage keys to the fr locale, file name: NoteMessage_fr.properties

```
STARTING_SYSTEM=Commencer le système...
LOADING_SETTINGS=Les arrangements de l'utilisateur de chargement...
```

Mapping NoteMessage keys to the fr and CA locale, file name:
NoteMessage_fr_CA.properties

```
LOADING_SETTINGS=Commencer à charger les arrangements de l'utilisateur..
```

Using Resource Bundles

Here is a sample program that shows how to request a resource bundle and lookup the bundle object of the a given key and a given locale:

```
/**
 * ResourceBundleTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.util.*;
public class ResourceBundleTest {
    public static void main(String[] a) {
        bundleClassTest();
        listBundleClassTest();
        propertyBundleFileTest();
    }
    public static void bundleClassTest() {
        ResourceBundle rb;
        Locale l;
        System.out.println("ResourceBundle Test:");
        System.out.println("The default resource:");
        rb = ResourceBundle.getBundle("ErrorMessage");
        System.out.println("  1:" + rb.getString("NO_FILE"));
        System.out.println("  2:" + rb.getString("FILE_EMPTY"));
        l = Locale.FRENCH;
        System.out.println("The French resource:");
        rb = ResourceBundle.getBundle("ErrorMessage", l);
        System.out.println("  1:" + rb.getString("NO_FILE"));
        System.out.println("  2:" + rb.getString("FILE_EMPTY"));
        l = new Locale("fr", "CA");
        System.out.println("The French (Cananda) resource:");
        rb = ResourceBundle.getBundle("ErrorMessage", l);
        System.out.println("  1:" + rb.getString("NO_FILE"));
        System.out.println("  2:" + rb.getString("FILE_EMPTY"));
    }
    public static void listBundleClassTest() {
        ResourceBundle rb;
        Locale l;
        System.out.println("ListResourceBundle Test:");
        System.out.println("The default resource:");
        rb = ResourceBundle.getBundle("WarningMessage");
        System.out.println("  1:" + rb.getString("LAST_MINUTE"));
        System.out.println("  2:" + rb.getString("HIGH_LOAD"));
        l = Locale.FRENCH;
        System.out.println("The French resource:");
        rb = ResourceBundle.getBundle("WarningMessage", l);
        System.out.println("  1:" + rb.getString("LAST_MINUTE"));
        System.out.println("  2:" + rb.getString("HIGH_LOAD"));
        l = new Locale("fr", "CA");
        System.out.println("The French (Cananda) resource:");
        rb = ResourceBundle.getBundle("WarningMessage", l);
        System.out.println("  1:" + rb.getString("LAST_MINUTE"));
        System.out.println("  2:" + rb.getString("HIGH_LOAD"));
    }
    public static void propertyBundleFileTest() {
        ResourceBundle rb;
        Locale l;
        System.out.println("PropertyResourceBundle Test:");
        System.out.println("The default resource:");
    }
}
```

```

        rb = ResourceBundle.getBundle("NoteMessage");
        System.out.println("  1:" + rb.getString("STARTING_SYSTEM"));
        System.out.println("  2:" + rb.getString("LOADING_SETTINGS"));
        l = Locale.FRENCH;
        System.out.println("The French resource:");
        rb = ResourceBundle.getBundle("NoteMessage", l);
        System.out.println("  1:" + rb.getString("STARTING_SYSTEM"));
        System.out.println("  2:" + rb.getString("LOADING_SETTINGS"));
        l = new Locale("fr", "CA");
        System.out.println("The French (Canada) resource:");
        rb = ResourceBundle.getBundle("NoteMessage", l);
        System.out.println("  1:" + rb.getString("STARTING_SYSTEM"));
        System.out.println("  2:" + rb.getString("LOADING_SETTINGS"));
    }
}

```

Output

```

ResourceBundle Test:
The default resource:
  1:File doesn't exist.
  2:File is empty.
The French resource:
  1:Document n'existe pas.
  2:Rien.
The French (Canada) resource:
  1:Dossier n'existe pas.
  2:Rien.
ListResourceBundle Test:
The default resource:
  1:Bus is leaving within one minute.
  2:System load is very high.
The French resource:
  1:L'autobus part d'une minute.
  2:La charge de système est très haute.
The French (Canada) resource:
  1:L'autobus part d'une minute.
  2:L'utilisation de système est très haute.
PropertyResourceBundle Test:
The default resource:
  1:Starting the system...
  2:Loading user's settings...
The French resource:
  1:Commencer le système...
  2:Les arrangements de l'utilisateur de chargement...
The French (Canada) resource:
  1:Commencer le système...
  2:Commencer à charger les arrangements de l'utilisateur...

```

Note that:

- All the bundle classes must be compiled. You may need the "-classpath ." option to help the "javac" command to locate the depending classes to compile a multi classes program.
- You may need the "-cp ." option to help the "java" command to locate the depending classes and the bundle property files.

Comparing the 3 different ways of internationalize text messages, the property file is the best:

- Easy to implement.
- No re-compilation is needed after modification.

Unnamed Packages

This chapter explains:

- What is an unnamed package.
- How classes in unnamed packages are created, and referred.
- How to avoid compilation issue with JDK 1.4.1 when using import statement on classes defined in unnamed packages.
- How to import classes defined in unnamed packages into named packages.

What Is an Unnamed Package

Unnamed packages contain compilation units (classes or interfaces) that have no package declaration statement. For example, the following class is a compilation unit in an unnamed package, Hello.java:

```
class Hello {  
    public static void main(String[] a) {  
        System.out.println("Hello world!");  
    }  
}
```

There are a number of questions about unnamed packages we need to answer:

- How many unnamed packages can JDK support?
- How compilation units in each unnamed package are stored?
- How to refer to the compilation units in unnamed packages?

Based on the documentation of JDK for Windows system, my guess is that JDK supports multiple unnamed packages. Each unnamed package is stored in the top directory of each class path specified in one of the following three ways:

- -classpath path;path;...
- set CLASSPATH=path;path;...
- "current directory"

Let's do some tests to answer these questions and to confirm my guess.

One Class in an Unnamed Package - Hello.java

I will begin with a simple case, where there is only one class that has no package declaration statement, so it can be stored in any unnamed package. Assuming that we have Hello.java entered in a directory called C:\hy\src, enter the following commands in a command window:

```
C:\hy\src>type Hello.java  
class Hello {
```

```
    public static void main(String[] a) {
        System.out.println("Hello world!");
    }
}

C:\hy\src>del Hello.class
Could Not Find C:\herong\src\Hello.class

C:\hy\src>set CLASSPATH=

C:\hy\src>\jdk1.3.1\bin\java -version
java version "1.3.1_01"
Java(TM) 2 Runtime Environment, Standard Edition (build 1.3.1_01)
Java HotSpot(TM) Client VM (build 1.3.1_01, mixed mode)

C:\hy\src>\jdk1.3.1\bin\javac Hello.java

C:\hy\src>\jdk1.3.1\bin\java Hello
Hello world!

C:\hy\src>del Hello.class
C:\hy\src>\jdk1.3.1\bin\javac -classpath \hy Hello.java

C:\hy\src>\jdk1.3.1\bin\java -classpath \hy Hello
Exception in thread "main" java.lang.NoClassDefFoundError: Hello

C:\hy\src>del Hello.class

C:\hy\src>\jdk1.3.1\bin\javac -classpath . Hello.java

C:\hy\src>\jdk1.3.1\bin\java -classpath . Hello
Hello world!

C:\hy\src>del Hello.class
```

This test tells us that:

- I am using JDK 1.3.1_01.
- If there is no -classpath command option specified, and no CLASSPATH environment variable defined, there is one default class path: current directory.
- Class paths are searched for the class being invoked in the execution process.

In the next test, let's move the current directory to a directory different than the Hello.java directory, and run the following commands:

```
C:\hy\src>mkdir ..\tmp

C:\hy\src>cd ..\tmp

C:\hy\tmp>del ..\src\Hello.class
Could Not Find C:\hy\src\Hello.class

C:\hy\tmp>\jdk1.3.1\bin\javac -classpath . Hello.java
error: cannot read: Hello.java
1 error

C:\hy\tmp>\jdk1.3.1\bin\javac -classpath . ..\src\Hello.java

C:\hy\tmp>\jdk1.3.1\bin\java -classpath . Hello
```

```
Exception in thread "main" java.lang.NoClassDefFoundError: Hello
C:\hy\tmp>del ..\src\Hello.class
C:\hy\tmp>\jdk1.3.1\bin\javac -classpath ..\src ..\src\Hello.java
C:\hy\tmp>\jdk1.3.1\bin\java -classpath ..\src Hello
Hello world!
C:\hy\tmp>del ..\src\Hello.class
```

The second test shows us that:

- Class to be compiled must be specified as command argument with full pathname. Class paths are not searched for the classes to be compiled.

Two Classes in Unnamed Packages - Hello.java and CallingHello.java

Testing with two classes is definitely more interesting. Let's enter another class called, CallingHello.java, in C:\hy\src directory, and run the following commands:

```
C:\hy\src>type CallingHello.java
/**
 * CallingHello.java
 * Copyright (c) 2003 by Dr. Herong Yang
 */
public class CallingHello {
    public static void main(String[] a) {
        System.out.println("Calling Hello.main()...");
        Hello.main(a);
    }
}

C:\hy\src>del *.class
Could Not Find C:\hy\src\*.class

C:\hy\src>\jdk1.3.1\bin\javac -classpath . CallingHello.java

C:\hy\src>\jdk1.3.1\bin\java -classpath . CallingHello
Calling Hello.main()...
Hello world!

C:\hy\src>del *.class

C:\hy\src>cd ..\tmp

C:\hy\tmp>\jdk1.3.1\bin\javac -classpath ..\src ..\src\CallingHello.java

C:\hy\tmp>\jdk1.3.1\bin\java -classpath ..\src CallingHello
Calling Hello.main()...
Hello world!

C:\hy\tmp>del ..\src\*.class
```

This test shows us that:

- The compiler is smart. It automatically compiles the source code of the referred class,

Hello, if Hello.class can not be found in the class paths.

In the next test, let's separate the classes into two directories, and run the following commands:

```
C:\hy\tmp>copy ..\src\CallingHello.java
C:\hy\tmp>del ..\src\CallingHello.java
C:\hy\tmp>del *.class
Could Not Find C:\hy\tmp\*.class
C:\hy\tmp>del ..\src\*.class
Could Not Find C:\hy\src\*.class
C:\hy\tmp>\jdk1.3.1\bin\javac -classpath . CallingHello.java
CallingHello.java:8: cannot resolve symbol
symbol  : variable Hello
location: class CallingHello
    Hello.main(a);
    ^
1 error
C:\hy\tmp>\jdk1.3.1\bin\javac -classpath ..\src CallingHello.java
C:\hy\tmp>\jdk1.3.1\bin\java -classpath . CallingHello
Calling Hello.main()...
Exception in thread "main" java.lang.NoClassDefFoundError: Hello
    at CallingHello.main(CallingHello.java:8)
C:\hy\tmp>\jdk1.3.1\bin\java -classpath .;..\src CallingHello
Calling Hello.main()...
Hello world!
C:\hy\tmp>del *.class
C:\hy\tmp>del ..\src\*.class
```

This test confirms that:

- The compiler is indeed smart. It will search for source code in class paths, even the referred class is not in the same directory as the class that is being compiled.

Importing Classes Defined in Unnamed Packages

For classes defined in unnamed packages, their simple names are identical to their fully qualified names. Therefore, there is no need to use import statement in the compilation units where they are referred.

But if you really want to use an import statement, it shouldn't be a problem. So let's enter another class, ImportingHello.java, in C:\hy\tmp directory, and run the following commands to see what happens with the import statement:

```
C:\hy\tmp>type ImportingHello.java
/**
 * ImportingHello.java
```



```
* Copyright (c) 2003 by Dr. Herong Yang
*/
import Hello;
public class ImportingHello {
    public static void main(String[] a) {
        System.out.println("Calling the imported Hello.main()...");
        Hello.main(a);
    }
}

C:\hy\tmp>del *.class
Could Not Find C:\hy\tmp\*.class

C:\hy\tmp>del ..\src\*.class
Could Not Find C:\hy\src\*.class

C:\hy\tmp>\jdk1.3.1\bin\javac -classpath ..\src ImportingHello.java

C:\hy\tmp>\jdk1.3.1\bin\java -classpath .;..\src ImportingHello
Calling the imported Hello.main()...
Hello world!

C:\hy\tmp>del *.class

C:\hy\tmp>del ..\src\*.class
```

As you can see, there is no problem. The import statement makes no impact on the compilation and execution processes.

Importing Classes Defined in Unnamed Packages Using JDK 1.4.1

However, the import statement will cause compilation problem if we use JDK 1.4.1, see the following commands and outputs:

```
C:\hy\tmp>\j2sdk1.4.1\bin\java -version
java version "1.4.1_01"
Java(TM) 2 Runtime Environment, Standard Edition (build 1.4.1_01-b01)
Java HotSpot(TM) Client VM (build 1.4.1_01-b01, mixed mode)

C:\hy\tmp>\j2sdk1.4.1\bin\javac -classpath ..\src ImportingHello.java
ImportingHello.java:5: '.' expected
import Hello;
      ^
1 error
```

Obviously, specification on import statement has been changed. With JDK 1.4.1, import statement can not be used on compilation units in unnamed packages. So, from now on, stop use import statements on classes or interfaces defined in unnamed packages.

Importing Classes Defined in Unnamed Packages to Named Packages

Recently, I received an email from a reader asking me how to refer to a class defined in an unnamed package if the calling class is in a named class with JDK 1.4.1. To answer this question, let's enter a new class called "ImportingHelloCom" in a package called

"com", and run the following commands:

```
C:\hy\tmp>type com\ImportingHelloCom.java
/**
 * ImportingHelloCom.java
 * Copyright (c) 2004 by Dr. Herong Yang
 */
package com;
import Hello;
public class ImportingHelloCom {
    public static void main(String[] a) {
        System.out.println("Calling the imported Hello.main()...");
        Hello.main(a);
    }
}

C:\hy\tmp>type Hello.java
class Hello {
    public static void main(String[] a) {
        System.out.println("Hello world!");
    }
}

C:\hy\tmp>del *.class
Could Not Find C:\hy\tmp\*.class

C:\hy\tmp>del com\*.class
Could Not Find C:\hy\tmp\com\*.class

C:\jdk1.3.1\bin\javac com\ImportingHelloCom.java
com\ImportingHelloCom.java:5: cannot resolve symbol
symbol: class Hello
import Hello;
^
com\ImportingHelloCom.java:9: cannot resolve symbol
symbol  : variable Hello
location: class ImportingHelloCom
    Hello.main(a);
    ^
2 errors

C:\jdk1.3.1\bin\javac -classpath . com\ImportingHelloCom.java
com\ImportingHelloCom.java:6: Hello is not public in empty package;
cannot be accessed from outside package
import Hello;
^
com\ImportingHelloCom.java:10: Hello is not public in empty package;
cannot be accessed from outside package
    Hello.main(a);
    ^
com\ImportingHelloCom.java:10: main(java.lang.String[]) in Hello is
not defined in a public class or interface; cannot be accessed from
outside package
    Hello.main(a);
    ^
3 errors
```

This test shows us that:

- Only public classes in an unnamed package can be imported into named packages.

- Comparing with the testing results from the previous sections, there is really only one unnamed package. All classes defined without a package name are considered to be in this single package. All classes can be imported into each other without the "public" attribute.

To fix the last compilation error, just declare the Hello class as "public". Now let's modify the classes and run the following commands:

```
C:\hy\tmp>type Hello.java
public class Hello {
    public static void main(String[] a) {
        System.out.println("Hello world!");
    }
}

C:\hy\tmp>del *.class
Could Not Find C:\hy\tmp\*.class

C:\hy\tmp>del com\*.class
Could Not Find C:\hy\tmp\com\*.class

C:\jdk1.3.1\bin\javac -classpath . com\ImportingHelloCom.java

C:\jdk1.3.1\bin\java -classpath . com.ImportingHelloCom
Calling the imported Hello.main()...
Hello world!
```

As you can see, importing a class defined in an unnamed package into a class defined in a named package is easy with JDK 1.3.1.

Importing Classes Defined in Unnamed Packages to Named Packages Using JDK 1.4.1

As we learned earlier in this chapter, if we re-run the previous test with JDK 1.4.1, we will get a compilation error on the "import Hello" statement.

In this case, if you try my earlier suggestion of removing the import statement, you will get another compilation error about not able to resolve the class name "Hello", because it is not defined in the same package as the calling class.

How to resolve this problem? I am suggesting two options:

- If you have the source code of the class to be imported from the unnamed package, you can modify the class to put it in a named package.
- If you don't have the source code of the class to be imported from the unnamed package, you can create a new class in a named package to wrapper the to-be-imported class and compile it with JDK 1.3.1. The wrapper class can then be easily imported into any class with JDK 1.4.1.

To approve my second option, let's continue with our previous tests, enter a wrapper class called HelloWrapper in a package called wrapper, then run the following commands:

```
C:\hy\tmp>type wrapper\HelloWrapper.java
/**
 * HelloWrapper.java
 * Wrapping Hello class from the unnamed package into a named package
 * To be compiled with JDK 1.3.1.
 * Copyright (c) 2004 by Dr. Herong Yang
 */
package wrapper;
import Hello;
public class HelloWrapper {
    public static void main(String[] a) {
        System.out.println("Wrapping Hello classs...");
        Hello.main(a);
    }
}

C:\hy\tmp>type com\ImportingHelloWrapperCom.java
/**
 * ImportingHelloWrapperCom.java
 * Copyright (c) 2004 by Dr. Herong Yang
 */
package com;
import wrapper.HelloWrapper;
public class ImportingHelloWrapperCom {
    public static void main(String[] a) {
        System.out.println("Calling the imported Hello.main()...");
        HelloWrapper.main(a);
    }
}

C:\jdk1.3.1\bin\javac -classpath . wrapper\HelloWrapper.java

C:\j2sdk1.4.1\bin\javac -classpath . com\ImportingHelloWrapperCom.java

C:\j2sdk1.4.1\bin\java -classpath . com.ImportingHelloWrapperCom
Calling the imported Hello.main()...
Wrapping Hello classs...
Hello world!
```

As you can see from the output, the problem is perfectly resolved by using both of versions of JDK together. So after you installed JDK 1.4.1, don't delete JDK 1.3.1, you might need it one day!

Conclusion

- The JDK compiler will compile the imported classes if not compiled yet and source codes are available in the class path.
- There is really only one unnamed package. All classes defined without a package name are considered to be in this single unnamed package.
- With JDK 1.3.1 and lower versions, import statement is allowed on classes defined in the unnamed package.
- With JDK 1.4.1, import statement is not allowed on classes defined in the unnamed package.
- If you have an old class defined in the unnamed package which can not be directly imported to any classes in named packages with JDK 1.4.1, and you don't have the

source code for that old class, you can create a wrapper class in a named package, and compile it with JDK 1.3.1. The wrapper class can then be imported to any classes with JDK 1.4.1.

Collections

This note is based on J2SDK 1.4.1_01.

Types of Collections

If you are looking for a utility class to organize a group of objects in your application, you will have a hard time to decide which one to use, because J2SDK offers so many of them: `AbstractCollection`, `AbstractList`, `AbstractMap`, `AbstractSequentialList`, `AbstractSet`, `ArrayList`, `Collection`, `Dictionary`, `HashMap`, `HashSet`, `HashTable`, `IdentityHashMap`, `LinkedHashMap`, `LinkedHashSet`, `LinkedList`, `List`, `Map`, `Set`, `SortedMap`, `SortedSet`, `Stack`, `TreeMap`, `TreeSet`, `Vector`, `WeakHashMap`.

To help you out, let me introduce you some basic concepts about data collections.

Collection: A group of data elements, or objects, organized together. Depending on how the elements can be stored and retrieved, collections are divided into 3 types: set, list, and map.

Set: Data elements are stored and retrieved without position numbers and keys.

The operation of storing an element into a set collection is simple. We can just put the element anywhere in the collection.

However the operation of retrieving an element out of a set collection is very difficult, since there is no clue on where that element is stored in the collection, no position number and no key. The only thing you have is the element, so you have to go through all elements in the collection, and compare each one of them with the given element. If the given element matches a particular element in the collection, you declare that you have found the element in the collection. There is no need to turn that element, because the user of the set collection already has the element. So the only use of the searching process is to identify if the given element is in the collection or not.

List: Data elements are stored and retrieved by position numbers. List is also called sequence.

The operation of storing an element into a list collection is simple but tricky. If you are asked to store an element into a collection with a specific position number, and there is already an existing element at that position, what are you going to do? The answer depends on additional rules about the list collection. If the list collection requires you to replace the existing element with given element, then replace it. If the list collection requires you to move the element at the requested position and all the subsequent elements forward by one position, and put the given element at the requested position, then you will have a lot of work to do.

The operation of retrieving an element out of a list collection is much easier than the set collection. With the given position number, just go to that position and return the element at that position.

Map: Date elements are stored and retrieved by keys. Map is also called dictionary.

The operation of storing an element into a map collection also simple and tricky. If you are asked to store an element into a collection with a key associated with it, and there is already an existing element with that same key, what you going to do? The answer depends on additional rules about the map collection. If duplicate keys are not allowed in the map collection, you should replace the existing element with given element.

The operation of retrieving an element out of a map collection is relatively easy. With the given key, you should through all the keys in the collection, and compare each one of them with the given key. If there is a match, return the element associate with the matching key. Comparing with the set collection, we are matching keys instead of the data elements to locate an element. Usually, the keys are designed to be easier to match. Most commonly used keys are numbers and strings.

In addition to operations of storing and retrieving elements, we should also be able to remove an element, search for a specific element, and iterate through all elements in a collections. So there are 5 major operations for any collections:

- Storing an element into the collection.
- Retrieving an element out of the collection.
- Removing an element from the collection.
- Searching for an element in the collection.
- Iterating through all the elements in the collection.

Data Structures of Collection Implementations

In the previous section, we only talked about the functionalities of collections, and divided collections into 3 types based on the differences in the their functionalities. In the section, we will look at what kind of data structures we are going to use to store the data elements. Choosing any data structure to implement a collection should not affect the accuracy of its functionalities, but it will affect the performance.

There 4 major types of data structures that can be used in the implementation of collections: array, linked list, binary tree, hash table.

Before going into any details on each data structures, we need to clarify one thing: when a user wants to store a data element into a collection, we only need to store the reference number of that data element in the collection. The reference member of a data element is usually the memory address where that data element is actually stored.

Array: Reference numbers of data elements are stored consecutively in the memory, one

immediately after another.

Linked List: Reference numbers of data elements are stored randomly in the memory, but they are linked with a pointer associated with each reference number pointing to the next reference number.

Binary Tree: Reference numbers of data elements are stored randomly in the memory, but they are linked with two pointers associated with each reference number pointing to two other reference numbers.

Hash table: Reference numbers of data elements are stored in one single section of memory. The exact location of the each reference number is based the hash number obtained from the data element.

Of course, I am only giving a rough description for each data structure here. For more details, please read any data structure text book.

Collection Implementations in J2SDK

J2SDK 1.4.1_02 provides the following implementations of collections:

	Array	Linked List	Binary Tree	Hash Table
Set			TreeSet	HashSet
Set				LinkedHashSet
List	ArrayList	LinkedList		
List	Vector			
List	Stack			
Map			TreeMap	HashMap
Map				IdentityHashMap
Map				WeakHashMap
Map				Hashtable

Search Operation Performance

Once a collection is created, search is probably the most frequently used operation. So let's compare the performance of the search operation among the collection classes offered in J2SDK. Here is the main program, CollectionTest:

```
import java.util.*;
class CollectionTest {
    public static void main(String[] a) {
        Object[] arr = getObjects();
        Collection col;
        System.out.println("Collection Class, Number of Objects, Time");
        col = new HashSet();
        testCollection(arr,col);
        col = new LinkedHashSet();
        testCollection(arr,col);
        // col = new TreeSet(); // requires element.compareTo()
        // testCollection(arr,col);
    }
}
```



```

        col = new Vector();
        testCollection(arr,col);
        col = new ArrayList();
        testCollection(arr,col);
        col = new LinkedList();
        testCollection(arr,col);
        Map dic;
        System.out.println("Map Class, Number of Objects, Time");
        dic = new TreeMap();
        testMap(arr,dic);
        dic = new HashMap();
        testMap(arr,dic);
        dic = new IdentityHashMap();
        testMap(arr,dic);
        dic = new WeakHashMap();
        testMap(arr,dic);
        dic = new Hashtable();
        testMap(arr,dic);
    }
    private static void testCollection(Object[] arr, Collection col) {
        for (int l=0; l<arr.length; l++) {
            col.add(arr[l]);
        }
        Date t1 = new Date();
        boolean ok = true;
        for (int l=0; l<arr.length; l++) {
            ok = ok && col.contains(arr[l]);
        }
        Date t2 = new Date();
        if (ok==false) System.out.println("Search failed.");
        System.out.println(col.getClass().getName()+" "+
            arr.length+" "+(t2.getTime()-t1.getTime()));
    }
    private static void testMap(Object[] arr, Map col) {
        for (int l=0; l<arr.length; l++) {
            col.put(new Integer(l), arr[l]);
        }
        Date t1 = new Date();
        boolean ok = true;
        for (int l=0; l<arr.length; l++) {
            ok = ok && col.containsValue(arr[l]);
        }
        Date t2 = new Date();
        if (ok==false) System.out.println("Search failed.");
        System.out.println(col.getClass().getName()+" "+
            arr.length+" "+(t2.getTime()-t1.getTime()));
    }
    private static Object[] getObjects() {
        int max_l = 5000;
        Object[] arr = new Object[max_l];
        for (int l=0; l<max_l; l++) {
            arr[l] = new MySimpleDate();
            // arr[l] = new MyDate(); // with element.compareTo() implemented
        }
        return arr;
    }
}

```

The idea of the test is simple. First, an array of objects of my own class, MySimpleDate, is created. Then, an object of the to-be-tested collection class is create, and filled with all the objects from the array. The performance test is concentrated on the CPU time used to

search all the objects in the collection one by one.

MySimpleDate class is really simple. Here is the source code:

```
import java.util.*;
public class MySimpleDate {
    private static Random r;
    private Date my_date;
    public MySimpleDate () {
        my_date = new Date();
        long l = my_date.getTime();
        int i = (int) (l/1000/60/60/24);
        if (r==null) r = new Random();
        i = r.nextInt(i);
        l = ((long) i)*24*60*60*1000;
        my_date.setTime(l);
    }
}
```

It contains only one piece of information, a instance of time with minutes, seconds and milliseconds truncated, and date randomized.

Since MySimpleDate class is not implementing the Comparable interface, its objects can not be managed by TreeSet collection. So I have commented out some lines in the CollectionTest program to avoid testing on TreeSet collection.

Here is output of the program compiled and executed by J2SDK 1.4.1_01:

```
Collection Class, Number of Objects, Time
java.util.HashSet, 5000, 10
java.util.LinkedHashSet, 5000, 0
java.util.Vector, 5000, 661
java.util.ArrayList, 5000, 651
java.util.LinkedList, 5000, 762
Map Class, Number of Objects, Time
java.util.TreeMap, 5000, 1021
java.util.HashMap, 5000, 1712
java.util.IdentityHashMap, 5000, 391
java.util.WeakHashMap, 5000, 1572
java.util.Hashtable, 5000, 3145
```

A couple of notes about the output:

- LinkedHashSet took no time (0 millisecond) to search 5000 elements. I can not believe this. Something must be wrong here.
- HashSet performed better than most other collection classes, as expected, because locations of elements in a HashSet are based the hash value of each elements, which will provide almost instant access time.
- Vector, and ArrayList, took about the same time, as expected, because they both are based on array data structure.
- LinkedList took longer time than ArrayList, as expected, because iterating through a linked list is slower than an array.
- TreeMap, HashMap, WeakHashMap and Hashtable took must longer time, because

maps are not really designed to search elements directly. We should search the keys instead of elements.

- IdentityHashMap out performed all others map collections. I don't know why.

I repeated the execution 3 times with different number of elements and recorded the following results:

Collection Class	Number of Elements		
	5000	10000	20000
HashSet	10	20	20
LinkedHashSet	0	10	20
Vector	661	2714	10936
ArrayList	651	2694	10676
LinkedList	762	3305	28122
TreeMap	1021	10256	52719
HashMap	1712	12629	60050
IdentityHashMap	391	1532	7000
WeakHashMap	1572	failed	failed
Hashtable	3145	21261	89103

The results show that:

- HashSet and LinkedHashSet maintained an almost constant performance level, while the number of elements doubled twice. They are perfectly designed for the search operation.
- Vector, ArrayList and LinkedList decreased their performance exponentially as the number of elements doubles.
- TreeMap, HashMap, IdentityHashMap and Hashtable decreased their performance exponentially as the number of elements doubles.
- WeakHashMap is not reliable as mentioned in the J2SDK specification.

Data Elements with Natural Order

In order to use TreeSet collection, data elements must be comparable. So I enhanced MySimpleDate class and rename it as MyDate class:

```
import java.util.*;
public class MyDate implements Comparable {
    private static Random r;
    private Date my_date;
    public MyDate () {
        my_date = new Date();
        long l = my_date.getTime();
        int i = (int) (l/1000/60/60/24);
        if (r==null) r = new Random();
        i = r.nextInt(i);
        l = ((long) i)*24*60*60*1000;
        my_date.setTime(l);
    }
    public long getTime() {
        return my_date.getTime();
    }
    public int compareTo(Object o) {
```

```

        if (getTime() < ((MyDate) o).getTime()) return -1;
        else if (getTime() == ((MyDate) o).getTime()) return 0;
        else return 1;
    }
    public boolean equals(Object o) {
        return compareTo(o) == 0;
    }
    public int hashCode() {
        return (int)(getTime() ^ (getTime() >>> 32));
    }
}

```

Note that:

- Comparable interface is implemented in this class. It requires to implement the compareTo() method.
- If the compareTo() method is implemented, the equals() method must be overridden, in order to maintain the following logic: *if x.compareTo(y) == 0, then x.equals(y) == true*. The default equals() method compares two objects based on their reference numbers (addresses in memory), not looking at any data inside the objects. So two separate objects, even with identical data, will always fail the default equals() method.
- If the equals() method is overridden, the hashCode() method must also be overridden, in order to maintain the following logic: *if x.equals(y) == true, then x.hashCode() == y.hashCode()*. The default hashCode() method generates hash value also based on the object's reference number (address in memory), not looking at any data inside the object. So two separate objects with identical data may not have the same hash value, if the default hashCode() method is used.
- I copied the hash code generation expression from the Long class.

Using the MyDate class with my own compareTo(), equals(), and hashCode() methods, I repeated the search operation performance tests. Here are the results:

Collection Class	With MySimpleDate Number of Elements			With MyDate Number of Elements		
	5000	10000	20000	5000	10000	20000
HashSet	10	20	20	70	10	30
LinkedHashSet	0	10	20	0	10	10
TreeSet	n/a	n/a	n/a	10	40	70
Vector	661	2714	10936	1502	9424	37626
ArrayList	651	2694	10676	1573	9314	37646
LinkedList	762	3305	28122	2193	14102	54902
TreeMap	1021	10256	52719	4276	18417	56205
HashMap	1712	12629	60050	4727	29024	92068
IdentityHashMap	391	1532	7000	381	1532	6980
WeakHashMap	1572	failed	failed	failed	failed	failed
Hashtable	3145	21261	89103	5518	23395	89094

The results show that:

- HashSet, LinkedHashSet, and TreeSet did very well. The well designed hash value helped to reduce the number of objects to be fetched for comparison.
- Vector, ArrayList, and LinkedList took 3 times longer than the MySimpleDate

version. My guess is that the overridden equals() method is 3 times slower than the default equals() method, which just checks the memory addresses of the objects.

- TreeMap and HashMap took a little bit longer than the MySimpleDate version.
- IdentityHashMap and Hashtable took about the same time as the MySimpleDate version.

Character Set and Encoding

Notes and sample codes bellow are based on J2SDK 1.4.1_01.

What is a Character Encoding

Character Encoding: A map scheme between code points of a code character set and sequences of bytes.

Coded Character Set: A character set in which each character has an assigned integral number.

Code Point: An integral number assigned to a character in a coded character set.

Unicode: A coded character set that contains all characters used in the written languages of the world and special symbols.

As of 1.4.1, J2SDK supports Unicode 3.0, based on the information provided in the reference document of java.lang.Character class.

I am not how JDK is going to support Unicode 3.1, because it now contains characters with code points greater than U+FFFF, which is the maximum value of 'char' type in Java.

Because of the 'char' limitation, JDK can only support encoding and decoding code points in the 16-bit range: U+0000 - U+FFFF.

Supported Character Encodings

JDK uses the java.nio.charset.Charset class to represent a character encoding, with both encode() method and decode() method. It also provides a method, availableCharsets(), to return all supported encodings. Here is a program to display all the supported character encodings:

```
/**
 * Encodings.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.nio.charset.*;
import java.util.*;
class Encodings {
    public static void main(String[] arg) {
        SortedMap m = Charset.availableCharsets();
        Set k = m.keySet();
        System.out.println("Canonical name, Display name,"
            + " Can encode, Aliases");
        Iterator i = k.iterator();
        while (i.hasNext()) {
            String n = (String) i.next();
```

```

        Charset e = (Charset) m.get(n);
        String d = e.displayName();
        boolean c = e.canEncode();
        System.out.print(n+", "+d+", "+c);
        Set s = e.aliases();
        Iterator j = s.iterator();
        while (j.hasNext()) {
            String a = (String) j.next();
            System.out.print(", "+a);
        }
        System.out.println("");
    }
}
}

```

Output:

```

Canonical name, Display name, Can encode, Aliases
Big5, Big5, true, csBig5
Big5-HKSCS, Big5-HKSCS, true, big5-hkscs, Big5_HKSCS, big5hkscs
EUC-CN, EUC-CN, true
EUC-JP, EUC-JP, true, eucjis, x-eucjp, csEUCPkFmtjapanese, eucjp,
    Extended_UNIX_Code_Packed_Format_for_Japanese, x-euc-jp, euc_jp
euc-jp-linux, euc-jp-linux, true, euc_jp_linux
EUC-KR, EUC-KR, true, ksc5601, 5601, ksc5601_1987, ksc_5601,
    ksc5601-1987, euc_kr, ks_c_5601-1987, euckr, csEUCKR
EUC-TW, EUC-TW, true, cns11643, euc_tw, euctw
GB18030, GB18030, true, gb18030-2000
GBK, GBK, true, GBK
ISCII91, ISCII91, true, isciii, ST_SEV_358-88, iso-ir-153,
    csISO153GOST1976874
ISO-2022-CN-CNS, ISO-2022-CN-CNS, true, ISO2022CN_CNS
ISO-2022-CN-GB, ISO-2022-CN-GB, true, ISO2022CN_GB
ISO-2022-KR, ISO-2022-KR, true, ISO2022KR, csISO2022KR
ISO-8859-1, ISO-8859-1, true, iso-ir-100, 8859_1, ISO_8859-1, ISO8859_1,
    819, csISOLatin1, IBM-819, ISO_8859-1:1987, latin1, cp819, ISO8859-1,
    IBM819, ISO_8859_1, l1
ISO-8859-13, ISO-8859-13, true
ISO-8859-15, ISO-8859-15, true, 8859_15, csISOLatin9, IBM923, cp923,
    923, L9, IBM-923, ISO8859-15, LATIN9, ISO_8859-15, LATIN0,
    csISOLatin0, ISO8859_15_FDIS, ISO-8859-15
ISO-8859-2, ISO-8859-2, true
ISO-8859-3, ISO-8859-3, true
ISO-8859-4, ISO-8859-4, true
ISO-8859-5, ISO-8859-5, true
ISO-8859-6, ISO-8859-6, true
ISO-8859-7, ISO-8859-7, true
ISO-8859-8, ISO-8859-8, true
ISO-8859-9, ISO-8859-9, true
JIS0201, JIS0201, true, X0201, JIS_X0201, csHalfWidthKatakana
JIS0208, JIS0208, true, JIS_C6626-1983, csISO87JISX0208, x0208,
    JIS_X0208-1983, iso-ir-87
JIS0212, JIS0212, true, jis_x0212-1990, x0212, iso-ir-159,
    csISO159JISC02121990
Johab, Johab, true, ms1361, ksc5601_1992, ksc5601-1992
KOI8-R, KOI8-R, true
Shift_JIS, Shift_JIS, true, shift-jis, x-sjis, ms_kanji, shift_jis,
    csShiftJIS, sjis, pck
TIS-620, TIS-620, true
US-ASCII, US-ASCII, true, IBM367, ISO646-US, ANSI_X3.4-1986, cp367,
    ASCII, iso_646.irv:1983, 646, us, iso-ir-6, csASCII, ANSI_X3.4-1968,

```

```

ISO_646.irv:1991
UTF-16, UTF-16, true, UTF_16
UTF-16BE, UTF-16BE, true, X-UTF-16BE, UTF_16BE, ISO-10646-UCS-2
UTF-16LE, UTF-16LE, true, UTF_16LE, X-UTF-16LE
UTF-8, UTF-8, true, UTF8
windows-1250, windows-1250, true
windows-1251, windows-1251, true
windows-1252, windows-1252, true, cp1252
windows-1253, windows-1253, true
windows-1254, windows-1254, true
windows-1255, windows-1255, true
windows-1256, windows-1256, true
windows-1257, windows-1257, true
windows-1258, windows-1258, true
windows-936, windows-936, true, ms936, ms_936
windows-949, windows-949, true, ms_949, ms949
windows-950, windows-950, true, ms950

```

Methods to Encode Characters

There are 4 methods to encode characters:

- `CharsetEncoder.encode()`
- `Charset.encode()`
- `String.getBytes()`
- `OutputStreamWriter.write()`

Here is a program that demonstrate how to encode characters in each of the above 4 methods:

```

/**
 * EncodingSampler.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.nio.*;
import java.nio.charset.*;
class EncodingSampler {
    static String dfltCharset = null;
    static char[] chars={0x0000, 0x003F, 0x0040, 0x007F, 0x0080, 0x00BF,
                        0x00C0, 0x00FF, 0x0100, 0x3FFF, 0x4000, 0x7FFF,
                        0x8000, 0xBFFF, 0xC000, 0xEFFF, 0xF000, 0xFFFF};
    static char hexDigit[] = {'0', '1', '2', '3', '4', '5', '6', '7',
                              '8', '9', 'A', 'B', 'C', 'D', 'E', 'F'};
    public static void main(String[] arg) {
        String charset = null;
        if (arg.length>0) charset = arg[0];
        OutputStreamWriter o = new OutputStreamWriter(
            new ByteArrayOutputStream());
        dfltCharset = o.getEncoding();
        if (charset==null) System.out.println("Default (" +dfltCharset
            +") encoding:");
        else System.out.println(charset+" encoding:");
        System.out.println("Char, String, Writer, Charset, Encoder");
        for (int i=0; i<chars.length; i++) {
            char c = chars[i];
            byte[] b1 = encodeByString(c,charset);
            byte[] b2 = encodeByWriter(c,charset);

```



```

        byte[] b3 = encodeByCharset(c, charset);
        byte[] b4 = encodeByEncoder(c, charset);
        System.out.print(charToHex(c)+" ");
        printBytes(b1);
        System.out.print(", ");
        printBytes(b2);
        System.out.print(", ");
        printBytes(b3);
        System.out.print(", ");
        printBytes(b4);
        System.out.println("");
    }
}

public static byte[] encodeByCharset(char c, String cs) {
    Charset cso = null;
    byte[] b = null;
    try {
        if (cs==null) cso = Charset.forName(dfltCharset);
        else cso = Charset.forName(cs);
        ByteBuffer bb = cso.encode(String.valueOf(c));
        b = copyBytes(bb.array(), bb.limit());
    } catch (IllegalCharsetNameException e) {
        System.out.println(e.toString());
    }
    return b;
}

public static byte[] encodeByEncoder(char c, String cs) {
    Charset cso = null;
    byte[] b = null;
    try {
        if (cs==null) cso = Charset.forName(dfltCharset);
        else cso = Charset.forName(cs);
        CharsetEncoder e = cso.newEncoder();
        e.reset();
        ByteBuffer bb = e.encode(CharBuffer.wrap(new char[] {c}));
        b = copyBytes(bb.array(), bb.limit());
    } catch (IllegalCharsetNameException e) {
        System.out.println(e.toString());
    } catch (CharacterCodingException e) {
        //System.out.println(e.toString());
        b = new byte[] {(byte)0x00};
    }
    return b;
}

public static byte[] encodeByString(char c, String cs) {
    String s = String.valueOf(c);
    byte[] b = null;
    if (cs==null) {
        b = s.getBytes();
    } else {
        try {
            b = s.getBytes(cs);
        } catch (UnsupportedEncodingException e) {
            System.out.println(e.toString());
        }
    }
    return b;
}

public static byte[] encodeByWriter(char c, String cs) {
    byte[] b = null;
    ByteArrayOutputStream bs = new ByteArrayOutputStream();
    OutputStreamWriter o = null;

```

```

        if (cs==null) {
            o = new OutputStreamWriter(bs);
        } else {
            try {
                o = new OutputStreamWriter(bs, cs);
            } catch (UnsupportedEncodingException e) {
                System.out.println(e.toString());
            }
        }
        String s = String.valueOf(c);
        try {
            o.write(s);
            o.flush();
            b = bs.toByteArray();
            o.close();
        } catch (IOException e) {
            System.out.println(e.toString());
        }
        return b;
    }
    public static byte[] copyBytes(byte[] a, int l) {
        byte[] b = new byte[l];
        for (int i=0; i<Math.min(l,a.length); i++) b[i] = a[i];
        return b;
    }
    public static void printBytes(byte[] b) {
        for (int j=0; j<b.length; j++)
            System.out.print(" "+byteToHex(b[j]));
    }
    public static String byteToHex(byte b) {
        char[] a = { hexDigit[(b >> 4) & 0x0f], hexDigit[b & 0x0f] };
        return new String(a);
    }
    public static String charToHex(char c) {
        byte hi = (byte) (c >>> 8);
        byte lo = (byte) (c & 0xff);
        return byteToHex(hi) + byteToHex(lo);
    }
}

```

Note that:

- If the same encoding is used, each of the encode method in the program should return the exactly the same byte sequence.
- `getEncoding()` is used on `OutputStreamWriter` class to get the name of the default encoding.
- There is now way to know the name of the default encoding on `String` class.
- There is no default instance of `Charset` and `Encoder`.
- In `encodeByEncoder()`, `0x00` is used as the output when the given character can not be encoded by the encoder.

Running this program without any argument will use the JVM's default encoding:

```

Default (Cp1252) encoding:
Char, String, Writer, Charset, Encoder
0000, 00, 00, 00, 00
003F, 3F, 3F, 3F, 3F

```

```

0040, 40, 40, 40, 40
007F, 7F, 7F, 7F, 7F
0080, 3F, 3F, 3F, 00
00BF, BF, BF, BF, BF
00C0, C0, C0, C0, C0
00FF, FF, FF, FF, FF
0100, 3F, 3F, 3F, 00
3FFF, 3F, 3F, 3F, 00
4000, 3F, 3F, 3F, 00
7FFF, 3F, 3F, 3F, 00
8000, 3F, 3F, 3F, 00
BFFF, 3F, 3F, 3F, 00
C000, 3F, 3F, 3F, 00
EFFF, 3F, 3F, 3F, 00
F000, 3F, 3F, 3F, 00
FFFF, 3F, 3F, 3F, 00

```

The results shows that:

- The default encoding of the String class seems to be the same as OutputStreamWriter: Cp1252.
- There are a number of characters that can not be encoded by Cp1252. The String, OutputStreamWriter, and Charset classes are returning 0x3F for those non-encodable characters.
- It's obvious that Cp1252 works on a character set in the 0x0000 - 0x00FF range.

Running the program again with 'CP1252' as argument should give us the same output as the previous run:

```

CP1252 encoding:
Char, String, Writer, Charset, Encoder
0000, 00, 00, 00, 00
003F, 3F, 3F, 3F, 3F
0040, 40, 40, 40, 40
007F, 7F, 7F, 7F, 7F
0080, 3F, 3F, 3F, 00
00BF, BF, BF, BF, BF
00C0, C0, C0, C0, C0
00FF, FF, FF, FF, FF
0100, 3F, 3F, 3F, 00
3FFF, 3F, 3F, 3F, 00
4000, 3F, 3F, 3F, 00
7FFF, 3F, 3F, 3F, 00
8000, 3F, 3F, 3F, 00
BFFF, 3F, 3F, 3F, 00
C000, 3F, 3F, 3F, 00
EFFF, 3F, 3F, 3F, 00
F000, 3F, 3F, 3F, 00
FFFF, 3F, 3F, 3F, 00

```

Let's try another encoding, ISO-8859-1:

```

ISO-8859-1 encoding:
Char, String, Writer, Charset, Encoder
0000, 00, 00, 00, 00
003F, 3F, 3F, 3F, 3F
0040, 40, 40, 40, 40
007F, 7F, 7F, 7F, 7F

```

```

0080, 80, 80, 80, 80
00BF, BF, BF, BF, BF
00C0, C0, C0, C0, C0
00FF, FF, FF, FF, FF
0100, 3F, 3F, 3F, 00
3FFF, 3F, 3F, 3F, 00
4000, 3F, 3F, 3F, 00
7FFF, 3F, 3F, 3F, 00
8000, 3F, 3F, 3F, 00
BFFF, 3F, 3F, 3F, 00
C000, 3F, 3F, 3F, 00
EFFF, 3F, 3F, 3F, 00
F000, 3F, 3F, 3F, 00
FFFF, 3F, 3F, 3F, 00

```

It appears to be the same as CP1252.

Let's try another one, US-ASCII:

```

US-ASCII encoding:
Char, String, Writer, Charset, Encoder
0000, 00, 00, 00, 00
003F, 3F, 3F, 3F, 3F
0040, 40, 40, 40, 40
007F, 7F, 7F, 7F, 7F
0080, 3F, 3F, 3F, 00
00BF, 3F, 3F, 3F, 00
00C0, 3F, 3F, 3F, 00
00FF, 3F, 3F, 3F, 00
0100, 3F, 3F, 3F, 00
3FFF, 3F, 3F, 3F, 00
4000, 3F, 3F, 3F, 00
7FFF, 3F, 3F, 3F, 00
8000, 3F, 3F, 3F, 00
BFFF, 3F, 3F, 3F, 00
C000, 3F, 3F, 3F, 00
EFFF, 3F, 3F, 3F, 00
F000, 3F, 3F, 3F, 00
FFFF, 3F, 3F, 3F, 00

```

It's obvious that US-ASCII works on a character set in the 0x0000 - 0x007F range.

Let's try an encoding that is designed for the Unicode character set, UTF-8:

```

UTF-8 encoding:
Char, String, Writer, Charset, Encoder
0000, 00, 00, 00, 00
003F, 3F, 3F, 3F, 3F
0040, 40, 40, 40, 40
007F, 7F, 7F, 7F, 7F
0080, C2 80, C2 80, C2 80, C2 80
00BF, C2 BF, C2 BF, C2 BF, C2 BF
00C0, C3 80, C3 80, C3 80, C3 80
00FF, C3 BF, C3 BF, C3 BF, C3 BF
0100, C4 80, C4 80, C4 80, C4 80
3FFF, E3 BF BF, E3 BF BF, E3 BF BF, E3 BF BF
4000, E4 80 80, E4 80 80, E4 80 80, E4 80 80
7FFF, E7 BF BF, E7 BF BF, E7 BF BF, E7 BF BF
8000, E8 80 80, E8 80 80, E8 80 80, E8 80 80

```

```
BFFF, EB BF BF, EB BF BF, EB BF BF, EB BF BF
C000, EC 80 80, EC 80 80, EC 80 80, EC 80 80
EFFF, EE BF BF, EE BF BF, EE BF BF, EE BF BF
F000, EF 80 80, EF 80 80, EF 80 80, EF 80 80
FFFF, EF BF BF, EF BF BF, EF BF BF, EF BF BF
```

UTF-8 generates multiple bytes sequences, starting with one byte (8 bits).

Let's try another Unicode related encoding, UTF-16:

```
UTF-16 encoding:
Char, String, Writer, Charset, Encoder
0000, FE FF 00 00, FE FF 00 00, FE FF 00 00, FE FF 00 00
003F, FE FF 00 3F, FE FF 00 3F, FE FF 00 3F, FE FF 00 3F
0040, FE FF 00 40, FE FF 00 40, FE FF 00 40, FE FF 00 40
007F, FE FF 00 7F, FE FF 00 7F, FE FF 00 7F, FE FF 00 7F
0080, FE FF 00 80, FE FF 00 80, FE FF 00 80, FE FF 00 80
00BF, FE FF 00 BF, FE FF 00 BF, FE FF 00 BF, FE FF 00 BF
00C0, FE FF 00 C0, FE FF 00 C0, FE FF 00 C0, FE FF 00 C0
00FF, FE FF 00 FF, FE FF 00 FF, FE FF 00 FF, FE FF 00 FF
0100, FE FF 01 00, FE FF 01 00, FE FF 01 00, FE FF 01 00
3FFF, FE FF 3F FF, FE FF 3F FF, FE FF 3F FF, FE FF 3F FF
4000, FE FF 40 00, FE FF 40 00, FE FF 40 00, FE FF 40 00
7FFF, FE FF 7F FF, FE FF 7F FF, FE FF 7F FF, FE FF 7F FF
8000, FE FF 80 00, FE FF 80 00, FE FF 80 00, FE FF 80 00
BFFF, FE FF BF FF, FE FF BF FF, FE FF BF FF, FE FF BF FF
C000, FE FF C0 00, FE FF C0 00, FE FF C0 00, FE FF C0 00
EFFF, FE FF EF FF, FE FF EF FF, FE FF EF FF, FE FF EF FF
F000, FE FF F0 00, FE FF F0 00, FE FF F0 00, FE FF F0 00
FFFF, FE FF FF FF, FE FF FF FF, FE FF FF FF, FE FF FF FF
```

This is a surprise to me. Why UTF-16 generates 32-bit sequences? Why not call it UTF32? I found the answer later on: 0xFEFF is a flag indicates that the following byte sequence is in UTF-16BE (Big Endian) format.

How about encoding, UTF16-BE:

```
UTF-16BE encoding:
Char, String, Writer, Charset, Encoder
0000, 00 00, 00 00, 00 00, 00 00
003F, 00 3F, 00 3F, 00 3F, 00 3F
0040, 00 40, 00 40, 00 40, 00 40
007F, 00 7F, 00 7F, 00 7F, 00 7F
0080, 00 80, 00 80, 00 80, 00 80
00BF, 00 BF, 00 BF, 00 BF, 00 BF
00C0, 00 C0, 00 C0, 00 C0, 00 C0
00FF, 00 FF, 00 FF, 00 FF, 00 FF
0100, 01 00, 01 00, 01 00, 01 00
3FFF, 3F FF, 3F FF, 3F FF, 3F FF
4000, 40 00, 40 00, 40 00, 40 00
7FFF, 7F FF, 7F FF, 7F FF, 7F FF
8000, 80 00, 80 00, 80 00, 80 00
BFFF, BF FF, BF FF, BF FF, BF FF
C000, C0 00, C0 00, C0 00, C0 00
EFFF, EF FF, EF FF, EF FF, EF FF
F000, F0 00, F0 00, F0 00, F0 00
FFFF, FF FF, FF FF, FF FF, FF FF
```

This seems to be the perfect encoding, output seems to be identical to input.

Let's try an encoding related to Chinese characters, GB18030:

```
GB18030 encoding:
Char, String, Writer, Charset, Encoder
0000, 00, 00, 00, 00
003F, 3F, 3F, 3F, 3F
0040, 40, 40, 40, 40
007F, 7F, 7F, 7F, 7F
0080, 81 30 81 30, 81 30 81 30, 81 30 81 30, 81 30 81 30
00BF, 81 30 86 37, 81 30 86 37, 81 30 86 37, 81 30 86 37
00C0, 81 30 86 38, 81 30 86 38, 81 30 86 38, 81 30 86 38
00FF, 81 30 8B 37, 81 30 8B 37, 81 30 8B 37, 81 30 8B 37
0100, 81 30 8B 38, 81 30 8B 38, 81 30 8B 38, 81 30 8B 38
3FFF, 82 32 A6 36, 82 32 A6 36, 82 32 A6 36, 82 32 A6 36
4000, 82 32 A6 37, 82 32 A6 37, 82 32 A6 37, 82 32 A6 37
7FFF, C2 52, C2 52, C2 52, C2 52
8000, D2 AB, D2 AB, D2 AB, D2 AB
BFFF, 83 31 D7 34, 83 31 D7 34, 83 31 D7 34, 83 31 D7 34
C000, 83 31 D7 35, 83 31 D7 35, 83 31 D7 35, 83 31 D7 35
EFFF, 83 38 96 36, 83 38 96 36, 83 38 96 36, 83 38 96 36
F000, 83 38 96 37, 83 38 96 37, 83 38 96 37, 83 38 96 37
FFFF, 84 31 A4 39, 84 31 A4 39, 84 31 A4 39, 84 31 A4 39
```

It looks complicate.

I think that's enough. You can run the program with any of the supported encodings as an argument yourself.

Methods to Decode Byte Sequences

There are 4 methods to decode characters:

- `CharsetDecoder.decode()`
- `Charset.decode()`
- `new String()`
- `InputStreamReader.read()`

The ways to use those methods are similar to the encode methods.

Exercise: Find out what is the default 'Charset' used in the 'String' class.

Encoding Maps

Notes and sample codes bellow are based on J2SDK 1.4.1_01.

Encoding Map Analyzer

As mentioned in my other note, "Character Set and Encoding", J2SDK 1.4.1_01 for Windows 2000 provides 48 build-in encodings.

I have the following program to analyze a given encoding and print a map between the code points (from 0x0000 to 0xFFFF) and the encoded byte sequences:

```
/**
 * EncodingAnalyzer.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
class EncodingAnalyzer {
    static char hexDigit[] = {'0', '1', '2', '3', '4', '5', '6', '7',
                              '8', '9', 'A', 'B', 'C', 'D', 'E', 'F'};
    public static void main(String[] a) {
        String charset = null;
        if (a.length>0) charset = a[0];
        if (charset==null) System.out.println("Default encoding:");
        else System.out.println(charset+" encoding:");
        int lastByte = 0;
        int lastLength = 0;
        byte[] startSequence = null;
        char startChar = 0;
        byte[] endSequence = null;
        char endChar = 0;
        boolean isFirstChar = true;
        for (int i=0; i<0x010000; i++) {
            char c = (char) i;
            String s = String.valueOf(c);
            byte[] b = null;
            if (charset==null) {
                b = s.getBytes();
            } else {
                try {
                    b = s.getBytes(charset);
                } catch (UnsupportedEncodingException e) {
                    System.out.println(e.toString());
                    break;
                }
            }
            int l = b.length;
            int lb = ((int) b[l-1]) & 0x00FF;
            if (isFirstChar==true) {
                isFirstChar = false;
                startSequence = b;
                startChar = c;
                lastByte = lb - 1;
                lastLength = l;
            }
            if (!(l==lastLength && (lb==lastByte+1 || lb==lastByte))) {
```

```

        System.out.print(charToHex(startChar)+" >");
        printBytes(startSequence);
        System.out.print(" - "+charToHex(endChar)+" >");
        printBytes(endSequence);
        System.out.println("");
        startSequence = b;
        startChar = c;
    }
    endSequence = b;
    endChar = c;
    lastLength = l;
    lastByte = lb;
}
System.out.print(charToHex(startChar)+" >");
printBytes(startSequence);
System.out.print(" - "+charToHex(endChar)+" >");
printBytes(endSequence);
System.out.println("");
}
public static void printBytes(byte[] b) {
    for (int j=0; j<b.length; j++)
        System.out.print(" "+byteToHex(b[j]));
}
public static String byteToHex(byte b) {
    char[] a = { hexDigit[(b >> 4) & 0x0f], hexDigit[b & 0x0f] };
    return new String(a);
}
public static String charToHex(char c) {
    byte hi = (byte) (c >>> 8);
    byte lo = (byte) (c & 0xff);
    return byteToHex(hi) + byteToHex(lo);
}
}

```

Note that:

- String.getBytes() is used to encode the code points stored as "char" type.
- Since Java can only encode code points in the 0x0000 - 0xFFFF range, only a subset of the character set will be encoded for some encodings, like UTF-8, which can encode code points upto 0x10FFFF.
- The encoding name should be specified as command argument.

The output of this program will be discussed in the sections bellow.

US-ASCII

US-ASCII encoding:

Code Point	Code Point
0000 > 00	007F > 7F
0080 > 3F	FFFF > 3F

- This is a very simple map.
- The encoded byte sequence is one byte only, taking the lower value byte of the code point.

- Valid code points only in the 0x0000 - 0x007F range.

ISO-8859-1 - Latin 1

ISO-8859-1 encoding:

Code Point	Code Point
0000 > 00	- 00FF > FF
0100 > 3F	- FFFF > 3F

- This is also a very simple map.
- The encoded byte sequence is one byte only, taking the lower value byte of the code point.
- Valid code points only in the 0x0000 - 0x00FF range, full range of the lower value byte.

CP1252 - Windows-1252

CP1252 encoding:

Code Point	Code Point
0000 > 00	- 007F > 7F
0080 > 3F	- 009F > 3F
00A0 > A0	- 00FF > FF
0100 > 3F	- 0151 > 3F
0152 > 8C	- 0152 > 8C
0153 > 9C	- 0153 > 9C
0154 > 3F	- 015F > 3F
0160 > 8A	- 0160 > 8A
0161 > 9A	- 0161 > 9A
0162 > 3F	- 0177 > 3F
0178 > 9F	- 0178 > 9F
0179 > 3F	- 017C > 3F
017D > 8E	- 017D > 8E
017E > 9E	- 017E > 9E
017F > 3F	- 0191 > 3F
0192 > 83	- 0192 > 83
0193 > 3F	- 02C5 > 3F
02C6 > 88	- 02C6 > 88
02C7 > 3F	- 02DB > 3F
02DC > 98	- 02DC > 98
02DD > 3F	- 2012 > 3F
2013 > 96	- 2014 > 97
2015 > 3F	- 2017 > 3F
2018 > 91	- 2019 > 92
201A > 82	- 201A > 82
201B > 3F	- 201B > 3F
201C > 93	- 201D > 94
201E > 84	- 201E > 84
201F > 3F	- 201F > 3F
2020 > 86	- 2021 > 87
2022 > 95	- 2022 > 95

```

2023 > 3F - 2025 > 3F
2026 > 85 - 2026 > 85
2027 > 3F - 202F > 3F
2030 > 89 - 2030 > 89
2031 > 3F - 2038 > 3F
2039 > 8B - 2039 > 8B
203A > 9B - 203A > 9B
203B > 3F - 20AB > 3F
20AC > 80 - 20AC > 80
20AD > 3F - 2121 > 3F
2122 > 99 - 2122 > 99
2123 > 3F - FFFF > 3F

```

- This is the default encoding used by HotSpot for Windows.
- The output sequence is always one byte.
- It is backward compatible with US-ASCII.
- Only one section, 0x00A0 - 0x00FF, in the 0x0080 - 0x00FF range is valid.
- Only a small number of code points in the 0x0100 - 0xFFFF range is valid.

UTF-8

UTF-8 encoding:

```

Code Point      Code Point
0000 > 00 - 007F > 7F
0080 > C2 80 - 00BF > C2 BF
00C0 > C3 80 - 00FF > C3 BF
0100 > C4 80 - 013F > C4 BF
.....
07C0 > DF 80 - 07FF > DF BF
0800 > E0 A0 80 - 083F > E0 A0 BF
0840 > E0 A1 80 - 087F > E0 A1 BF
0880 > E0 A2 80 - 08BF > E0 A2 BF
.....
0FC0 > E0 BF 80 - 0FFF > E0 BF BF
1000 > E1 80 80 - 103F > E1 80 BF
1040 > E1 81 80 - 107F > E1 81 BF
1080 > E1 82 80 - 10BF > E1 82 BF
.....
1FC0 > E1 BF 80 - 1FFF > E1 BF BF
2000 > E2 80 80 - 203F > E2 80 BF
2040 > E2 81 80 - 207F > E2 81 BF
2080 > E2 82 80 - 20BF > E2 82 BF
.....
2FC0 > E2 BF 80 - 2FFF > E2 BF BF
3000 > E3 80 80 - 303F > E3 80 BF
3040 > E3 81 80 - 307F > E3 81 BF
3080 > E3 82 80 - 30BF > E3 82 BF
.....
3FC0 > E3 BF 80 - 3FFF > E3 BF BF
4000 > E4 80 80 - 403F > E4 80 BF
4040 > E4 81 80 - 407F > E4 81 BF
4080 > E4 82 80 - 40BF > E4 82 BF
.....
4FC0 > E4 BF 80 - 4FFF > E4 BF BF
5000 > E5 80 80 - 503F > E5 80 BF

```

```

5040 > E5 81 80 - 507F > E5 81 BF
5080 > E5 82 80 - 50BF > E5 82 BF
.....
5FC0 > E5 BF 80 - 5FFF > E5 BF BF
6000 > E6 80 80 - 603F > E6 80 BF
6040 > E6 81 80 - 607F > E6 81 BF
6080 > E6 82 80 - 60BF > E6 82 BF
.....
6FC0 > E6 BF 80 - 6FFF > E6 BF BF
7000 > E7 80 80 - 703F > E7 80 BF
7040 > E7 81 80 - 707F > E7 81 BF
7080 > E7 82 80 - 70BF > E7 82 BF
.....
7FC0 > E7 BF 80 - 7FFF > E7 BF BF
8000 > E8 80 80 - 803F > E8 80 BF
8040 > E8 81 80 - 807F > E8 81 BF
8080 > E8 82 80 - 80BF > E8 82 BF
.....
8FC0 > E8 BF 80 - 8FFF > E8 BF BF
9000 > E9 80 80 - 903F > E9 80 BF
9040 > E9 81 80 - 907F > E9 81 BF
9080 > E9 82 80 - 90BF > E9 82 BF
.....
9FC0 > E9 BF 80 - 9FFF > E9 BF BF
A000 > EA 80 80 - A03F > EA 80 BF
A040 > EA 81 80 - A07F > EA 81 BF
A080 > EA 82 80 - A0BF > EA 82 BF
.....
AFC0 > EA BF 80 - AFFF > EA BF BF
B000 > EB 80 80 - B03F > EB 80 BF
B040 > EB 81 80 - B07F > EB 81 BF
B080 > EB 82 80 - B0BF > EB 82 BF
.....
BFC0 > EB BF 80 - BFFF > EB BF BF
C000 > EC 80 80 - C03F > EC 80 BF
C040 > EC 81 80 - C07F > EC 81 BF
C080 > EC 82 80 - C0BF > EC 82 BF
.....
CFC0 > EC BF 80 - CFFF > EC BF BF
D000 > ED 80 80 - D03F > ED 80 BF
D040 > ED 81 80 - D07F > ED 81 BF
D080 > ED 82 80 - D0BF > ED 82 BF
.....
D7C0 > ED 9F 80 - D7FF > ED 9F BF
D800 > 3F - DFFF > 3F
E000 > EE 80 80 - E03F > EE 80 BF
E040 > EE 81 80 - E07F > EE 81 BF
E080 > EE 82 80 - E0BF > EE 82 BF
.....
EFC0 > EE BF 80 - EFFF > EE BF BF
F000 > EF 80 80 - F03F > EF 80 BF
F040 > EF 81 80 - F07F > EF 81 BF
F080 > EF 82 80 - F0BF > EF 82 BF
.....
FFC0 > EF BF 80 - FFFF > EF BF BF

```

- This is the most popular encoding used for the Unicode character set.
- The output sequence has variable number of bytes.
- It is backward compatible with US-ASCII.
- This map only valid for Unicode 3.0 and older versions. So it is a partial UTF-8 map.

- One section of code points is not valid: 0xD800 - 0xDFFF.

UTF-16

UTF-16 encoding:

Code Point		Code Point
0000	> FE FF 00 00 -	00FF > FE FF 00 FF
0100	> FE FF 01 00 -	01FF > FE FF 01 FF
0200	> FE FF 02 00 -	02FF > FE FF 02 FF
.....		
D700	> FE FF D7 00 -	D7FF > FE FF D7 FF
D800	> FE FF FF FD -	DFFF > FE FF FF FD
E000	> FE FF E0 00 -	E0FF > FE FF E0 FF
E100	> FE FF E1 00 -	E1FF > FE FF E1 FF
E200	> FE FF E2 00 -	E2FF > FE FF E2 FF
.....		
FF00	> FE FF FF 00 -	FFFF > FE FF FF FF

- This is another encoding used for the Unicode character set.
- The output sequence is a fixed length, 2 bytes. Note that the leading 0xFEFF is a format flag.
- It is not backward compatible with US-ASCII.
- One section of code points is not valid: 0xD800 - 0xDFFF.

UTF-16LE

UTF-16LE encoding:

Code Point		Code Point
0000	> 00 00 -	D7FF > FF D7
D800	> FD FF -	DFFF > FD FF
E000	> 00 E0 -	FFFF > FF FF

- This is another encoding used for the Unicode character set.
- The output sequence is a fixed length, 2 bytes.
- It is not backward compatible with US-ASCII.
- One section of code points is not valid: 0xD800 - 0xDFFF.
- The rest of the code points is encoded by reversing the two bytes of the code points.

UTF-16BE

UTF-16BE encoding:

Code Point		Code Point
0000	> 00 00 -	00FF > 00 FF

```
0100 > 01 00 - 01FF > 01 FF
0200 > 02 00 - 02FF > 02 FF
.....
D700 > D7 00 - D7FF > D7 FF
D800 > FF FD - DFFF > FF FD
E000 > E0 00 - E0FF > E0 FF
E100 > E1 00 - E1FF > E1 FF
E200 > E2 00 - E2FF > E2 FF
.....
FF00 > FF 00 - FFFF > FF FF
```

- This is another encoding used for the Unicode character set.
- The output sequence is a fixed length, 2 bytes.
- It is not backward compatible with US-ASCII.
- One section of code points is not valid: 0xD800 - 0xDFFF.
- The rest of the code points is encoded by carbon copying the two bytes of the code points.

Encoding Map Counts

Notes and sample codes bellow are based on J2SDK 1.4.1_01.

Encoding Map Counter

As mentioned in my other note, "Character Set and Encoding", J2SDK 1.4.1_01 for Windows 2000 provides 48 build-in encodings.

I have the following program to count the number of mappable code points in the 0x0000 - 0xFFFF range for a given encoding:

```
/**
 * EncodingCounter.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.nio.*;
import java.nio.charset.*;
class EncodingCounter {
    static char hexDigit[] = {'0', '1', '2', '3', '4', '5', '6', '7',
                              '8', '9', 'A', 'B', 'C', 'D', 'E', 'F'};
    public static void main(String[] a) {
        String charset = "CP1252";
        if (a.length>0) charset = a[0];
        System.out.println(charset+" encoding:");
        int lastByte = 0;
        int lastLength = 0;
        byte[] startSequence = null;
        char startChar = 0;
        byte[] endSequence = null;
        char endChar = 0;
        boolean isFirstChar = true;
        int validCount = 0;
        int subCount = 0;
        int totalCount = 0x010000;
        for (int i=0; i<totalCount; i++) {
            subCount++;
            char c = (char) i;
            byte[] b = encodeByEncoder(c, charset);
            int l = 0;
            int lb = 0;
            if (b!=null) {
                l = b.length;
                lb = ((int) b[l-1]) & 0x00FF;
                validCount++;
            }
            if (isFirstChar==true) {
                isFirstChar = false;
                startSequence = b;
                startChar = c;
                lastByte = lb - 1;
                lastLength = l;
            }
            if (!(l==lastLength)) {
                System.out.print(charToHex(startChar)+" >");
            }
        }
    }
}
```

```

        printBytes(startSequence);
        System.out.print(" - "+charToHex(endChar)+" >");
        printBytes(endSequence);
        System.out.println(" = "+(subCount-1));
        startSequence = b;
        startChar = c;
        subCount = 1;
    }
    endSequence = b;
    endChar = c;
    lastLength = l;
    lastByte = lb;
}
System.out.print(charToHex(startChar)+" >");
printBytes(startSequence);
System.out.print(" - "+charToHex(endChar)+" >");
printBytes(endSequence);
System.out.println(" = "+(subCount));
System.out.println("Total characters = "+totalCount);
System.out.println("Valid characters = "+validCount);
System.out.println("Invalid characters = "
    +(totalCount-validCount));
}
public static byte[] encodeByEncoder(char c, String cs) {
    Charset cso = null;
    byte[] b = null;
    try {
        cso = Charset.forName(cs);
        CharsetParameter e = cso.newEncoder();
        e.reset();
        ByteBuffer bb = e.encode(CharBuffer.wrap(new char[] {c}));
        if (bb.limit()>0) b = copyBytes(bb.array(),bb.limit());
    } catch (IllegalCharsetParameter e) {
        System.out.println(e.toString());
    } catch (CharsetParameterException e) {
        // invalid character, return null
    }
    return b;
}
public static void printBytes(byte[] b) {
    if (b!=null) {
        for (int j=0; j<b.length; j++)
            System.out.print(" "+byteToHex(b[j]));
    } else {
        System.out.print(" XX");
    }
}
public static byte[] copyBytes(byte[] a, int l) {
    byte[] b = new byte[l];
    for (int i=0; i<Math.min(l,a.length); i++) b[i] = a[i];
    return b;
}
public static String byteToHex(byte b) {
    char[] a = { hexDigit[(b >> 4) & 0x0f], hexDigit[b & 0x0f] };
    return new String(a);
}
public static String charToHex(char c) {
    byte hi = (byte) (c >>> 8);
    byte lo = (byte) (c & 0xff);
    return byteToHex(hi) + byteToHex(lo);
}
}

```

Note that:

- `CharsetEncoder.encode()` is used to encode the code points stored as "char" type.
- Since Java can only encode code points in the 0x0000 - 0xFFFF range, only a subset of the character set will be encoded for some encodings, like UTF-8, which can encode code points up to 0x10FFFF.
- The encoding name should be specified as command argument.

Run this program with US-ASCII as argument, you will get:

```
US-ASCII encoding:
0000 > 00 - 007F > 7F = 128
0080 > XX - FFFF > XX = 65408
Total characters = 65536
Valid characters = 128
Invalid characters = 65408
```

This tells us that the US-ASCII character set has only 128 characters.

Run this program with ISO-8859-1 (Latin 1) as argument, you will get:

```
ISO-8859-1 encoding:
0000 > 00 - 00FF > FF = 256
0100 > XX - FFFF > XX = 65280
Total characters = 65536
Valid characters = 256
Invalid characters = 65280
```

This tells us that the US-ASCII character set has only 256 characters.

Comparison of Encoding Maps

The following table is based on the output of the EncodingCounter program with different supported encoding names. It provides a brief comparison between the some different encodings.

Encoding Name	Map Size	US-ASCII Compatible	Notes
US-ASCII	128	Y	7-bit characters only
ISO-8859-1	256	Y	8-bit (single byte) characters
CP1252	251	Y	One byte output, with code points up to 0x2122
UTF-8	63488	Y	1-3 bytes,
UTF-16BE	63488	N	2 bytes, carbon copying the code points
UTF-16LE	63488	N	2 bytes, reversing the code points
UTF-16	63488	N	4 bytes, last 2 bytes = UTF-16BE
GBK	24068	Y	1-2 bytes, Chinese 1993 standard
GB18030	63488	Y	1-4 bytes, superset of GBK, 2000 standard

Encoding Conversion

Notes and sample codes bellow are based on J2SDK 1.4.1_01.

Unicode Data Entry

Encoding conversion is about reading characters stored in a file encoded with encoding A, and writing them into another file encoded with encoding B.

Before going into details of encoding conversion, let's talk briefly about Unicode data entry. How do we enter Unicode characters into a file? There are a couple of ways to do that:

- Using encoding specific word processors. Usually, one word processor will allow you to enter characters of a particular language.
- Using Hex editors to enter directly the byte sequences representing the desired characters in a specific encoding.
- Using Unicode based programming language to enter the desired characters as string literals.

Word processors are too specific to be discussed here.

Hex editors are ultimate data entry tools for Unicode characters. They can also be used to inspect and repair encoded text files. But Hex editors are very hard to use.

Notepad on Windows is not a Hex editor.

UltraEdit on Windows is a Hex editor.

Using Unicode based programming language, like Java, to enter Unicode characters into a file is very interesting. Here is a sample program, UnicodeHello.java:

```
/**
 * UnicodeHello.java
 * Copyright (c) 2002 by Dr. Herong Yang
 *
 * This program is a simple tool to allow you to enter several lines of
 * text, and writh them into a file with of the specified encoding
 * (charset name). The input text lines uses Java string convention,
 * which allows you to enter ASCII characters directly, and any non
 * ASCII characters with escape sequences.
 *
 * This version of the program is to write out the "Hello world!"
 * message in some different languages.
 */
import java.io.*;
class UnicodeHello {
    public static void main(String[] a) {
        // The following Array contains text to be saved into the output
        // File. To enter your own text, just replace this Array.
```

```

        String[] text = {
            "Hello computer! - English", // ASCII
            "\u7535\u8111\u4F60\u597D\uFF01 - Simplified Chinese", // GB2312
            "\u96FB\u8166\u4F60\u597D\uFE57 - Traditional Chinese" // Big5
        };
        String outFile = "hello.utf-16be";
        if (a.length>0) outFile = a[0];
        String outCharset = "utf-16be";
        if (a.length>1) outCharset = a[1];
        String crlf = System.getProperty("line.separator");
        try {
            OutputStreamWriter out = new OutputStreamWriter(
                new FileOutputStream(outFile), outCharset);
            for (int i=0; i<text.length; i++) {
                out.write(text[i]);
                out.write(crlf);
            }
            out.close();
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
}

```

As you can see from the source code, this program will write the "Hello computer!" message in several languages. Let's compile this program and run it to get the characters saved into a file with UTF-8 encoding:

```

javac UnicodeHello.java
java UnicodeHello hello.utf-16be utf-16be

```

Since the text file contains non-ASCII characters, we need to convert it into Hex decimal digits to be able check the code values of the saved characters. Remember UTF-16BE encoding breaks the code values into two bytes directly without any changes. Here is a program to convert any data file into Hex decimal digits:

```

/**
 * HexWriter.java
 * Copyright (c) 2002 by Dr. Herong Yang
 * This program allows you to convert and data file to a new data
 * in Hex format with 16 bytes (32 Hex digits) per line.
 */
import java.io.*;
class HexWriter {
    static char hexDigit[] = {'0', '1', '2', '3', '4', '5', '6', '7',
                              '8', '9', 'A', 'B', 'C', 'D', 'E', 'F'};
    public static void main(String[] a) {
        String inFile = a[0];
        String outFile = a[1];
        int bufSize = 16;
        byte[] buffer = new byte[bufSize];
        String crlf = System.getProperty("line.separator");
        try {
            FileInputStream in = new FileInputStream(inFile);
            OutputStreamWriter out = new OutputStreamWriter(
                new FileOutputStream(outFile));
            int n = in.read(buffer,0,bufSize);
            String s = null;
            int count = 0;

```

```

        while (n!=-1) {
            count += n;
            s = bytesToHex(buffer,0,n);
            out.write(s);
            out.write(crlf);
            n = in.read(buffer,0,bufSize);
        }
        in.close();
        out.close();
        System.out.println("Number of input bytes: "+count);
    } catch (IOException e) {
        System.out.println(e.toString());
    }
}

public static String bytesToHex(byte[] b, int off, int len) {
    StringBuffer buf = new StringBuffer();
    for (int j=0; j<len; j++)
        buf.append(byteToHex(b[off+j]));
    return buf.toString();
}

public static String byteToHex(byte b) {
    char[] a = { hexDigit[(b >> 4) & 0x0f], hexDigit[b & 0x0f] };
    return new String(a);
}
}

```

Compile this program and run it to convert hello.utf-16be:

```

javac HexWriter.java
java HexWriter hello.utf-16be hello.hex

```

Okay, here is the content of hello.hex:

```

00480065006C006C006F00200063006F
006D0070007500740065007200210020
002D00200045006E0067006C00690073
0068000D000A753581114F60597DFF01
0020002D002000530069006D0070006C
00690066006900650064002000430068
0069006E006500730065000D000A96FB
81664F60597DFE570020002D00200054
007200610064006900740069006F006E
0061006C0020004300680069006E0065
00730065000D000A

```

If you know how to read Hex number, you should be able to see:

- "00480065006C006C006F" represents "Hello".
- "753581114F60597DFF01" represents the Simplified Chinese message.
- "96FB81664F60597DFE57" represents the Traditional Chinese message.

Unicode Encoding Conversion

Now we have a text file with Unicode characters. Let's write an encoding conversion program:

```

/**

```

```
* EncodingConverter.java
* Copyright (c) 2002 by Dr. Herong Yang
*
* This program allows you to convert a text file in one encoding
* to another file in a different encoding.
*/
import java.io.*;
class EncodingConverter {
    public static void main(String[] a) {
        String inFile = a[0];
        String inCharset = a[1];
        String outFile = a[2];
        String outCharset = a[3];
        try {
            InputStreamReader in = new InputStreamReader(
                new FileInputStream(inFile), inCharset);
            OutputStreamWriter out = new OutputStreamWriter(
                new FileOutputStream(outFile), outCharset);
            int c = in.read();
            int n = 0;
            while (c != -1) {
                out.write(c);
                n++;
                c = in.read();
            }
            in.close();
            out.close();
            System.out.println("Number of characters: " + n);
            System.out.println("Number of input bytes: "
                + (new File(inFile)).length());
            System.out.println("Number of output bytes: "
                + (new File(outFile)).length());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
}
```

Compile this program and use it to convert our hello message file into several encodings:

```
javac EncodingConverter.java
java EncodingConverter hello.utf-16be utf-16be hello.ascii ascii
java EncodingConverter hello.utf-16be utf-16be hello.iso-8859-1 iso-...
java EncodingConverter hello.utf-16be utf-16be hello.utf-8 utf-8
java EncodingConverter hello.utf-16be utf-16be hello.gbk gbk
java EncodingConverter hello.utf-16be utf-16be hello.big5 big5
java EncodingConverter hello.utf-16be utf-16be hello.shift_jis shift_jis
```

By observing the output files, you should notice the followings:

hello.ascii - In this file, only the English message is good, because it contains only ASCII characters. Both Simplified Chinese and Traditional Chinese messages are not good. Characters in these messages are replaced by 0x3F, an indication of invalid code.

hello.iso-8859-1 - This is identical to hello.ascii, because there is no characters in the 0x80 - 0xFF range.

hello.utf-8 - This file contains all messages with no damages. The ASCII characters are

stored as one-byte characters as expected.

hello.gbk - In this file, the Simplified Chinese message is good. In fact, characters in the Simplified Chinese message are stored as code values in GBK character set standard. The English message is also good, because GBK is ASCII backward compatible. We are lucky with the Traditional Chinese message, because the Big5 characters used in the message are also valid in GBK standard. If you use some Big5 special characters, the result could be different.

hello.big5 - In this file, the Traditional Chinese message is good. In fact, characters in the Traditional Chinese message are stored as code values in Big5 character set standard. The English message is also good, because Big5 is ASCII backward compatible. We are not lucky with the Simplified Chinese message, two GB characters used in the message are not valid in Big5 standard. 0x3F was stored for those characters.

hello.shift_jis - In this file, the English message is still good. Some of the characters from both Simplified and Traditional Chinese messages are invalid, replaced by 0x3F placeholders. Some of the Chinese characters are still valid in Shift_JIS character set. This is not so surprising, because there are many shared characters in Chinese and Japanese.

Viewing Unicode Text

Now, we have this greeting messages saved in many different encodings. The next question is how do display them as glyph of the corresponding languages on the screen. One of the ways I have used in the past is to run a multi-language enabled Web browser like IE to view the text files. To do this, we have to mark up the text into a html file, by using program like this one here:

```
/**
 * EncodingHtml.java
 * Copyright (c) 2002 by Dr. Herong Yang
 *
 * This program allows you to mark up a text file into html file.
 */
import java.io.*;
import java.util.*;
class EncodingHtml {
    static HashMap charsetMap = new HashMap();
    public static void main(String[] a) {
        String inFile = a[0];
        String inCharset = a[1];
        String outFile = inFile + ".html";
        try {
            InputStreamReader in = new InputStreamReader(
                new FileInputStream(inFile), inCharset);
            OutputStreamWriter out = new OutputStreamWriter(
                new FileOutputStream(outFile), inCharset);
            writeHead(out, inCharset);
            int c = in.read();
            int n = 0;
```

```
        while (c!=-1) {
            out.write(c);
            n++;
            c = in.read();
        }
        writeTail(out);
        in.close();
        out.close();
        System.out.println("Number of characters: "+n);
    } catch (IOException e) {
        System.out.println(e.toString());
    }
}

public static void writeHead(OutputStreamWriter out, String cs)
    throws IOException {
    out.write("<html><head>\n");
    out.write("<meta http-equiv=\"Content-Type\"+
        \" content=\"text/html; charset="+cs+"\">\n");
    out.write("</head><body><pre>");
}

public static void writeTail(OutputStreamWriter out)
    throws IOException {
    out.write("</pre></body></html>\n");
}
}
```

Now, let's compile this program and run it with hello.utf-8:

```
javac EncodingHtml.java
java EncodingHtml hello.utf-8 utf-8
```

If you have installed IE with the Chinese language supports, you should be able to open the output file, hello.utf-8.html, and enjoy reading the messages in English, Simplified Chinese, and Traditional Chinese.

Then, run EncodingHtml.java with other encodings,

```
java EncodingHtml hello.gbk gbk
java EncodingHtml hello.big5 big5
java EncodingHtml hello.shift_jis shift_jis
```

View the output files with IE, and compare the results:

- hello.utf-8.html - IE auto sets View/Encoding to utf-8. All messages are perfect.
- hello.gbk.html - IE auto sets View/Encoding to gb2312. All messages are perfect.
- hello.big5.html - IE auto sets View/Encoding to big5. Simplified Chinese message has two bad characters.
- hello.shift_jis - IE auto sets View/Encoding to shift_jis. Both Simplified and Traditional Chinese messages have bad characters.

If you manually change the setting of View/Encoding, IE will not be able to show the message with the right glyph.

Unicode Signs in Different Encodings

I wanted to play with my utility programs mentioned in this note one more time with this some Unicode signs. So I copied UnicodeHello.java and made UnicodeSign.java:

```
/**
 * UnicodeSign.java
 * Copyright (c) 2002 by Dr. Herong Yang
 *
 * This program is a simple tool to allow you to enter several lines of
 * text, and writh them into a file with of the specified encoding
 * (charset name). The input text lines uses Java string convention,
 * which allows you to enter ASCII characters directly, and any non
 * ASCII characters with escape sequences.
 *
 * This version of the program is to write out some interesting signs.
 */
import java.io.*;
class UnicodeSign {
    public static void main(String[] a) {
        // The following Array contains text to be saved into the output
        // File. To enter your own text, just replace this Array.
        String[] text = {
            "U+005C(\\)REVERSE SOLIDUS", //\u005C is '\', cannot be entered directly
            "U+007E(\u007E)TILDE",
            "U+00A2(\u00A2)CENT SIGN",
            "U+00A3(\u00A3)POUND SING",
            "U+00A5(\u00A5)YEN SIGN",
            "U+00A6(\u00A6)BROKEN BAR",
            "U+00A7(\u00A7)SECTION SIGN",
            "U+00A9(\u00A9)COPYRIGHT SIGN",
            "U+00AC(\u00AC)NOT SIGN",
            "U+00AE(\u00AE)REGISTERED SIGN",
            "U+2022(\u2022)BULLET",
            "U+2023(\u2023)TRIANGULAR BULLET",
            "U+203B(\u203B)REFERENCE MARK",
            "U+2043(\u2043)HYPHEN BULLET",
            "U+FF04(\uFF04)FULLWIDTH DOLLAR SIGN",
            "U+FF05(\uFF05)FULLWIDTH PERCENT SIGN",
            "U+FF08(\uFF08)FULLWIDTH LEFT PARENTHESIS",
            "U+FF09(\uFF09)FULLWIDTH RIGHT PARENTHESIS",
            "U+FF10(\uFF10)FULLWIDTH DIGIT ZERO",
            "U+FF11(\uFF11)FULLWIDTH DIGIT ONE",
            "U+FF21(\uFF21)FULLWIDTH LATIN CAPITAL LETTER A",
            "U+FF22(\uFF22)FULLWIDTH LATIN CAPITAL LETTER B",
            "U+FF41(\uFF41)FULLWIDTH LATIN SMALL LETTER A",
            "U+FF42(\uFF42)FULLWIDTH LATIN SMALL LETTER B",
            "U+FFE0(\uFFE0)FULLWIDTH CENT SIGN",
            "U+FFE1(\uFFE1)FULLWIDTH POND SIGN",
            "U+FFE5(\uFFE5)FULLWIDTH YEN SIGN"
        };
        String outFile = "sign.utf-16be";
        if (a.length>0) outFile = a[0];
        String outCharsetName = "utf-16be";
        if (a.length>1) outCharsetName = a[1];
        String crlf = System.getProperty("line.separator");
        try {
            OutputStreamWriter out = new OutputStreamWriter(
                new FileOutputStream(outFile), outCharsetName);
            for (int i=0; i<text.length; i++) {
                out.write(text[i]);
                out.write(crlf);
            }
        }
    }
}
```

```
        }  
        out.close();  
    } catch (IOException e) {  
        System.out.println(e.toString());  
    }  
}  
}
```

Then I ran this program, and converted the output file with different encodings:

```
javac UnicodeSign.java  
java UnicodeSign sign.utf-16be utf-16be  
java EncodingConverter sign.utf-16be utf-16be sign.utf-8 utf-8  
java EncodingHtml sign.utf-8 utf-8  
java EncodingConverter sign.utf-16be utf-16be sign.gbk gbk  
java EncodingHtml sign.gbk gbk  
java EncodingConverter sign.utf-16be utf-16be sign.shift_jis shift_jis  
java EncodingHtml sign.shif_jis shift_jis  
java EncodingConverter sign.utf-16be utf-16be sign.johab johab  
java EncodingHtml sign.johab johab
```

Then I viewed the different encoded test files with IE, and noticed the following:

- sign.utf-8.html - The signs looked very good except two: TRIANGULAR BULLET and DASH BULLET.
- sign.gbk.html - Many low-code-point signs were wrong, like CENT SIGN.
- sign.shift_jis.html - Some signs were wrong, like FULLWIDTH CENT SIGN. but CENT SIGN is correct.
- sign.johab.html - Like the gbk encoding, many low-code-point signs were wrong, like CENT SIGN.

Conclusion:

- Java program seems to be good way of stored Unicode text into a file, if you don't have good word processor that handles Unicode text.
- When converting text file from one encoding to another, you need to make sure that all characters in the text file are valid characters in the character set of the output encoding.
- IE is a very good tool to view Unicode text, if you installed all the required fonts.

Exercise: Adding more messages in other languages in UnicodeHello.java.

Socket Communication

Notes and sample codes bellow are based on J2SDK 1.4.1_01.

What Is a Socket

Socket: An concept represents one end-point of a two-way communication link between two programs running on the Internet network.

Internet Address: A unique number to identify each computer system on the Internet.

Port: A number representing an entry point of on a computer system on the Internet, where a two-way communication link can be established.

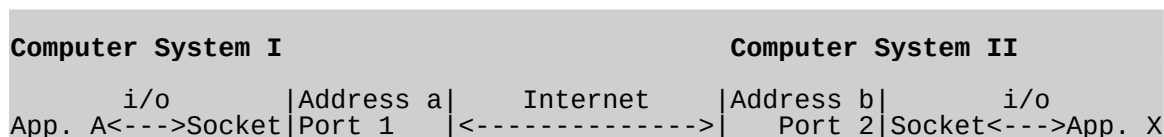
Local System: The computer system at this end of a two-way communication link.

Remote System: The computer system at the other end of a two-way communication link.

A two-way communication link on the Internet involves:

- Two computer systems, each has its own Internet address.
- A port number on each computer system that is used only by this communication link.
- An application program on each computer system that uses this communication link.
- A socket object created in the application program that represents this communication link.
- The application program can use the socket as a logical input or output device to receive or send data from or to the application program running at the other end of the communication link.
- The application program can also query the socket for information about the communication link, like: The Internet address and port number at this end of the communication link; The Internet address and port number at the other end of the communication link.

The follwing diagram illustrates how an application program can use a socket to talk with other application program running on another computer system throught a two-way communication link on the Internet:



Establishing a Socket Communication

In the previous section, we only discussed about how socket works with a communication link that has been established already. Now, let's see how two application programs can establish a communication link between them.

To establish a communication link, one application program must act as a server, create a server socket with a given port number, and set the server socket in the listen mode waiting for a connection request from other program.

With one program running as a server listening for a connection request at a specific port number, the other program can now create socket with a given local port number, the Internet address of the computer system where the first program is running, and the port number where the server socket is listening. At this time, a connect request will be send over to the server socket. The server socket should then accept the connect request and instantiate a socket object to complete communication link.

J2SDK offers two main classes to support socket communication:

java.net.ServerSocket with methods:

- bind(): setting the server socket with the local system address, and a given local port number.
- accept(): listening for a connection request, and instantiating a socket object when the request comes.

java.net.Socket with methods:

- bind(): setting the socket with the local system address, and a given local port number.
- connect(): sending a connection request to a given remote system address, and a given remote port number.

The following diagram shows the steps involved in establishing a two-way communication link using the methods provided by ServerSocket and Socket classes:

Step	Client System Internet Address #a Available Port #p	Server System Internet Address #b Available Port #q
1		ss = new ServerSocket()
2		ss.bind(#b+#q)
3		Socket cs = ss.accept()
4	cs = new Socket()	(waiting)
5	cs.bind(#a+#p)	(waiting)
6	cs.connect(#b+#q)	(receiving request)
7	(establishing the link)	(establishing the link)
8	(cs is ready)	(cs is ready)

ReverseEchoer - A Simple Server Socket Application

The following program called ReverseEchoer is a simple server socket application, which listens with a server socket for a connect request. Once connected, it reads what lines of

text from the remote application, reverses the text lines, and echos back to the remote application:

```
/**
 * ReverseEchoer.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
public class ReverseEchoer {
    public static void main(String[] args) {
        try {
            ServerSocket s = new ServerSocket(8888);
            printServerSocketInfo(s);
            Socket c = s.accept();
            printSocketInfo(c);
            BufferedWriter w = new BufferedWriter(new OutputStreamWriter(
                c.getOutputStream()));
            BufferedReader r = new BufferedReader(new InputStreamReader(
                c.getInputStream()));
            String m = "Welcome to Reverse Echo Server."+
                " Please type in some words.";
            w.write(m,0,m.length());
            w.newLine();
            w.flush();
            while ((m=r.readLine())!= null) {
                if (m.equals(".")) break;
                char[] a = m.toCharArray();
                int n = a.length;
                for (int i=0; i<n/2; i++) {
                    char t = a[i];
                    a[i] = a[n-1-i];
                    a[n-i-1] = t;
                }
                w.write(a,0,n);
                w.newLine();
                w.flush();
            }
            w.close();
            r.close();
            c.close();
            s.close();
        } catch (IOException e) {
            System.err.println(e.toString());
        }
    }
    private static void printSocketInfo(Socket s) {
        System.out.println("Remote address = "
            +s.getInetAddress().toString());
        System.out.println("Remote port = "
            +s.getPort());
        System.out.println("Local socket address = "
            +s.getLocalSocketAddress().toString());
        System.out.println("Local address = "
            +s.getLocalAddress().toString());
        System.out.println("Local port = "
            +s.getLocalPort());
    }
    private static void printServerSocketInfo(ServerSocket s) {
        System.out.println("Server socker address = "
```

```
        +s.getInetAddress().toString());  
        System.out.println("Server socket port = "  
        +s.getLocalPort());  
    }  
}
```

Note that:

- The ServerSocket object, s, instantiated in this program is bound to the Internet address of the computer system on which it is running. The port number, 8888, in this binding is supplied as a parameter to constructor.
- Socket.getInputStream() and Socket.getOutputStream() are used to create an input stream and output stream, so the application program can talk to the remote application through the socket object.
- If a text line received only contains a single character '.', the communication link will be terminated by the Socket.close() method.

Run ReverseEchoer, you will get the following output on the console window:

```
Server socket address = 0.0.0.0/0.0.0.0  
Server socket port = 8888
```

This tells us that the program is listening at address 0.0.0.0 and port 8888.

To test ReverseEchoer, we can use the an existing client program called telnet to initiate the communication request and talk to ReverseEchoer. Open another command window and type in the following command:

```
telnet 127.0.0.1 8888
```

Immediately, you will more output on the console window of ReverseEchoer:

```
Remote address = /127.0.0.1  
Remote port = 1068  
Local socket address = /127.0.0.1:8888  
Local address = /127.0.0.1  
Local port = 8888
```

This tells us that the server socket received a connection request, and a communication link has been established with the remote application, which the telnet program running on the same machine. The local address and remote address are the same, 127.0.0.1. The remote port is 1068, which is picked up by the telnet program.

In the telnet window, type in the following text:

```
Fish, I love you and respect you very much.  
But I will kill you dead before this day ends.  
.
```

The text will be reversed and returned back from the ReverseEchoer:

```
Welcome to Reverse Echo Server. Please type in some words.  
.hcnu yrev uoy tcepsr dna uoy evol I ,hsif
```

```
.sdne yad siht erofeb daed uoy llik lliw I tuB
.
```

Note that the telnet program is not displaying text you typed in. It only displays the text received from ReverseEchoer program.

SocketClient - A Simple Client Socket Application

Instead of using the telnet program, we can use the following simple socket application to communicate with ReverseEchoer:

```
/**
 * SocketClient.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
public class SocketClient {
    public static void main(String[] args) {
        BufferedReader in = new BufferedReader(new InputStreamReader(
            System.in));
        PrintStream out = System.out;
        try {
            Socket c = new Socket("localhost",8888);
            printSocketInfo(c);
            BufferedWriter w = new BufferedWriter(new OutputStreamWriter(
                c.getOutputStream()));
            BufferedReader r = new BufferedReader(new InputStreamReader(
                c.getInputStream()));
            String m = null;
            while ((m=r.readLine())!= null) {
                out.println(m);
                m = in.readLine();
                w.write(m,0,m.length());
                w.newLine();
                w.flush();
            }
            w.close();
            r.close();
            c.close();
        } catch (IOException e) {
            System.err.println(e.toString());
        }
    }
    private static void printSocketInfo(Socket s) {
        System.out.println("Remote address = "
            +s.getInetAddress().toString());
        System.out.println("Remote port = "
            +s.getPort());
        System.out.println("Local socket address = "
            +s.getLocalSocketAddress().toString());
        System.out.println("Local address = "
            +s.getLocalAddress().toString());
        System.out.println("Local port = "
            +s.getLocalPort());
    }
}
```

Note that:

- The constructor used to instantiate the socket also performs the functions of bind() and connect().

Now, run ReverseEchoer first, then run SocketClient in a separate command window. If you type in the same text as in the previous test, you will get the following output:

```
Remote address = localhost/127.0.0.1
Remote port = 8888
Local socket address = /127.0.0.1:1073
Local address = /127.0.0.1
Local port = 1073
Welcome to Reverse Echo Server. Please type in some words.
Fish, I love you and respect you very much.
.hcum yrev uoy tcepsr dna uoy evol I ,hsiF
But I will kill you dead before this day ends.
.sdne yad siht erofeb daed uoy llik lliw I tuB
.
```

Note that socket constructor automatically picked up a free local port, 1073.

ReverseEchoServer - A Simple Multi-Connection Socket Server

ReverseEchoer runs perfectly if there is only one client program talking to it. If we run another copy of SocketClient, while a copy of SocketClient is running and connected to ReverseEchoer, the second copy of SocketClient will not be able establish a communication link with ReverseEchoer, because it is busy with the first copy of SocketClient.

To create a server application that can handle multiple connections, one thread needs to be created for each connection. The following program, called ReverseEchoServer, offers the same function to client program. But it can handle multiple connections.

```
/**
 * ReverseEchoServer.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
public class ReverseEchoServer implements Runnable {
    private Socket con = null;
    public static void main(String[] args) {
        try {
            ServerSocket s = new ServerSocket(8888);
            printServerSocketInfo(s);
            while (true) {
                Socket c = s.accept();
                printSocketInfo(c);
                ReverseEchoServer v = new ReverseEchoServer(c);
                Thread t = new Thread(v);
                t.start();
            }
        } catch (IOException e) {
            System.err.println(e.toString());
        }
    }
}
```

```

public ReverseEchoServer(Socket c){
    con = c;
}
public void run() {
    try {
        BufferedWriter w = new BufferedWriter(new OutputStreamWriter(
            con.getOutputStream()));
        BufferedReader r = new BufferedReader(new InputStreamReader(
            con.getInputStream()));
        String m = "Welcome to Reverse Echo Server."+
            " Please type in some words.";
        w.write(m,0,m.length());
        w.newLine();
        w.flush();
        while ((m=r.readLine())!= null) {
            if (m.equals(".")) break;
            char[] a = m.toCharArray();
            int n = a.length;
            for (int i=0; i<n/2; i++) {
                char t = a[i];
                a[i] = a[n-1-i];
                a[n-i-1] = t;
            }
            w.write(a,0,n);
            w.newLine();
            w.flush();
        }
        w.close();
        r.close();
        con.close();
    } catch (IOException e) {
        System.err.println(e.toString());
    }
}

private static void printSocketInfo(Socket s) {
    System.out.println("Remote address = "
        +s.getInetAddress().toString());
    System.out.println("Remote port = "
        +s.getPort());
    System.out.println("Local socket address = "
        +s.getLocalSocketAddress().toString());
    System.out.println("Local address = "
        +s.getLocalAddress().toString());
    System.out.println("Local port = "
        +s.getLocalPort());
}

private static void printServerSocketInfo(ServerSocket s) {
    System.out.println("Server socker address = "
        +s.getInetAddress().toString());
    System.out.println("Server socker port = "
        +s.getLocalPort());
}
}

```

Here is the output displayed on the ReverseEchoServer console window, with two copies of SocketClient running:

```

Server socker address = 0.0.0.0/0.0.0.0
Server socker port = 8888
Remote address = /127.0.0.1
Remote port = 1084

```

```
Local socket address = /127.0.0.1:8888
Local address = /127.0.0.1
Local port = 8888
Remote address = /127.0.0.1
Remote port = 1085
Local socket address = /127.0.0.1:8888
Local address = /127.0.0.1
Local port = 8888
```

Binding Sockets to Local Ports

As I mentioned previously, I used a special socket constructor to allow the system to bind the socket an un-specified local address and port number. This gives the system the freedom to pickup an un-used port number. If I change SocketClient.java from:

```
Socket c = new Socket("localhost", 8888);
```

to:

```
Socket c = new Socket();
c.bind(new InetSocketAddress("localhost", 4444));
c.connect(new InetSocketAddress("localhost", 8888));
```

the first running instance of SocketClient will work, but additional running instances will get the following error:

```
java.net.BindException: Address already in use: JVM_Bind
```

Why? Because, at any given time, only one socket can be bound to a given port.

So, allowing the system to pick up a free local port number is a better idea.

However, the sockets created by the ServerSocket.accept() method seems to be allowed to bind to the same local port. See the output of ReverseEchoServer. When two or more client programs are connected to it, sockets created for the connections are all bound to the same port number, 8888.

Exercise: Can re-bind a bound socket to new port number?

Datagram Communication

Notes and sample codes bellow are based on J2SDK 1.4.1_01.

What Is a Datagram

Datagram: An independent, self-contained message sent over the network whose arrival, arrival time, and content are not guaranteed. This definition is copied from the Sun's Java tutorial document. It should at least contain the following information:

- Address: The Internet address of the remote system.
- Port: The port number of the remote system.
- Data: The data contained in the datagram.

Datagram Socket: A logical concept represents the contact point on the local system of a datagram communication link.

Internet Address: A unique number to identify each computer system on the Internet.

Port: A number representing an entry point of on a computer system on the Internet, where a two-way communication link can be established.

Local System: The computer system at this end of a communication link.

Remote System: The computer system at the other end of a two-way communication link.

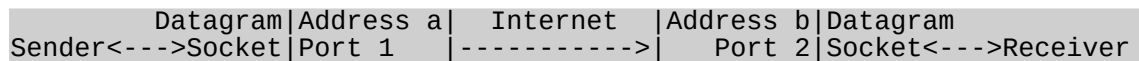
A datagram communication link on the Internet involves:

- Two computer systems, each has its own Internet address.
- A port number on each computer system that is used only by this communication link during the transmission period.
- A program on each computer system that uses this communication link. One program must act as the receiver of the datagram, and the other program must act as the sender.
- The receiver must create a datagram socket to represent the receiving point of the communication link.
- The sender must create a datagram socket to represent the sending point of the communication link.

The following diagram illustrates how application programs can use datagram socket send data to another computer systems through a datagram communication link on the Internet:

Computer System I

Computer System II



Establishing a Datagram Communication

To establish a datagram communication link, one application program must act as a receiver, create a datagram socket with a given local port number, and set the datagram socket in the receiving mode waiting for a datagram to arrive.

With receiver program running and waiting for a datagram to arrive, the other program can now act as a sender, create a datagram socket with a given local port number, prepare a datagram with the remote address and port number in in, and send it over.

J2SDK offers two main classes to support datagram communication:

java.net.DatagramSocket with methods:

- DatagramSocket(): constructing a datagram socket with a given local address, and a given local port number.
- receive(): waiting for a datagram to arrive, and returning the datagram to the caller.
- send(): sending a datagram through the datagram socket.

java.net.DatagramPacket representing the datagram with methods:

- getAddress(): getting the remote address from the datagram.
- getPort(): getting the remote port number from the datagram.
- getData(): getting the data from the datagram.
- setAddress(): setting the remote address to the datagram.
- setPort(): setting the remote port to the datagram.
- setData(): setting the data to the datagram.

The following diagram shows the steps involved in sending a datagram from one program to another program using the methods provided by DatagramSocket and DatagramPacket classes:

Step	Sender Program Internet Address #a Available Port #p	Receiver Program Internet Address #b Available Port #q
1		ds = new DatagramSocket(#b+#q)
2		dp = new DatagramPacket()
3		ds.receive(dp)
4	ds = new DatagramSocket()	(waiting)
5	dp = new DataPacket()	(waiting)
6	dp.setAddress()	(waiting)
7	dp.setPort()	(waiting)
8	dp.setData()	(waiting)
9	ds.send(dp)	(receiving datagram)
10		(dp is ready)

Datagram Socket CClient And Server Programs

The following program, called DatagramServer, is a simple datagram socket application, which acts a server listening on given port number for datagram packets to arrive:

```
/**
 * DatagramServer.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
public class DatagramServer {
    public static void main(String[] args) {
        byte[] buf = new byte[1024];
        DatagramPacket dp = new DatagramPacket(buf, buf.length);
        try {
            // binding to the default address and port 7777
            DatagramSocket ds = new DatagramSocket(7777);
            printDatagramSocketInfo(ds);
            while (true) {
                ds.receive(dp);
                printDatagramSocketInfo(ds);
                printDatagramPacketInfo(dp);
                // no need to call buf = dp.getData();
                // reversing the characters, assuming one-type characters
                int n = dp.getLength();
                for (int i=0; i<n/2; i++) {
                    byte t = buf[i];
                    buf[i] = buf[n-1-i];
                    buf[n-1-i] = t;
                }
                ds.send(dp);
            }
        } catch (IOException e) {
            System.err.println(e.toString());
        }
    }
    private static void printDatagramSocketInfo(DatagramSocket s) {
        System.out.println("Datagram Socket Info:");
        if (s.isConnected()) {
            System.out.println("    Connected to a remote system.");
            System.out.println("    Remote address = "
                +s.getInetAddress().toString());
            System.out.println("    Remote port = "
                +s.getPort());
        } else {
            System.out.println("    Not connected to a remote system.");
        }
        System.out.println("    Local socket address = "
            +s.getLocalSocketAddress().toString());
        System.out.println("    Local address = "
            +s.getLocalAddress().toString());
        System.out.println("    Local port = "
            +s.getLocalPort());
    }
    private static void printDatagramPacketInfo(DatagramPacket d) {
        System.out.println("Datagram Packet Info:");
        System.out.println("    Remote socket address = "
            +d.getSocketAddress().toString());
    }
}
```

```
        System.out.println("    Remote address = "
            +d.getAddress().toString());
        System.out.println("    Remote port = "
            +d.getPort());
        System.out.println("    Data length = "
            +d.getLength());
    }
}
```

Whenever DatagramServer receives a datagram packet, it takes the data, reverses the data, and sends the data back to where it comes from.

Run DatagramServer in a command window, you will see the following output:

```
Datagram Socket Info:
Not connected to a remote system.
Local socket address = 0.0.0.0/0.0.0.0:7777
Local address = 0.0.0.0/0.0.0.0
Local port = 7777
```

The server is ready to receive packets at port 7777.

The following program, called DatagraClient, acts as a client program to DatagramServer. It repeatedly sends a datagram packet to DatagramServer, and waits a return datagram packet:

```
/**
 * DatagramClient.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
public class DatagramClient {
    public static void main(String[] args) {
        BufferedReader in = new BufferedReader(new InputStreamReader(
            System.in));
        PrintStream out = System.out;
        byte[] buf = new byte[1024];
        try {
            DatagramPacket dp = new DatagramPacket(buf, buf.length);
            dp.setAddress(InetAddress.getByName("localhost"));
            dp.setPort(7777);
            // binding to the default address and any free port
            DatagramSocket ds = new DatagramSocket();
            printDatagramSocketInfo(ds);
            String m = null;
            while (true) {
                // sending the text line from console to the remote system
                m = in.readLine();
                byte[] b = m.getBytes();
                dp.setData(b);
                dp.setLength(b.length);
                ds.send(dp);
                // sending the text line from the remote system to console
                dp.setData(buf);
                ds.receive(dp);
                m = new String(buf, 0, dp.getLength());
                out.println(m);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        if (m.equals(".")) break;
    }
} catch (IOException e) {
    System.err.println(e.toString());
}
}
private static void printDatagramSocketInfo(DatagramSocket s) {
    System.out.println("Datagram Socket Info:");
    if (s.isConnected()) {
        System.out.println("    Connected to a remote system.");
        System.out.println("    Remote address = "
            +s.getInetAddress().toString());
        System.out.println("    Remote port = "
            +s.getPort());
    } else {
        System.out.println("    Not connected to a remote system.");
    }
    System.out.println("    Local socket address = "
        +s.getLocalSocketAddress().toString());
    System.out.println("    Local address = "
        +s.getLocalAddress().toString());
    System.out.println("    Local port = "
        +s.getLocalPort());
}
}

```

One `DatagramPacket` object is used for both sending data and receiving data. Two byte arrays are used in the program, `buf` contains the data received, `b` contains the data to be send out.

With `DatagramServer` running in a command window, run `DatagramClient` in another command window, and type in

```

Fish, I love you and respect you very much.
But I will kill you dead before this day ends.
.

```

you will see:

```

Datagram Socket Info:
    Not connected to a remote system.
    Local socket address = 0.0.0.0/0.0.0.0:1035
    Local address = 0.0.0.0/0.0.0.0
    Local port = 1035
Fish, I love you and respect you very much.
.hcum yrev uoy tcepsr dna uoy evol I ,hsiF
But I will kill you dead before this day ends.
.sdne yad siht erofeb daed uoy llik lliw I tuB
.

```

Note that the datagram socket constructor automatically picked up a free local port, 1035.

In the `DatagramServer` window, you will see the following output:

```

Datagram Socket Info:
    Not connected to a remote system.
    Local socket address = 0.0.0.0/0.0.0.0:7777
    Local address = 0.0.0.0/0.0.0.0

```

```
Local port = 7777
Datagram Packet Info:
  Remote socket address = /127.0.0.1:1035
  Remote address = /127.0.0.1
  Remote port = 1035
  Data length = 43
Datagram Socket Info:
  Not connected to a remote system.
  Local socket address = 0.0.0.0/0.0.0.0:7777
  Local address = 0.0.0.0/0.0.0.0
  Local port = 7777
Datagram Packet Info:
  Remote socket address = /127.0.0.1:1035
  Remote address = /127.0.0.1
  Remote port = 1035
  Data length = 46
Datagram Socket Info:
  Not connected to a remote system.
  Local socket address = 0.0.0.0/0.0.0.0:7777
  Local address = 0.0.0.0/0.0.0.0
  Local port = 7777
Datagram Packet Info:
  Remote socket address = /127.0.0.1:1035
  Remote address = /127.0.0.1
  Remote port = 1035
  Data length = 1
```

As you can see, DatagramServer and DatagramClient worked perfectly. You can even run multiple instances of DatagramClient, and DatagramServer will handle them correctly.

Document Object Model (DOM)

What is Document Object Model (DOM)

DOM: An Application Programming Interface (API) that represents an XML file as a document object, which allows application programs to manage the information contained in the document object.

DOM has been implemented in Java in J2SDK 1.4.1_01, which is already installed on my system. So I am ready to play with XML files through DOM in Java.

Parsing XML Files with DOM

Here is a program to show how different packages are used together to parse an XML file into a DOM document object:

```
/**
 * DOMParser.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import javax.xml.parsers.*;
import org.w3c.dom.*;
import org.xml.sax.*;
class DOMParser {
    public static void main(String[] args) {
        try {
            File x = new File(args[0]);
            DocumentBuilderFactory f
                = DocumentBuilderFactory.newInstance();
            System.out.println(f.toString());
            DocumentBuilder b = f.newDocumentBuilder();
            System.out.println(b.toString());
            Document d = b.parse(x);
            System.out.println(d.toString());
            DOMImplementation i = d.getImplementation();
            System.out.println(i.toString());
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
}
```

Output:

```
org.apache.crimson.jaxp.DocumentBuilderFactoryImpl@1c78e57
org.apache.crimson.jaxp.DocumentBuilderImpl@13e8d89
org.apache.crimson.tree.XmlDocument@1cfb549
org.apache.crimson.tree.DOMImplementationImpl@1820dda
```

Note that:

- `javax.xml.parsers.DocumentBuilderFactory.newInstance()` method is used to create a new factory instance using a factory implementation from the `org.apache.crimson.jaxp.*` package.
- `javax.xml.parsers.DocumentBuilderFactory.newDocumentBuilder()` method is used to create a new builder instance using a builder implementation from `org.apache.crimson.jaxp.*` package.
- `javax.xml.parsers.DocumentBuilder.parse()` method is used to parse the XML file into an `org.w3c.dom.Document` object implemented with `org.apache.crimson.tree.XmlDocument` class.

The DOM Tree Structure

In DOM, an XML file is represented with a tree structure, called "document". Every piece of information in an XML file is abstracted as a Node object, and represented by a node in the tree.

Node is actually an interface. It is implemented into many DOM classes to represent different types of information in an XML file. Features that are common to DOM classes are defined as methods in the Node interface. Major get methods of Node include:

- `getNodeTypes()`: Returns the node type.
- `getNodeName()`: Returns the node name.
- `getNodeValue()`: Returns the value associated with this node.
- `getChildNodes()`: Returns a list of nodes nested inside this node.
- `getAttributes()`: Returns a list of nodes that represents the attributes of this node.

Here is a list of node types that are supported by DOM:

```
2 ATTRIBUTE_NODE
4 CDATA_SECTION_NODE
8 COMMENT_NODE
11 DOCUMENT_FRAGMENT_NODE
9 DOCUMENT_NODE
10 DOCUMENT_TYPE_NODE
1 ELEMENT_NODE
6 ENTITY_NODE
5 ENTITY_REFERENCE_NODE
12 NOTATION_NODE
7 PROCESSING_INSTRUCTION_NODE
3 TEXT_NODE
```

The following program illustrates how an XML file can be parse into a DOM document tree, and how the get methods of Node can be used to browse the tree:

```
/**
 * DOMBrowser.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
```



```

import java.io.*;
import javax.xml.parsers.*;
import org.w3c.dom.*;
import org.xml.sax.*;
class DOMBrowser {
    public static void main(String[] args) {
        try {
            File x = new File(args[0]);
            DocumentBuilderFactory f
                = DocumentBuilderFactory.newInstance();
            DocumentBuilder b = f.newDocumentBuilder();
            Document d = b.parse(x);
            printNode(d, "");
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
    static void printNode(Node n, String p) {
        NodeList l = n.getChildNodes();
        NamedNodeMap m = n.getAttributes();
        int ml = -1;
        if (m!=null) ml = m.getLength();
        System.out.println(p+n.getNodeName()+": "+n.getNodeType()+", "
            +l.getLength()+", "+ml+", "+n.getNodeValue());
        for (int i=0; i<ml; i++) {
            Node c = m.item(i);
            printNode(c,p+" |-");
        }
        for (int i=0; i<l.getLength(); i++) {
            Node c = l.item(i);
            printNode(c,p+" ");
        }
    }
}

```

Now let's use this program to browse my first XML file, hello.xml:

```

<?xml version="1.0"?>
<body>Hello world!</body>

#document: 9, 1, -1, null
  body: 1, 1, 0, null
    #text: 3, 0, -1, Hello world!

```

Here is how to read the output:

- The Document object is also a Node object, which is presented by the first line in the output.
- The "xml" processing instruction is not part of the document object.
- The second line in the output says that the root element is named as "body", of type 1, has 1 child node, has 0 attribute, and has no value.
- The third line in the output says that there is child node nested inside the "body" node. The child node is called "#text", of type 3, has 0 child node, could not have any attribute, and has a value of string "Hello world!".

- Note that the text enclosed by the "body" tags is parsed into a node separated from the "body" node. So how can we link that text with the tag name "body"?

Here is another XML file with more elements, user.xml:

```
<?xml version="1.0"?>
<user status="active">
  <!-- This is not a real user. -->
  <first_name>John</first_name>
  <last_name>Smith</last_name>
</user>
```

Run DOMBrowser with this XML file, you will get:

```
#document: 9, 1, -1, null
user: 1, 7, 1, null
  |-status: 2, 0, -1, active
  #text: 3, 0, -1,

  #comment: 8, 0, -1, This is not a real user.
  #text: 3, 0, -1,

  first_name: 1, 1, 0, null
  #text: 3, 0, -1, John
  #text: 3, 0, -1,

  last_name: 1, 1, 0, null
  #text: 3, 0, -1, Smith
  #text: 3, 0, -1,
```

The output is more interesting:

- Line breaks are also parsed into "#text" nodes. This is why node "user" has 7 child nodes: 4 line breaks, 1 comment, and 2 elements: "first_name" and "last_name".
- For a node that represents an attribute of element, the node value is the attribute value. See node "status" under "user".

Building a New DOM Document

DOM documents can also be created from scratch instead of parsed from an XML file. This can be easily done by using the following methods:

- DocumentBuilder.newDocument(): Create an empty DOM document.
- Document.createElement(): Create a new Element.
- Document.createTextNode(): Create a new TextNode.
- Node.appendChild(): Insert a sub Node into the current Node.
- Element.setAttribute(): Add new attribute into an element.

The following program shows how to create a simple DOM document with these methods:

```
/**
```

```

* DOMNewDoc.java
* Copyright (c) 2002 by Dr. Herong Yang
*/
import java.io.*;
import javax.xml.parsers.*;
import org.w3c.dom.*;
class DOMNewDoc {
    public static void main(String[] args) {
        try {
            DocumentBuilderFactory f
                = DocumentBuilderFactory.newInstance();
            DocumentBuilder b = f.newDocumentBuilder();
            Document d = b.newDocument();
            Element r = d.createElement("dictionary");
            d.appendChild(r);
            Element w = d.createElement("word");
            r.appendChild(w);
            Element e = d.createElement("update");
            w.appendChild(e);
            e.setAttribute("date", "2002-12-24");
            e = d.createElement("name");
            w.appendChild(e);
            e.setAttribute("is_acronym", "true");
            e.appendChild(d.createTextNode("DTD"));
            e = d.createElement("definition");
            w.appendChild(e);
            e.appendChild(d.createTextNode("Document Type Definition"));
            printNode(d, "");
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        }
    }
    static void printNode(Node n, String p) {
        NodeList l = n.getChildNodes();
        NamedNodeMap m = n.getAttributes();
        int ml = -1;
        if (m!=null) ml = m.getLength();
        System.out.println(p+n.getNodeName()+": "+n.getNodeType()+", "
            +l.getLength()+", "+ml+", "+n.getNodeValue());
        for (int i=0; i<ml; i++) {
            Node c = m.item(i);
            printNode(c, p+" |-");
        }
        for (int i=0; i<l.getLength(); i++) {
            Node c = l.item(i);
            printNode(c, p+" ");
        }
    }
}

```

Output:

```

#document: 9, 1, -1, null
dictionary: 1, 1, 0, null
  word: 1, 3, 0, null
    update: 1, 0, 1, null
      |-date: 2, 0, -1, 2002-12-24
      name: 1, 1, 1, null
        |-is_acronym: 2, 0, -1, true
      #text: 3, 0, -1, DTD
    definition: 1, 1, 0, null

```

```
#text: 3, 0, -1, Document Type Definition
```

Converting DOM Documents to XML Files

Once you have a DOM document in memory, for sure you want to know how to write it out into an XML file. One way to do this is to use the XML transform package offered in J2SDK.

The main class in the XML transform package is called Transformer, which can be used process XML from a variety of sources and write the transformation output to a variety of sinks.

Here is a program to illustrate how to use Transformer to convert a DOM document into a XML file:

```
/**
 * DOMToXML.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import javax.xml.parsers.*;
import org.w3c.dom.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.*;
import javax.xml.transform.stream.*;
class DOMToXML {
    public static void main(String[] args) {
        try {
            File x = new File(args[0]);
            DocumentBuilderFactory f
                = DocumentBuilderFactory.newInstance();
            DocumentBuilder b = f.newDocumentBuilder();
            Document d = b.newDocument();
            Element r = d.createElement("dictionary");
            d.appendChild(r);
            Element w = d.createElement("word");
            r.appendChild(w);
            Element e = d.createElement("update");
            w.appendChild(e);
            e.setAttribute("date", "2002-12-24");
            e = d.createElement("name");
            w.appendChild(e);
            e.setAttribute("is_acronym", "true");
            e.appendChild(d.createTextNode("DTD"));
            e = d.createElement("definition");
            w.appendChild(e);
            e.appendChild(d.createTextNode("Document Type Definition"));
            TransformerFactory tf = TransformerFactory.newInstance();
            Transformer m = tf.newTransformer();
            System.out.println(m.toString());
            DOMSource source = new DOMSource(d);
            StreamResult result = new StreamResult(x);
            m.transform(source, result);
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (TransformerConfigurationException e) {
            System.out.println(e.toString());
        }
    }
}
```

```
    } catch (TransformerException e) {  
        System.out.println(e.toString());  
    }  
}
```

Run this program with "java DOMToXML word.xml, you will get the following on the Java console window:

```
org.apache.xalan.transformer.TransformerIdentityImpl@743399
```

This tells us that the factory pickup an implementation of Transformer from the org.apache.xalan.transformer package.

word.xml, the XML file generated by the program, looks very good:

```
<?xml version="1.0" encoding="UTF-8"?>  
<dictionary><word><update date="2002-12-24"/>  
<name is_acronym="true">DTD</name>  
<definition>Document Type Definition</definition></word>  
</dictionary>
```

Note that entire root element is stored in a single line. It was split into multiple lines to be included in here.

Source: Herong's Notes on XML.

Simple API for XML (SAX)

What is Simple API for XML (SAX)

SAX: An Application Programming Interface (API) that allows the parser to fire events based on the XML structure while parsing an XML file. It allows application programs to process the contents of the XML by handling the events fired by the parser.

SAX has been implemented in Java in J2SDK 1.4.1_01, which is already installed on my system. So I am ready to play with XML files through SAX in Java.

SAX Implementations in J2SDK

I have a simple program here, SAXClassChecker.java, to show what are the implementation classes for the SAX interfaces:

```
/**
 * SAXClassChecker.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.File;
import java.io.IOException;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.ParserConfigurationException;
import org.xml.sax.helpers.DefaultHandler;
import org.xml.sax.SAXException;
class SAXClassChecker {
    public static void main(String[] args) {
        try {
            File x = new File(args[0]);
            SAXParserFactory f = SAXParserFactory.newInstance();
            System.out.println(f.toString());
            SAXParser p = f.newSAXParser();
            System.out.println(p.toString());
            DefaultHandler h = new DefaultHandler();
            System.out.println(h.toString());
            p.parse(x,h);
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
}
```

Run the program as: java SAXClassChecker hello.xml, you will get:

```
org.apache.crimson.jaxp.SAXParserFactoryImpl@1c78e57
org.apache.crimson.jaxp.SAXParserImpl@1e63e3d
org.xml.sax.helpers.DefaultHandler@1be2d65
```

Note that:

- `javax.xml.parsers.SAXParserFactory.newInstance()` method is used to create a new factory instance using the factory implementation class, `org.apache.crimson.jaxp.SAXParserFactoryImpl`.
- `javax.xml.parsers.SAXParserFactory.newSAXParser()` method is used to create a new parser instance using the parser implementation class, `org.apache.crimson.jaxp.SAXParserImpl`.
- `javax.xml.parsers.SAXParser.parse()` method is used to invoke the parsing process. It requires two parameters, a `File` object representing the XML file, and an parsing event handler object.
- `org.xml.sax.helpers.DefaultHandler` is dummy handler, which handles all the events fired by the parser, but does nothing. This is why we are getting nothing from the XML file when running this program.

SAX Content Handler Interface

As I mentioned earlier, while the SAX parser is parsing an XML file, it fires events. The first group of such events are designed into an interface, `org.xml.sax.ContentHandler`. It contains the following XML content related events:

- `startDocument()`: When parsing reaches the beginning of the XML document.
- `endDocument()`: When parsing reaches the end of the XML document.
- `startElement()`: When parsing reaches the beginning of an XML element.
- `endElement()`: When parsing reaches the end of an XML element.
- `characters()`: When parsing reaches the end of a character section.
- `ignorableWhitespace()`: When parsing reaches any ignorable white spaces between elements.

Of course, some of the events also pass information parsed from the XML file to the event handler as parameters. For example:

- `startElement()` passes all the attributes as an `org.xml.sax.Attributes` object.
- `characters()` passes all the characters of the parsed text as `char[]` object.

My SAX Based XML Browser

Let's build a simple SAX based XML browser by handling the events in the `ContentHandler` interface:

```
/**
 * SAXBrowser.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.File;
import java.io.IOException;
import javax.xml.parsers.SAXParserFactory;
```

```

import javax.xml.parsers.SAXParser;
import javax.xml.parsers.ParserConfigurationException;
import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.helpers.DefaultHandler;
class SAXBrowser {
    public static void main(String[] args) {
        try {
            File x = new File(args[0]);
            SAXParserFactory f = SAXParserFactory.newInstance();
            SAXParser p = f.newSAXParser();
            DefaultHandler h = new MyContentHandler();
            p.parse(x,h);
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
    private static class MyContentHandler extends DefaultHandler {
        static String p = "_";
        public void startDocument() throws SAXException {
            System.out.println("Starting document...");
        }
        public void endDocument() throws SAXException {
            System.out.println("Ending document...");
        }
        public void startElement(String ns, String sName, String qName,
            Attributes attrs) throws SAXException {
            String eName = sName;
            if (sName.equals("")) eName = qName;
            System.out.println("e"+p+eName);
            if (attrs!=null) {
                for (int i=0; i<attrs.getLength(); i++) {
                    String aName = attrs.getLocalName(i);
                    if (aName.equals("")) aName = attrs.getQName(i);
                    System.out.println("a"+p+" "+aName+"="
                        +attrs.getValue(i));
                }
            }
            p = p + "_";
        }
        public void endElement(String ns, String sName, String qName)
            throws SAXException {
            p = p.replaceFirst("_", "");
        }
        public void characters(char buf[], int offset, int len)
            throws SAXException {
            String s = new String(buf, offset, len);
            System.out.println("c"+p+s);
        }
        public void ignorableWhitespace(char buf[], int offset, int len)
            throws SAXException {
            String s = new String(buf, offset, len);
            System.out.println("i"+p+s);
        }
    }
}

```

Note that:

- I cheated a little bit. Instead of implementing the `ContentHandler` interface directly, I extended the `DefaultHandler` class, which implemented handling methods for all events (by doing nothing). In this way, I only need to override the handling methods that I am interested in.
- `"_"` character is used to indent sub-elements in nested elements.

Let's try this with `hello.xml`:

```
<?xml version="1.0"?>
<p>Hello world!</p>
```

Run `java SAXBrowser hello.xml`, I got:

```
Starting document...
e_p
c__Hello world!
Ending document...
```

Excellent. The program seems to be working.

Let's try it with `user.xml`:

```
<?xml version="1.0"?>
<user status="active">
  <!-- This is not a real user. -->
  <first_name>John</first_name>
  <last_name>Smith</last_name>
</user>
```

Run `java SAXBrowser user.xml`, I got:

```
Starting document...
e_user
a_ status=active
c__
c__
c__
c__
c__
e__first_name
c__John
c__
c__
e__last_name
c__Smith
c__
c__
Ending document...
```

The program still works. But why the parser fired so many `"characters()"` events? It looks

like the parser didn't group the space character, line feed, and cartridge return into a single `char[]` and fire one `"characters()"` event. It fired multiple events, one per character.

Source: Herong's Notes on XML.

DTD Validation in Java

Validation with DOM

J2SDK 1.4.1_01 offers a document builder interface, import `javax.xml.parsers.DocumentBuilder`, to represent classes that can parse XML files into document objects described in the Document Object Model (DOM). See my notes on DOM for more details.

The document builder also supports DTD validation during the parsing process. To do this:

- The document builder object must be set to validate DTD.
- You must create your own error handler class.
- The document builder object must be provided with an error handler object.

Here is a sample program, `DOMValidator`, that shows how to use a document builder class to validate XML files against the specified DTD statements.

```
/**
 * DOMValidator.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.File;
import java.io.IOException;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.ParserConfigurationException;
import org.w3c.dom.Document;
import org.xml.sax.SAXException;
import org.xml.sax.SAXParseException;
import org.xml.sax.ErrorHandler;
import org.xml.sax.helpers.DefaultHandler;
class DOMValidator {
    public static void main(String[] args) {
        try {
            File x = new File(args[0]);
            DocumentBuilderFactory f
                = DocumentBuilderFactory.newInstance();
            f.setValidating(true); // Default is false
            DocumentBuilder b = f.newDocumentBuilder();
            // ErrorHandler h = new DefaultHandler();
            ErrorHandler h = new MyErrorHandler();
            b.setErrorHandler(h);
            Document d = b.parse(x);
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
}
private static class MyErrorHandler implements ErrorHandler {
```

```

    public void warning(SAXParseException e) throws SAXException {
        System.out.println("Warning: ");
        printInfo(e);
    }
    public void error(SAXParseException e) throws SAXException {
        System.out.println("Error: ");
        printInfo(e);
    }
    public void fatalError(SAXParseException e) throws SAXException {
        System.out.println("Fatal error: ");
        printInfo(e);
    }
    private void printInfo(SAXParseException e) {
        System.out.println("    Public ID: "+e.getPublicId());
        System.out.println("    System ID: "+e.getSystemId());
        System.out.println("    Line number: "+e.getLineNumber());
        System.out.println("    Column number: "+e.getColumnNumber());
        System.out.println("    Message: "+e.getMessage());
    }
}

```

To test DOMValidator, I created the following XML file with DTD statements, `invalid_dtd.xml`:

```

<?xml version="1.0"?>
<!-- dictionary_dtd.xml
    Copyright (c) 2002 by Dr. Herong Yang
-->
<!DOCTYPE dictionary [
  <!ELEMENT dictionary (note, word+)>
  <!ELEMENT note ANY>
  <!ELEMENT word (update?, name, definition+, usage*)>
  <!ELEMENT update EMPTY>
  <!ATTLIST update
    date CDATA #REQUIRED
    editor CDATA #IMPLIED
  >
  <!ELEMENT name (#PCDATA)>
  <!ATTLIST name is_acronym (true | false) "false">
  <!ELEMENT definition (#PCDATA)>
  <!ELEMENT usage (#PCDATA | i)*>
  <!ELEMENT i (#PCDATA)>
  <!ENTITY herong "Dr. Herong Yang">
]>
<dictionary>
  <note>Copyright (c) 2002 by &herong;</note>
  <word>
    <name is_acronym="true" language="EN">POP</name>
    <definition>Post Office Protocol</definition>
    <definition>Point Of Purchase</definition>
  </word>
  <word>
    <update date="2002-12-23"/>
    <name is_acronym="yes">XML</name>
    <definition>eXtensible Markup Language</definition>
    <note>XML comes from SGML</note>
  </word>
  <word>
    <update editor="Herong Yang"/>
    <name>markup</name>
  </word>
</dictionary>

```

```
<definition>The amount added to the cost price to calculate  
the selling price - <i>Webster</i></definition>  
</word>  
</dictionary>
```

After entering, `java DOMValidator invalid_dtd.xml`, I got:

```
Error:  
Public ID: null  
System ID: file:C:/herong/invalid_dtd.xml  
Line number: 24  
Column number: -1  
Message: Attribute "language" is not declared for element "name".  
Error:  
Public ID: null  
System ID: file:C:/herong/invalid_dtd.xml  
Line number: 30  
Column number: -1  
Message: Value "yes" is not one of the enumerated values  
for this attribute.  
Error:  
Public ID: null  
System ID: file:C:/herong/invalid_dtd.xml  
Line number: 32  
Column number: -1  
Message: Element "word" does not allow "note" here.  
Error:  
Public ID: null  
System ID: file:C:/herong/invalid_dtd.xml  
Line number: 35  
Column number: -1  
Message: Attribute value for "date" is #REQUIRED.  
Error:  
Public ID: null  
System ID: file:C:/herong/invalid_dtd.xml  
Line number: 38  
Column number: -1  
Message: Element "definition" does not allow "i" -- (#PCDATA)
```

This is perfect. It tells you where the error is, and why it's an error.

Validation with SAX

Actually, SAX and DOM are sharing the same XML validation implementation. In this implementation, error events are fired during the parsing process. Application programs that are interested in validation errors must implement the error event handlers defined in the `org.xml.sax.ErrorHandler` interface.

In the SAX design, the `ErrorHandler` interface is integrated into the `DefaultHandler` class. Application programs just need to extend `DefaultHandler` and override those error event handlers.

Here is my SAX based XML validation program, `SAXValidator.java`:

```
/**  
 * SAXValidator.java
```

```

* Copyright (c) 2002 by Dr. Herong Yang
*/
import java.io.File;
import java.io.IOException;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.ParserConfigurationException;
import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.SAXParseException;
import org.xml.sax.helpers.DefaultHandler;
class SAXValidator {
    public static void main(String[] args) {
        try {
            File x = new File(args[0]);
            SAXParserFactory f = SAXParserFactory.newInstance();
            f.setValidating(true); // Default is false
            SAXParser p = f.newSAXParser();
            DefaultHandler h = new MyErrorHandler();
            p.parse(x,h);
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
    private static class MyErrorHandler extends DefaultHandler {
        public void warning(SAXParseException e) throws SAXException {
            System.out.println("Warning: ");
            printInfo(e);
        }
        public void error(SAXParseException e) throws SAXException {
            System.out.println("Error: ");
            printInfo(e);
        }
        public void fatalError(SAXParseException e) throws SAXException {
            System.out.println("Fattal error: ");
            printInfo(e);
        }
        private void printInfo(SAXParseException e) {
            System.out.println("    Public ID: "+e.getPublicId());
            System.out.println("    System ID: "+e.getSystemId());
            System.out.println("    Line number: "+e.getLineNumber());
            System.out.println("    Column number: "+e.getColumnNumber());
            System.out.println("    Message: "+e.getMessage());
        }
    }
}

```

Note that:

- SAXParserFactory.setValidating(true) is called to turn on the validation funtion.
- The structure of this program is 99% identical to SAXBrowser.java. Since I am only interested the error events, only the error event handlers are overrode. If I copy the contend event handlers here, and this program will be working as a browser and a validator at the same time.

Run java SAXValidator invalid_dtd.xml, you will get the same output as DOMValidator.

Source: Herong's Notes on XML.

XSD Validation in Java

Validation with XMLReader

Unfortunately, I couldn't find any XML parsers provided in J2SDK 1.4.1_02 that can validate XML structure against XSD rules. So I downloaded one of the most popular XML parsers in the public domain, xerces-j 2.3.0, at: <http://xml.apache.org/dist/xerces-j>.

Once I downloaded Xerces-J-bin.2.3.0.zip, I unzipped it into \local\xerces-2_3_0 directory. Make sure that xercesImpl.jar is in that directory.

Now I am ready to write a simple Java program to use "org.apache.xerces.parsers.SAXParser" to validate any XML files against the specified XSD files:

```
/**
 * XMLReaderValidator.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.IOException;
import org.xml.sax.XMLReader;
import org.xml.sax.helpers.DefaultHandler;
import org.xml.sax.helpers.XMLReaderFactory;
import org.xml.sax.SAXException;
import org.xml.sax.SAXParseException;
class XMLReaderValidator {
    public static void main(String[] args) {
        String parserClass = "org.apache.xerces.parsers.SAXParser";
        String validationFeature
            = "http://xml.org/sax/features/validation";
        String schemaFeature
            = "http://apache.org/xml/features/validation/schema";
        try {
            String x = args[0];
            XMLReader r = XMLReaderFactory.createXMLReader(parserClass);
            r.setFeature(validationFeature, true);
            r.setFeature(schemaFeature, true);
            r.setErrorHandler(new MyErrorHandler());
            r.parse(x);
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
    private static class MyErrorHandler extends DefaultHandler {
        public void warning(SAXParseException e) throws SAXException {
            System.out.println("Warning: ");
            printInfo(e);
        }
        public void error(SAXParseException e) throws SAXException {
            System.out.println("Error: ");
            printInfo(e);
        }
        public void fatalError(SAXParseException e) throws SAXException {
```



```

        System.out.println("Fatal error: ");
        printInfo(e);
    }
    private void printInfo(SAXParseException e) {
        System.out.println("    Public ID: "+e.getPublicId());
        System.out.println("    System ID: "+e.getSystemId());
        System.out.println("    Line number: "+e.getLineNumber());
        System.out.println("    Column number: "+e.getColumnNumber());
        System.out.println("    Message: "+e.getMessage());
    }
}
}

```

Here is an XML file, dictionary_invalid_xsd.xml:

```

<?xml version="1.0"?>
<dictionary xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="dictionary.xsd">
<!-- dictionary_invalid_xsd.xml
  Copyright (c) 2002 by Dr. Herong Yang
-->
  <word acronym="yes">
    <name>XML</name>
    <definition reference="Herong's Notes">eXtensible Markup
Language.</definition>
    <update date="23-Dec-2003"/>
  </word>
  <word symbol="true">
    <name><</name>
    <definition>Mathematical symbol representing the "less than" logical
operation, like: 1<2.</definition>
    <definition>Reserved symbol in XML representing the beginning of
tags, like: <![CDATA[<p>Hello world!</p>]]>
    </definition>
    <update editor="Herong Yang"/>
  </word>
  <word symbol="no" acronym="false">
    <name>extensible</name>
    <definition>Capable of being extended.</definition>
  </word>
</dictionary>

```

Here is the linked XSD file, dictionary.xsd:

```

<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="dictionary" type="dictionaryType"/>
  <xsd:complexType name="dictionaryType">
    <xsd:sequence>
      <xsd:element name="word" type="wordType" maxOccurs="unbounded"/>
    </xsd:sequence>
  </xsd:complexType>
  <xsd:complexType name="wordType">
    <xsd:sequence>
      <xsd:element name="name" type="xsd:string"/>
      <xsd:element name="definition" type="definitionType"
maxOccurs="unbounded"/>
      <xsd:element name="update" type="updateType" minOccurs="0"/>
    </xsd:sequence>
    <xsd:attribute name="acronym" type="xsd:boolean" use="optional"/>
    <xsd:attribute name="symbol" type="xsd:boolean" use="optional"/>
  </xsd:complexType>

```

```

</xsd:complexType>
<xsd:complexType name="definitionType" mixed="true">
  <xsd:attribute name="reference" type="xsd:string"/>
</xsd:complexType>
<xsd:complexType name="updateType">
  <xsd:attribute name="date">
    <xsd:simpleType>
      <xsd:restriction base="xsd:string">
        <xsd:pattern value="\p{Nd}{4}-\p{Nd}{2}-\p{Nd}{2}"/>
      </xsd:restriction>
    </xsd:simpleType>
  </xsd:attribute>
</xsd:complexType>
</xsd:schema>

```

Now run the XMLReaderValidator program:

```

java -cp .;\local\xerces-2_3_0\xercesImpl.jar XMLReaderValidator
dictrionary_invalid_xsd.xml

```

you will get:

```

Error:
  Public ID: null
  System ID: file:///D:/herong/dictionary_invalid_xsd.xml
  Line number: 7
  Column number: 22
  Message: cvc-datatype-valid.1.2.1: 'yes' is not a valid 'boolean'
value.
Error:
  Public ID: null
  System ID: file:///D:/herong/dictionary_invalid_xsd.xml
  Line number: 7
  Column number: 22
  Message: cvc-attribute.3: The value 'yes' of attribute 'acronym' on
element 'word' is not valid with respect to its type.
Error:
  Public ID: null
  System ID: file:///D:/herong/src/dictionary_invalid_xsd.xml
  Line number: 11
  Column number: 31
  Message: cvc-pattern-valid: Value '23-Dec-2003' is not facet-valid
with respect to pattern '\p{Nd}{4}-\p{Nd}{2}-\p{Nd}{2}'.
Error:
  Public ID: null
  System ID: file:///D:/herong/dictionary_invalid_xsd.xml
  Line number: 11
  Column number: 31
  Message: cvc-attribute.3: The value '23-Dec-2003' of attribute 'date'
on element 'update' is not valid with respect to its type.
Error:
  Public ID: null
  System ID: file:///D:/herong/dictionary_invalid_xsd.xml
  Line number: 20
  Column number: 33
  Message: cvc-complex-type.3.2.2: Attribute 'editor' is not allowed to
appear in element 'update'.
Error:
  Public ID: null
  System ID: file:///D:/herong/dictionary_invalid_xsd.xml
  Line number: 22

```

```
Column number: 36
Message: cvc-datatype-valid.1.2.1: 'no' is not a valid 'boolean'
value.
Error:
  Public ID: null
  System ID: file:///D:/herong/dictionary_invalid_xsd.xml
  Line number: 22
  Column number: 36
  Message: cvc-attribute.3: The value 'no' of attribute 'symbol' on
element 'word' is not valid with respect to its type.
```

This is perfect. It tells you where the error is, and why it's an error.

Using SAXParserFactory to Load Parsers

Xerces-J package can also be loaded by the SAXParserFactory.newInstance() method. Here is my SAXValidator.java:

```
/**
 * SAXValidator.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.File;
import java.io.IOException;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.ParserConfigurationException;
import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.SAXParseException;
import org.xml.sax.helpers.DefaultHandler;
class SAXValidator {
    public static void main(String[] args) {
        String schemaFeature
            = "http://apache.org/xml/features/validation/schema";
        try {
            File x = new File(args[0]);
            SAXParserFactory f = SAXParserFactory.newInstance();
            System.out.println(f.toString());
            f.setValidating(true);
            f.setFeature(schemaFeature, true);
            SAXParser p = f.newSAXParser();
            System.out.println(p.toString());
            DefaultHandler h = new MyErrorHandler();
            p.parse(x, h);
        } catch (ParserConfigurationException e) {
            System.out.println(e.toString());
        } catch (SAXException e) {
            System.out.println(e.toString());
        } catch (IOException e) {
            System.out.println(e.toString());
        }
    }
    private static class MyErrorHandler extends DefaultHandler {
        public void warning(SAXParseException e) throws SAXException {
            System.out.println("Warning: ");
            printInfo(e);
        }
        public void error(SAXParseException e) throws SAXException {
```

```
        System.out.println("Error: ");
        printInfo(e);
    }
    public void fatalError(SAXParseException e) throws SAXException {
        System.out.println("Fattal error: ");
        printInfo(e);
    }
    private void printInfo(SAXParseException e) {
        System.out.println("    Public ID: "+e.getPublicId());
        System.out.println("    System ID: "+e.getSystemId());
        System.out.println("    Line number: "+e.getLineNumber());
        System.out.println("    Column number: "+e.getColumnNumber());
        System.out.println("    Message: "+e.getMessage());
    }
}
}
```

Note that the schema feature to the SAXParserFactory object.

Now if you run the program with:

```
java -cp . SAXValidator dictrionary_invalid_xsd.xml
```

You will get:

```
org.apache.crimson.jaxp.SAXParserFactoryImpl@1004901
org.xml.sax.SAXNotRecognizedException: Feature: http://apache.org/xml
/features/validation/schema
```

Note that the crimson package was loaded from the J2SDK library, and it doesn't support XSD validation.

But if you run the program with:

```
java -cp .;\local\xerces-2_3_0\xercesImpl.jar SAXValidator
    dictrionary_invalid_xsd.xml
```

you will get:

```
org.apache.xerces.jaxp.SAXParserFactoryImpl@f72617
org.apache.xerces.jaxp.SAXParserImpl@173831b
...
```

Note that the xerces package was loaded, and XSD validation was performed.

Source: Herong's Notes on XML.

XSL - Transformer in Java

XSL Implementations in J2SDK

J2SDK 1.4.1_01 offers the following interfaces to support XSL:

- javax.xml.transform.TransformerFactory: The transformer factory class to help to create a transformer object.
- javax.xml.transform.Transformer: The abstract class representing a transformer.
- javax.xml.transform.Source: The interface representing source objects for the transformer to process.
- javax.xml.transform.Result: The interface representing result objects produced by the transformer.

One thing is missing here: who represents the transformation style rules? The answer is that javax.xml.transform.Source also represents transformation style rules, because they written in XML.

I have a simple program here, XSLClassChecker.java, to show what are the XSL implementation classes:

```
/**
 * XSLClassChecker.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import javax.xml.transform.Source;
import javax.xml.transform.Result;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerConfigurationException;
import javax.xml.transform.stream.StreamSource;
import javax.xml.transform.stream.StreamResult;
class XSLClassChecker {
    public static void main(String[] args) {
        try {
            TransformerFactory f = TransformerFactory.newInstance();
            System.out.println(f.toString());
            Transformer t = f.newTransformer();
            System.out.println(t.toString());
            Source s = new StreamSource();
            System.out.println(s.toString());
            Result r = new StreamResult();
            System.out.println(r.toString());
            t.transform(s,r);
        } catch (TransformerConfigurationException e) {
            System.out.println(e.toString());
        } catch (TransformerException e) {
            System.out.println(e.toString());
        }
    }
}
```

Output:

```
org.apache.xalan.processor.TransformerFactoryImpl@119298d
org.apache.xalan.transformer.TransformerIdentityImpl@1f33675
javax.xml.transform.stream.StreamSource@1690726
javax.xml.transform.stream.StreamResult@9931f5
javax.xml.transform.TransformerException: No output specified
```

Note that:

- So J2SDK 1.4.1_01 is using the org.apache.xalan package for XSL transformation.
- The Identity transformer was used, because the program is specify any transformation rules.
- I got the "No output specified" exception, because the program was transforming from nothing (empty source object) to nowhere (empty result object).

A Simple XSL Transformer Program

Here is my first XSL transformation program in Java:

```
/**
 * XSLTransformer.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import javax.xml.transform.*;
import javax.xml.transform.stream.*;
class XSLTransformer {
    public static void main(String[] args) {
        try {
            File sf = new File(args[0]); // source file
            File rf = new File(args[1]); // result file
            File tf = new File(args[2]); // template file
            TransformerFactory f = TransformerFactory.newInstance();
            Transformer t = f.newTransformer(new StreamSource(tf));
            Source s = new StreamSource(sf);
            Result r = new StreamResult(rf);
            t.transform(s,r);
        } catch (TransformerConfigurationException e) {
            System.out.println(e.toString());
        } catch (TransformerException e) {
            System.out.println(e.toString());
        }
    }
}
```

Let's try this program with my hello.xml as the source file:

```
<?xml version="1.0"?>
<p>Hello world!</p>
```

and my hello.xsl as the template file that contains the transformation style rules:

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
```

```
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="p">Hello world.! - From hello.xsl.</xsl:template>
</xsl:stylesheet>
```

Run "java XSLTransformer hello.xml hello.out hello.xsl", I got in hello.out:

```
<?xml version="1.0" encoding="UTF-8"?>
Hello world! - From hello.xsl.
```

Source: Herong's Notes on XML.

JCA - Message Digest

This chapter describes some parts of the JCA (Java Cryptography Architecture) which has been included in JDK since 1.1:

- What is message digest?
- The MessageDigest class in JDK.
- A sample program on MessageDigest class.

What Is Message Digest

Message digest is a one-way hash function that takes arbitrary-sized data and outputs a fixed-length hash value.

Known message digest algorithms are:

- SHA - The Secure Hash Algorithm, as defined in Secure Hash Standard, NIST FIPS 180-1.
- MD2 - The MD2 message digest algorithm as defined in RFC 1319.
- MD5 - The MD5 message digest algorithm as defined in RFC 1321.

The MessageDigest Class

`java.security.MessageDigest` is an abstract class providing a link to implementation classes of message digest algorithms provided by various security package providers. Major methods in the `MessageDigest` class:

`getInstance()` - Returns a message digest object of the specified algorithm from the implementation of the specified provider. If provider is not specified, the default implementation is used. This is a static method.

`update()` - Adds more data to the current message digest object for processing.

`digest()` - Returns the digest data of the input data processed so far in the current message digest object. It also removes the input data received in the object.

`getAlgorithm()` - Returns the algorithm name of the current message digest object.

`getProvider()` - Returns the provider as a provider object of the current message digest object.

Message Digest Sample Program - `JcaMessageDigest.java`

The following sample program shows you how to invoke the message digest algorithms implemented by the default provider, Sun, and digest the data from the input file.


```
/**
 * JcaMessageDigest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
class JcaMessageDigest {
    public static void main(String[] a) {
        if (a.length<3) {
            System.out.println("Usage:");
            System.out.println("java JcaMessageDigest input output"
                +" algorithm");
            return;
        }
        String input = a[0];
        String output = a[1];
        String algorithm = a[2]; // SHA, MD5
        try {
            digest(input,output,algorithm);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
    private static void digest(String input, String output,
        String algorithm) throws Exception {
        MessageDigest md = MessageDigest.getInstance(algorithm);
        System.out.println();
        System.out.println("MessageDigest Object Info: ");
        System.out.println("Algorithm = "+md.getAlgorithm());
        System.out.println("Provider = "+md.getProvider());
        System.out.println("Digest Length = "+md.getDigestLength());
        System.out.println("toString = "+md.toString());
        FileInputStream in = new FileInputStream(input);
        int bufSize = 1024;
        byte[] buffer = new byte[bufSize];
        int n = in.read(buffer,0,bufSize);
        int count = 0;
        while (n!=-1) {
            count += n;
            md.update(buffer,0,n);
            n = in.read(buffer,0,bufSize);
        }
        in.close();
        FileOutputStream out = new FileOutputStream(output);
        byte[] digest = md.digest();
        out.write(digest);
        out.close();
        System.out.println();
        System.out.println("Message Digest Processing Info: ");
        System.out.println("Number of input bytes = "+count);
        System.out.println("Number of output bytes = "+digest.length);
    }
}
```

Here is the result of my first test. It is done with JDK 1.3.1.

```
javac -classpath . JcaMessageDigest.java
java -cp . JcaMessageDigest JcaMessageDigest.class digest.sha SHA
```

```
MessageDigest Object Info:
Algorithm = SHA
Provider = SUN version 1.2
Digest Length = 20
toString = SHA Message Digest from SUN, <initialized>

Message Digest Processing Info:
Number of input bytes = 2060
Number of output bytes = 20

dir *.class *.sha
07:00p    2,060 JcaMessageDigest.class
07:02p      20 digest.sha
```

The program seems to be working:

- Since I am not specifying the provider name, the implementation of the SHA algorithm provided in the default security package was selected. Of course, Sun is the provider of the default security package.
- The digest length is 20 bytes for the default implementation of the SHA algorithm.
- The program reads the input file, JcaMessageDigest.class, into a byte array buffer, then passes the buffer to the message digest object to process by calling the update(). When the entire input file is processed, digest() method is called to output the digest data.
- The number of bytes reported by the program matches well with the file sizes.

In order to see the digest data, I need to use my other program, HexWriter.java, to convert binary data to hex numbers. See chapter "Encoding Conversion" for details.

Here is how to look at the digest data in hex numbers, 16 bytes per line:

```
javac HexWriter.java
java -cp . HexWriter digest.sha digest_sha.hex
type digest_sha.hex
9E6BA180ED152D8F58A4C9F33161DE2A
4F9B304A
```

My second test is to see the digest data of the same input data, JcaMessageDigest.class, using MD5 algorithm:

```
java -cp . JcaMessageDigest JcaMessageDigest.class digest.md5 MD5

MessageDigest Object Info:
Algorithm = MD5
Provider = SUN version 1.2
Digest Length = 16
toString = MD5 Message Digest from SUN, <initialized>

Message Digest Processing Info:
Number of input bytes = 2060
Number of output bytes = 16

type digest_md5.hex
```

CADFA031B79763775D861878062ACA4F

My third test is to see how the digest data changes if I add one byte to the input data:

```
copy con dot.tmp
.^Z

type dot.tmp >> JcaMessageDigest.class

java -cp . JcaMessageDigest JcaMessageDigest.class digest_1.md5 MD5

MessageDigest Object Info:
Algorithm = MD5
Provider = SUN version 1.2
Digest Length = 16
toString = MD5 Message Digest from SUN, <initialized>

Message Digest Processing Info:
Number of input bytes = 2061
Number of output bytes = 16

java -cp . HexWriter digest_1.md5 digest_1_md5.hex

type digest_1_md5.hex
7772FF57E50B447FE979CAE36C520206
```

A couple of interesting notes here:

- The first command is a special trick I learned from DOS system. It allows you to enter data into a file directly from keyboard without using any editor. In the command, "con" is used as a special file name to represent the console, keyboard and monitor. When entering data from keyboard, Control-Z must be used to simulate the end-of-file sign.
- The second command adds the one-byte file, dot.tmp, to the end of the class file, JcaMessageDigest.class.
- The third command generates MD5 digest data from the modified class file. The new digest data is very different than old digest data.
- One surprise is that the modified class file is still working! Why JVM is not checking the class file?

JCA - Private and Public Key Pairs

This chapter describes some parts of the JCA (Java Cryptography Architecture) which has been included in JDK since 1.1:

- What are private and public key pairs?
- Classes in JDK to generate private and public key pairs.
- A sample program on generating private and public key pairs.

Sample programs listed in this chapter have been tested with JDK 1.3.1, 1.4.1 and 1.5.0.

What Are Private and Public Key Pairs?

Private and public key pairs are used in public key encryption algorithms.

Known private and public key pair generation algorithms are:

- RSA - Developed by MIT professors: Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman in 1977.
- DSA - The Digital Signature Algorithm.
- DiffieHellman

The Key Interfaces

Private and public key pairs are represented 3 interfaces.

1. `java.security.Key` is the interface acting as a base to support common features of both private key and public key. Major methods include:

`getAlgorithm()` - Returns the algorithm name used to generate the key.

`getEncoded()` - Returns the key as a byte array in its primary encoding format, or null if this key does not support encoding.

`getFormat()` - Returns the name of the primary encoding format of this key, or null if this key does not support encoding.

2. `java.security.PrivateKey` is the interface representing a private key. It extends `java.security.Key` interface with no additional methods.

3. `java.security.PublicKey` is the interface representing a public key. It extends `java.security.Key` interface with no additional methods.

The KeyPair Class

java.security.KeyPair is a final class representing a key pair (a public key and a private key). Major methods in the MessageDigest class:

getPrivate() - Returns a PrivateKey object representing the private key in the key pair.

getPublic() - Returns a PublicKey object representing the public key in the key pair.

The KeyPairGenerator Class

java.security.KeyPairGenerator is an abstract class providing a link to implementation classes of private and public key pair generation algorithms provided by various security package providers. Major methods in the KeyPairGenerator class:

getInstance() - Returns a KeyPairGenerator object of the specified algorithm from the implementation of the specified provider. If provider is not specified, the default implementation is used. This is a static method.

initialize() - Initializes the key pair generator with the specified key size.

generateKeyPair() - Generates a key pair and returns a KeyPair object.

getAlgorithm() - Returns the algorithm name of the current key pair generator object.

getProvider() - Returns the provider as a provider object of the current key pair generator object.

Key Pair Sample Program - JcaKeyPair.java

The following sample program shows you how to invoke the key pair generation algorithms implemented by the default provider, Sun, and generate key pairs.

```
/**
 * JcaKeyPair.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.math.*;
import java.security.*;
import java.security.interfaces.*;
class JcaKeyPair {
    public static void main(String[] a) {
        if (a.length<3) {
            System.out.println("Usage:");
            System.out.println("java JcaKeyPair keySize output"
                +" algorithm");
            return;
        }
        int keySize = Integer.parseInt(a[0]);
        String output = a[1];
        String algorithm = a[2]; // RSA, DSA
        try {
```

```

        getKeys(keySize, output, algorithm);
    } catch (Exception e) {
        System.out.println("Exception: "+e);
        return;
    }
}
private static void getKeys(int keySize, String output,
    String algorithm) throws Exception {
    KeyPairGenerator kg = KeyPairGenerator.getInstance(algorithm);
    kg.initialize(keySize);
    System.out.println();
    System.out.println("KeyPairGenerator Object Info: ");
    System.out.println("Algorithm = "+kg.getAlgorithm());
    System.out.println("Provider = "+kg.getProvider());
    System.out.println("Key Size = "+keySize);
    System.out.println("toString = "+kg.toString());
    KeyPair pair = kg.generateKeyPair();
    PrivateKey priKey = pair.getPrivate();
    PublicKey pubKey = pair.getPublic();
    String fl = output+".pri";
    FileOutputStream out = new FileOutputStream(fl);
    byte[] ky = priKey.getEncoded();
    out.write(ky);
    out.close();
    System.out.println();
    System.out.println("Private Key Info: ");
    System.out.println("Algorithm = "+priKey.getAlgorithm());
    System.out.println("Saved File = "+fl);
    System.out.println("Size = "+ky.length);
    System.out.println("Format = "+priKey.getFormat());
    System.out.println("toString = "+priKey.toString());
    fl = output+".pub";
    out = new FileOutputStream(fl);
    ky = pubKey.getEncoded();
    out.write(ky);
    out.close();
    System.out.println();
    System.out.println("Public Key Info: ");
    System.out.println("Algorithm = "+pubKey.getAlgorithm());
    System.out.println("Saved File = "+fl);
    System.out.println("Size = "+ky.length);
    System.out.println("Format = "+pubKey.getFormat());
    System.out.println("toString = "+pubKey.toString());
}
}

```

Here is the result of my first test. It is done with JDK 1.3.1.

```

javac -classpath . JcaKeyPair.java
java -cp . JcaKeyPair 512 dsa dsa

KeyPairGenerator Object Info:
Algorithm = DSA
Provider = SUN version 1.2
Key Size = 512
toString = sun.security.provider.DSAKeyPairGenerator@2f6684

Private Key Info:
Algorithm = DSA
Saved File = dsa.pri

```

```

Size = 201
Format = PKCS#8
toString = Sun DSA Private Key
parameters:
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddcc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
x: 3a46e9a6da9a90ee7c7cfedad597e260988f4e6a

Public Key Info:
Algorithm = DSA
Saved File = dsa.pub
Size = 244
Format = X.509
toString = Sun DSA Public Key
Parameters:
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddcc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
y:
e803dccb c3292909 c589b7ca c3a18e97 d09b5a84 5b90e26d 525f6cb2 d10e987a
4dc7309b 706e8901 eca22c15 9d172763 619067a7 ec2cf389 b73c6133 7630d9cd

```

The program seems to be working:

- Since I am not specifying the provider name, the implementation of the DSA algorithm provided in the default security package was selected. Of course, Sun is the provider of the default security package.
- The key pair generated from the `generateKeyPair()` method indeed has two keys, a private key and a public key.
- The private key was written to a file using PKCS#8 format, and the public key was written to another file using X.509 format.

In order to see the keys, I need to use my other program, `HexWriter.java`, to convert binary data to hex numbers. See chapter "Encoding Conversion" for details.

Here is how to look at the keys data in hex numbers, 16 bytes per line:

```

javac HexWriter.java
java -cp . HexWriter dsa.pri dsa_pri.hex
type dsa_pri.hex
3081C60201003081A806072A8648CE38
040130819C024100FCA682CE8E12CABA

```

```

26EFCF7110E526DB078B05EDECBCD1E
B4A208F3AE1617AE01F35B91A47E6DF6
3413C5E12ED0899BCD132ACD50D99151
BDC43EE737592E17021500962EDDCC36
9CBA8EBB260EE6B6A126D9346E38C502
40678471B27A9CF44EE91A49C5147DB1
A9AAF244F05A434D6486931D2D14271B
9E35030B71FD73DA179069B32E293563
0E1C2062354D0DA20A6C416E50BE794C
A4041602143A46E9A6DA9A90EE7C7CFE
DAD597E260988F4E6A

```

```
java -cp . HexWriter dsa.pub dsa_pub.hex
```

```

type dsa_pub.hex
3081F13081A806072A8648CE38040130
819C024100FCA682CE8E12CABA26EFCF
F7110E526DB078B05EDECBCD1EB4A208
F3AE1617AE01F35B91A47E6DF63413C5
E12ED0899BCD132ACD50D99151BDC43E
E737592E17021500962EDDCC369CBA8E
BB260EE6B6A126D9346E38C502406784
71B27A9CF44EE91A49C5147DB1A9AAF2
44F05A434D6486931D2D14271B9E3503
0B71FD73DA179069B32E2935630E1C20
62354D0DA20A6C416E50BE794CA40344
00024100E803DCCBC3292909C589B7CA
C3A18E97D09B5A845B90E26D525F6CB2
D10E987A4DC7309B706E8901ECA22C15
9D172763619067A7EC2CF389B73C6133
7630D9CD

```

Now let's see the private key and public key generated by the RSA algorithm:

```
java -cp . JcaKeyPair 512 rsa rsa
```

```
KeyPairGenerator Object Info:
```

```
Algorithm = rsa
```

```
Provider = SunRsaSign version 1.0
```

```
Key Size = 512
```

```
toString = java.security.KeyPairGenerator$Delegate@7172ea
```

```
Private Key Info:
```

```
Algorithm = RSA
```

```
Saved File = rsa.pri
```

```
Size = 346
```

```
Format = PKCS8
```

```
toString = com.sun.rsajca.JSA_RSAPrivateKey@45a877
```

```
Public Key Info:
```

```
Algorithm = RSA
```

```
Saved File = rsa.pub
```

```
Size = 94
```

```
Format = X509
```

```
toString = com.sun.rsajca.JSA_RSAPublicKey@2f0db
```

```
java -cp . HexWriter rsa.pri rsa_pri.hex
```

```
type rsa_pri.hex
```

```

30820156020100300D06092A864886F7
0D0101010500048201403082013C0201

```



```

00024100D1565D698FA30419CED84049
7E346F00F3F451D6C43315850B13C639
AE25EEE009FD9A574251127A2F9A14A0
47ADBB2DC39253D2858E8F4A0BC6DDF7
D09DEE7F020301000102410094CAA43E
C1BF86CCA93F0002BCD79C06BB5F6F44
0DD4997925347AF2C89B61BB470DB3D6
3B53B03AF3F8D0B09F1451974E0D7283
C2A742956B18CC4F53AE8541022100F4
52141F336B9FB350523381E0F65D437B
52C15F39CE8A32C4FC41B47527870F02
2100DB582BFD8E19628FA0E250C2DDCA
A72CBA1D62C35F62D4085FBD9C622216
A191022100DAAC0FE8711C759E87585B
5376A6EA88787B7429A7D28D6CAD545D
F68718EECF022068670D79577489BF47
9AB14A2F7B781B07F358C119B3D6ACAD
B578AC1501A4A1022100C7BEBD09E4D6
C020542841FF966FF3646CDB8F5C62E2
1E7E29E367C69DF70FD4

```

```
java -cp . HexWriter rsa.pub rsa_pub.hex
```

```

type rsa_pub.hex
305C300D06092A864886F70D01010105
00034B003048024100D1565D698FA304
19CED840497E346F00F3F451D6C43315
850B13C639AE25EEE009FD9A57425112
7A2F9A14A047ADBB2DC39253D2858E8F
4A0BC6DDF7D09DEE7F0203010001

```

Comparing the default implementation of DSA algorithm with RSA algorithm, the DSA implementation returns all the internal parameters of the keys in the toString() method.

Now let's see the private key and public key generated by the DiffieHellman algorithm:

```
java -cp . JcaKeyPair 512 diff DiffieHellman
```

```

KeyPairGenerator Object Info:
Algorithm = DiffieHellman
Provider = SunJCE version 1.5
Key Size = 512
toString = java.security.KeyPairGenerator$Delegate@66848c

Private Key Info:
Algorithm = DH
Saved File = diff.pri
Size = 228
Format = PKCS#8
toString = SunJCE Diffie-Hellman Private Key:
x:
0590104f 019c57c1 c6cfd079 6e7805d2 6961fea1 99d1ab50 5c5d9c1f b6859e21
2e312451 880c6727 c71010e2 f8dbb489 0d2f8f26 bea1b064 84ff5e17 0e6afa05
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
l:

```

507

Public Key Info:

Algorithm = DH

Saved File = diff.pub

Size = 226

Format = X.509

toString = SunJCE Diffie-Hellman Public Key:

y:

1d5d094c 75a3264e b65f16e1 16caeb20 5a9bde14 27d6653f da7ed69c da7c4800
df18a6d2 0314bbac 65eaed6b 5b11fc09 02a422ea ec648c67 ff70f202 4ee62444

p:

fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17

g:

678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4

l:

507

java -cp . HexWriter diff.pri diff_pri.hex

Number of input bytes: 228

type diff_pri.hex

3081E102010030819706092A864886F7
0D010301308189024100FCA682CE8E12
CABA26EFCCF7110E526DB078B05EDECB
CD1EB4A208F3AE1617AE01F35B91A47E
6DF63413C5E12ED0899BCD132ACD50D9
9151BDC43EE737592E170240678471B2
7A9CF44EE91A49C5147DB1A9AAF244F0
5A434D6486931D2D14271B9E35030B71
FD73DA179069B32E2935630E1C206235
4D0DA20A6C416E50BE794CA4020201FB
044202400590104F019C57C1C6CFD079
6E7805D26961FEA199D1AB505C5D9C1F
B6859E212E312451880C6727C71010E2
F8DBB4890D2F8F26BEA1B06484FF5E17
0E6AFA05

java -cp . HexWriter diff.pub diff_pub.hex

Number of input bytes: 226

type diff_pub.hex

3081DF30819706092A864886F70D0103
01308189024100FCA682CE8E12CABA26
EFCCF7110E526DB078B05EDECB
CD1EB4A208F3AE1617AE01F35B91A47E6DF634
13C5E12ED0899BCD132ACD50D99151BD
C43EE737592E170240678471B27A9CF4
4EE91A49C5147DB1A9AAF244F05A434D
6486931D2D14271B9E35030B71FD73DA
179069B32E2935630E1C2062354D0DA2
0A6C416E50BE794CA4020201FB034300
02401D5D094C75A3264EB65F16E116CA
EB205A9BDE1427D6653FDA7ED69CDA7C
4800DF18A6D20314BBAC65EAED6B5B11
FC0902A422EAEC648C67FF70F2024EE6
2444

Conclusion

- JDK 1.5.0 supports 3 algorithms to generate private and public pairs: RSA, DSA, and DiffieHellman.

Question: Key pairs are used to encrypt and decrypt data. But how to do this? Which classes and methods to use?

JCA - Key Factory - Encoding Tool

This chapter describes some parts of the JCA (Java Cryptography Architecture) which has been included in JDK since 1.1:

- What is key encoding?
- Key factory class and encoded key specification classes.
- A sample program to show you how to read in private and public key pairs.

What is Key Encoding?

Key encoding is the process of converting an encryption or decryption key into a specific encoding format for storing and transmitting.

JDK offers a group of classes to support key encoding:

- `KeyFactory` - A class to convert keys between `Key` objects and `EncodedKeySpec` objects.
- `EncodedKeySpec` - An abstract class offers a grouping point of all sub classes that represent various key encoding standards.
- `PKCS8EncodedKeySpec` - A sub class of `EncodedKeySpec` represents the ASN.1 encoding of a private key based PKCS#8 standard.
- `X509EncodedKeySpec` - A sub class of `EncodedKeySpec` represents the ASN.1 encoding of a public key based on X.509 standard.

`java.security.spec.PKCS8EncodedKeySpec` Class

`java.security.spec.PKCS8EncodedKeySpec` - A sub class of `EncodedKeySpec` represents the ASN.1 encoding of a private key based PKCS#8 standard. It has three methods:

- `PKCS8EncodedKeySpec()` - Constructs a `PKCS8EncodedKeySpec` object from the specified byte array that contains the PKCS#8 encoded private key.
- `getEncoded()` - Returns the encoded key in a byte array of this object.
- `getFormat()` - Returns the name of encoding used in this object.

`java.security.spec.X509EncodedKeySpec` Class

`java.security.spec.X509EncodedKeySpec` - A sub class of `EncodedKeySpec` represents the ASN.1 encoding of a private key based X.509 standard. It has three methods:

- `X509EncodedKeySpec()` - Constructs a `X509EncodedKeySpec` object from the specified byte array that contains the X509 encoded public key.
- `getEncoded()` - Returns the encoded key in a byte array of this object.
- `getFormat()` - Returns the name of encoding used in this object.

java.security.KeyFactory Class

java.security.KeyFactory - A class to convert keys between Key objects and EncodedKeySpec objects. Major methods offered:

getInstance() - Returns a KeyFactory object of the specified algorithm and the specified security package provider. If not specified, the default security package provider will be used.

generatePrivate() - Generates a PrivateKey object based on the specified EncodedKeySpec object, and returns it.

generatePublic() - Generates a PublicKey object based on the specified EncodedKeySpec object, and returns it.

getKeySpec() - Converts a Key object to an EncodedKeySpec object, and returns it.

getAlgorithm() - Returns the algorithm name of this object.

getProvider() - Returns the provider as a provider object of this object.

Key Factory Sample Program - JcaKeyFactoryTest.java

The following sample program shows you how to use the KeyFactory class to read in encoded keys from files:

```
/**
 * JcaKeyFactoryTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.math.*;
import java.security.*;
import java.security.interfaces.*;
import java.security.spec.*;
class JcaKeyFactoryTest {
    public static void main(String[] a) {
        if (a.length<3) {
            System.out.println("Usage:");
            System.out.println("java JcaKeyFactoryTest keySize output"
                +" algorithm");
            return;
        }
        int keySize = Integer.parseInt(a[0]);
        String output = a[1];
        String algorithm = a[2]; // RSA, DSA
        try {
            writeKeys(keySize,output,algorithm);
            readKeys(output,algorithm);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
}
```

```

    }
}
private static void writeKeys(int keySize, String output,
    String algorithm) throws Exception {
    KeyPairGenerator kg = KeyPairGenerator.getInstance(algorithm);
    kg.initialize(keySize);
    System.out.println();
    System.out.println("KeyPairGenerator Object Info: ");
    System.out.println("Algorithm = "+kg.getAlgorithm());
    System.out.println("Provider = "+kg.getProvider());
    System.out.println("Key Size = "+keySize);
    System.out.println("toString = "+kg.toString());
    KeyPair pair = kg.generateKeyPair();
    PrivateKey priKey = pair.getPrivate();
    PublicKey pubKey = pair.getPublic();

    String fl = output+".pri";
    FileOutputStream out = new FileOutputStream(fl);
    byte[] ky = priKey.getEncoded();
    out.write(ky);
    out.close();
    System.out.println();
    System.out.println("Private Key Info: ");
    System.out.println("Algorithm = "+priKey.getAlgorithm());
    System.out.println("Saved File = "+fl);
    System.out.println("Length = "+ky.length);
    System.out.println("Format = "+priKey.getFormat());
    System.out.println("toString = "+priKey.toString());

    fl = output+".pub";
    out = new FileOutputStream(fl);
    ky = pubKey.getEncoded();
    out.write(ky);
    out.close();
    System.out.println();
    System.out.println("Public Key Info: ");
    System.out.println("Algorithm = "+pubKey.getAlgorithm());
    System.out.println("Saved File = "+fl);
    System.out.println("Length = "+ky.length);
    System.out.println("Format = "+pubKey.getFormat());
    System.out.println("toString = "+pubKey.toString());
}
private static void readKeys(String input,
    String algorithm) throws Exception {
    KeyFactory keyFactory = KeyFactory.getInstance(algorithm);
    System.out.println();
    System.out.println("KeyFactory Object Info: ");
    System.out.println("Algorithm = "+keyFactory.getAlgorithm());
    System.out.println("Provider = "+keyFactory.getProvider());
    System.out.println("toString = "+keyFactory.toString());

    String priKeyFile = input+".pri";
    FileInputStream priKeyStream = new FileInputStream(priKeyFile);
    int priKeyLength = priKeyStream.available();
    byte[] priKeyBytes = new byte[priKeyLength];
    priKeyStream.read(priKeyBytes);
    priKeyStream.close();
    PKCS8EncodedKeySpec priKeySpec
        = new PKCS8EncodedKeySpec(priKeyBytes);
    PrivateKey priKey = keyFactory.generatePrivate(priKeySpec);
    System.out.println();
    System.out.println("Private Key Info: ");

```

```

        System.out.println("Algorithm = "+priKey.getAlgorithm());
        System.out.println("Saved File = "+priKeyFile);
        System.out.println("Length = "+priKeyBytes.length);
        System.out.println("toString = "+priKey.toString());

        String pubKeyFile = input+".pub";
        FileInputStream pubKeyStream = new FileInputStream(pubKeyFile);
        int pubKeyLength = pubKeyStream.available();
        byte[] pubKeyBytes = new byte[pubKeyLength];
        pubKeyStream.read(pubKeyBytes);
        pubKeyStream.close();
        X509EncodedKeySpec pubKeySpec
            = new X509EncodedKeySpec(pubKeyBytes);
        PublicKey pubKey = keyFactory.generatePublic(pubKeySpec);
        System.out.println();
        System.out.println("Public Key Info: ");
        System.out.println("Algorithm = "+pubKey.getAlgorithm());
        System.out.println("Saved File = "+pubKeyFile);
        System.out.println("Length = "+pubKeyBytes.length);
        System.out.println("toString = "+pubKey.toString());
    }
}

```

Here is the result of my first test. It is done with JDK 1.3.1.

```

javac -classpath . JcaKeyFactoryTest.java
java -cp . JcaKeyFactoryTest 512 dsa DSA

KeyPairGenerator Object Info:
Algorithm = DSA
Provider = SUN version 1.2
Key Size = 512
toString = sun.security.provider.DSAKeyPairGenerator@738798

Private Key Info:
Algorithm = DSA
Saved File = dsa.pri
Length = 202
Format = PKCS#8
toString = Sun DSA Private Key
parameters:
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
x: 91a8c9462732fa91aa4cbc97ca7ae43c9bcac41b

Public Key Info:
Algorithm = DSA
Saved File = dsa.pub
Length = 244
Format = X.509
toString = Sun DSA Public Key
Parameters:
p:

```

```

fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
y:
8f799b3a eead766a 163c603d e1008521 4c946a94 98abb83e e5cbd6c0 1c38a0a6
3a23dac4 69065303 22c6fde7 10d3b567 b4c0b89f df6971e7 30178015 2a23d77f

KeyFactory Object Info:
Algorithm = DSA
Provider = SUN version 1.2
toString = java.security.KeyFactory@26e431

Private Key Info:
Algorithm = DSA
Saved File = dsa.pri
Length = 202
toString = Sun DSA Private Key
parameters:DSA
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
x: 91a8c9462732fa91aa4cbc97ca7ae43c9bcac41b

Public Key Info:
Algorithm = DSA
Saved File = dsa.pub
Length = 244
toString = Sun DSA Public Key
Parameters:DSA
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
y:
8f799b3a eead766a 163c603d e1008521 4c946a94 98abb83e e5cbd6c0 1c38a0a6
3a23dac4 69065303 22c6fde7 10d3b567 b4c0b89f df6971e7 30178015 2a23d77f

```

The program seems to be working:

- A pair of keys is generated using the DSA algorithm first.
- The private key is then stored in a file using the default encoding format.
- The public key is also stored in a file using the default encoding format.
- A KeyFactory object is then created with DSA algorithm using the default security package provider.

- The private key is then read into the program and decoded into PrivateKey object with the help of PKCS8EncodedKeySpec class.
- The public key is also read into the program and decoded into PublicKey object with the help of X509EncodedKeySpec class.

Here is the result of my second test with RSA algorithm.

```
java -cp . JcaKeyFactoryTest 512 rsa RSA

KeyPairGenerator Object Info:
Algorithm = RSA
Provider = SunRsaSign version 1.0
Key Size = 512
toString = java.security.KeyPairGenerator$Delegate@2f6684

Private Key Info:
Algorithm = RSA
Saved File = rsa.pri
Length = 343
Format = PKCS8
toString = com.sun.rsajca.JSA_RSAPrivateKey@372a1a

Public Key Info:
Algorithm = RSA
Saved File = rsa.pub
Length = 94
Format = X509
toString = com.sun.rsajca.JSA_RSAPublicKey@12d342

KeyFactory Object Info:
Algorithm = RSA
Provider = SunRsaSign version 1.0
toString = java.security.KeyFactory@5ff48b

Private Key Info:
Algorithm = RSA
Saved File = rsa.pri
Length = 343
toString = com.sun.rsajca.JSA_RSAPrivateKey@fe7c3

Public Key Info:
Algorithm = RSA
Saved File = rsa.pub
Length = 94
toString = com.sun.rsajca.JSA_RSAPublicKey@3e2d65
```

JCA - Digital Signature

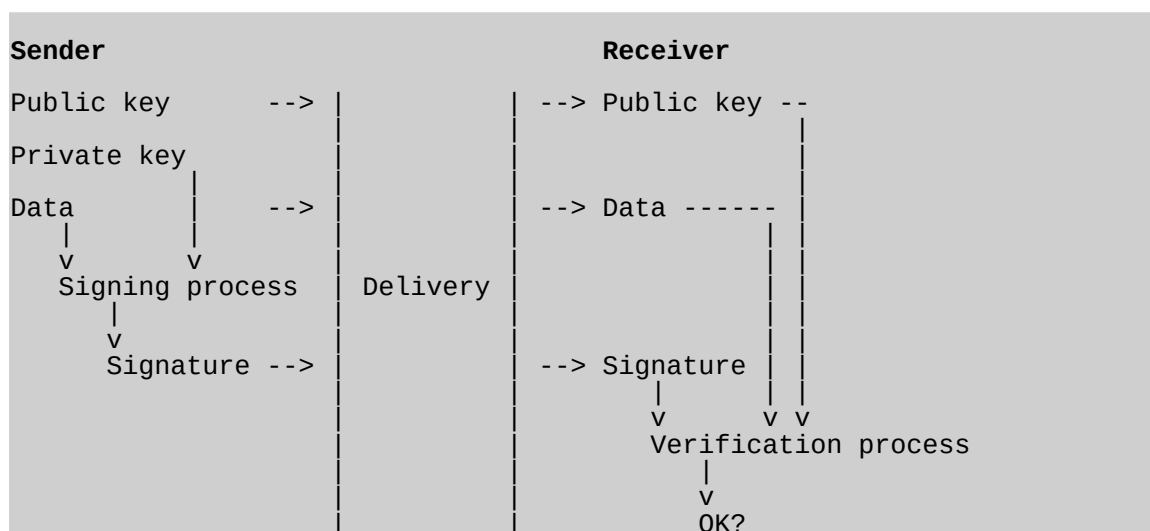
This chapter describes some parts of the JCA (Java Cryptography Architecture) which has been included in JDK since 1.1:

- What digital signature?
- The Signature class in JDK to generate and verify digital signature.
- A sample program to show you how to use the Signature class.
- ..

What is Digital Signature?

Digital signature is a computer technology to protect digital data in two security areas: authentication and integrity.

The following diagram illustrates a common way to applying the digital signature technology to protect your data when sending it to someone else:



As you can see in the above diagram, in order to apply the digital signature technology, the sender needs to:

- Generate a private and public key pair.
- Generate a signature from the data file with the private key.
- Send the data, the signature, and the public key to the receiver.

The receiver needs to:

- Verify the data with the signature and the public key.

The Signing Process and the Verification Process

Of course, the most important part of the digital signature technology is the signing process and the verification process.

The goal of the signing process is to produce a signature based on the original data, and the sender's private key. One way to do this is:

- Generate a message digest out of the data.
- Encrypt the message digest with the private key.
- Collect the encrypted message digest as the signature.

Of course, the verification process needs to match the signing process:

- Decrypt the received signature with the public key. The decrypted signature is the message digest of the original data.
- Generate a message digest out of the received data using the same algorithm as in the signing process.
- Comparing the message digest decrypted from the signature with the message digest generated from the received data. If they match, signature is verified ok. If not, verification failed.

The Signature Class

`java.security.Signature` is an abstract class providing a link to implementation classes of digital signature algorithms provided by various security package providers. Major methods in the `KeyPairGenerator` class:

`getInstance()` - Returns a `Signature` object of the specified algorithm from the implementation of the specified provider. If provider is not specified, the default implementation is used. This is a static method.

`initSign()` - Initializes the current `Signature` object with the specified private key to be ready to take input data for generating a new signature.

`initVerify()` - Initializes the current `Signature` object with the specified public key to be ready to take input data for verifying an existing signature.

`update()` - Adds more data to the current `Signature` object for signature generation or signature verification.

`sign()` - Generates a new signature for the input data received so far in the current `Signature` object, and returns the signature as a byte array. It also removes the input data.

`verify()` - Verifies the input data received so far in the current `Signature` object against the specified signature, and returns true or false. It also removes the input data.

`getAlgorithm()` - Returns the algorithm name of the current `Signature` object.

getProvider() - Returns the provider as a Provider object of the current Signature object.

Signature Sample Program - JcaSignatureTest.java

The following sample program shows you how to use signature algorithms provided by the default provider, generate signatures and verify the signatures.

```
/**
 * JcaSignatureTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
class JcaSignatureTest {
    public static void main(String[] a) {
        if (a.length<4) {
            System.out.println("Usage:");
            System.out.println("java JcaSignatureTest input output"
                +" keyAlgo signAlgo");
            return;
        }
        String input = a[0];
        String output = a[1];
        String keyAlgo = a[2];
        String signAlgo = a[3];
        try {
            KeyPair pair = getKeys(keyAlgo);
            PrivateKey priKey = pair.getPrivate();
            PublicKey pubKey = pair.getPublic();
            byte[] sign = sign(input,output,signAlgo,priKey);
            verify(input,signAlgo,sign,pubKey);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
    private static KeyPair getKeys(String algorithm) throws Exception {
        KeyPairGenerator kg = KeyPairGenerator.getInstance(algorithm);
        int keySize = 512;
        kg.initialize(keySize);
        KeyPair pair = kg.generateKeyPair();
        return pair;
    }
    private static byte[] sign(String input, String output,
        String algorithm, PrivateKey priKey) throws Exception {
        Signature sg = Signature.getInstance(algorithm);
        sg.initSign(priKey);
        System.out.println();
        System.out.println("Signature Object Info: ");
        System.out.println("Algorithm = "+sg.getAlgorithm());
        System.out.println("Provider = "+sg.getProvider());
        FileInputStream in = new FileInputStream(input);
        int bufSize = 1024;
        byte[] buffer = new byte[bufSize];
        int n = in.read(buffer,0,bufSize);
        int count = 0;
        while (n!=-1) {
            count += n;
        }
    }
}
```

```

        sg.update(buffer,0,n);
        n = in.read(buffer,0,bufSize);
    }
    in.close();
    FileOutputStream out = new FileOutputStream(output);
    byte[] sign = sg.sign();
    out.write(sign);
    out.close();
    System.out.println();
    System.out.println("Sign Processing Info: ");
    System.out.println("Number of input bytes = "+count);
    System.out.println("Number of output bytes = "+sign.length);
    return sign;
}
private static boolean verify(String input, String algorithm,
    byte[] sign, PublicKey pubKey) throws Exception {
    Signature sg = Signature.getInstance(algorithm);
    sg.initVerify(pubKey);
    System.out.println();
    System.out.println("Signature Object Info: ");
    System.out.println("Algorithm = "+sg.getAlgorithm());
    System.out.println("Provider = "+sg.getProvider());
    FileInputStream in = new FileInputStream(input);
    int bufSize = 1024;
    byte[] buffer = new byte[bufSize];
    int n = in.read(buffer,0,bufSize);
    int count = 0;
    while (n!=-1) {
        count += n;
        sg.update(buffer,0,n);
        n = in.read(buffer,0,bufSize);
    }
    in.close();
    boolean ok = sg.verify(sign);
    System.out.println("Verify Processing Info: ");
    System.out.println("Number of input bytes = "+count);
    System.out.println("Verification result = "+ok);
    return ok;
}
}

```

Here is the result of my first test. It is done with JDK 1.3.1.

```

javac -classpath . JcaSignatureTest.java

java -cp . JcaSignatureTest JcaSignatureTest.class
    sign.dsa DSA SHA1withDSA

Signature Object Info:
Algorithm = SHA1withDSA
Provider = SUN version 1.2

Sign Processing Info:
Number of input bytes = 3154
Number of output bytes = 46

Signature Object Info:
Algorithm = SHA1withDSA
Provider = SUN version 1.2
Verify Processing Info:
Number of input bytes = 3154

```

```
Verification result = true

java HexWriter sign.dsa sign_dsa.hex
Number of input bytes: 46

type sign_dsa.hex
302C0214008C8936A9A4C5B68D297C23
F867351E036231C00214199BF3F4FF82
C4C4AC4A086F2013290A8FAEFC05
```

The program seems to be working:

- A pair of keys is generated using the DSA algorithm first.
- Then a signature is generated for input file, JcaSignatureTest.class, using the SHA1withDSA algorithm, which uses SHA-1 for the message digest generation, and DSA for encryption.
- Finally, the signature is verified with the public key and the result shows that the signature is valid.

Of course, you can choose other combinations of different key pair generation algorithms and message digest algorithms. For example:

```
java -cp . JcaSignatureTest JcaSignatureTest.class
    sign.rsa RSA SHA1withRSA

Signature Object Info:
Algorithm = SHA1withRSA
Provider = SunRsaSign version 1.0

Sign Processing Info:
Number of input bytes = 3154
Number of output bytes = 64

Signature Object Info:
Algorithm = SHA1withRSA
Provider = SunRsaSign version 1.0
Verify Processing Info:
Number of input bytes = 3154
Verification result = true

java -cp . JcaSignatureTest JcaSignatureTest.class
    sign_2.rsa RSA MD5withRSA

Signature Object Info:
Algorithm = MD5withRSA
Provider = SunRsaSign version 1.0

Sign Processing Info:
Number of input bytes = 3154
Number of output bytes = 64

Signature Object Info:
Algorithm = MD5withRSA
Provider = SunRsaSign version 1.0
Verify Processing Info:
Number of input bytes = 3154
Verification result = true
```

```
java -cp . JcaSignatureTest JcaSignatureTest.class
sign_3.rsa RSA MD2withRSA
```

```
Signature Object Info:
Algorithm = MD2withRSA
Provider = SunRsaSign version 1.0
```

```
Sign Processing Info:
Number of input bytes = 3154
Number of output bytes = 64
```

```
Signature Object Info:
Algorithm = MD2withRSA
Provider = SunRsaSign version 1.0
Verify Processing Info:
Number of input bytes = 3154
Verification result = true
```

This sample program generates the signature and verify the signature in single program. But in the real world, we must have at least two programs, one to be used by the sender of the data to generate the signature, and the other to be used by the receiver to verify the signature.

Signature Generation Program - JcaSign.java

The following program is a standalone program that reads in an input file and a private key file, and generates a signature file based on the specified digital signature algorithm.

```
/**
 * JcaSign.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
import java.security.spec.*;
class JcaSign {
    public static void main(String[] a) {
        if (a.length<5) {
            System.out.println("Usage:");
            System.out.println("java JcaSign input signFile signAlgo"
                + " keyFile keyAlgo");
            return;
        }
        String input = a[0];
        String signFile = a[1];
        String signAlgo = a[2]; // SHA1withDSA, SHA1withRSA,
        String keyFile = a[3];
        String keyAlgo = a[4]; // DSA, RSA

        try {
            PrivateKey priKey = readPrivateKey(keyFile, keyAlgo);
            sign(input, signFile, signAlgo, priKey);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
    private static PrivateKey readPrivateKey(String input,
```

```

        String algorithm) throws Exception {
    KeyFactory keyFactory = KeyFactory.getInstance(algorithm);
    System.out.println();
    System.out.println("KeyFactory Object Info: ");
    System.out.println("Algorithm = "+keyFactory.getAlgorithm());
    System.out.println("Provider = "+keyFactory.getProvider());
    System.out.println("toString = "+keyFactory.toString());

    FileInputStream priKeyStream = new FileInputStream(input);
    int priKeyLength = priKeyStream.available();
    byte[] priKeyBytes = new byte[priKeyLength];
    priKeyStream.read(priKeyBytes);
    priKeyStream.close();
    PKCS8EncodedKeySpec priKeySpec
        = new PKCS8EncodedKeySpec(priKeyBytes);
    PrivateKey priKey = keyFactory.generatePrivate(priKeySpec);
    System.out.println();
    System.out.println("Private Key Info: ");
    System.out.println("Algorithm = "+priKey.getAlgorithm());
    System.out.println("Saved File = "+input);
    System.out.println("Length = "+priKeyBytes.length);
    System.out.println("toString = "+priKey.toString());
    return priKey;
}
private static byte[] sign(String input, String output,
    String algorithm, PrivateKey priKey) throws Exception {
    Signature sg = Signature.getInstance(algorithm);
    sg.initSign(priKey);
    System.out.println();
    System.out.println("Signature Object Info: ");
    System.out.println("Algorithm = "+sg.getAlgorithm());
    System.out.println("Provider = "+sg.getProvider());
    FileInputStream in = new FileInputStream(input);
    int bufSize = 1024;
    byte[] buffer = new byte[bufSize];
    int n = in.read(buffer, 0, bufSize);
    int count = 0;
    while (n != -1) {
        count += n;
        sg.update(buffer, 0, n);
        n = in.read(buffer, 0, bufSize);
    }
    in.close();
    FileOutputStream out = new FileOutputStream(output);
    byte[] sign = sg.sign();
    out.write(sign);
    out.close();
    System.out.println();
    System.out.println("Sign Processing Info: ");
    System.out.println("Number of input bytes = "+count);
    System.out.println("Number of output bytes = "+sign.length);
    return sign;
}
}

```

As you can see, this program also uses the KeyFactory class to read in the private key stored in an encoded file, which can be generated by my other program, JcaKeyPair.java.

Here is result of my first test using DSA as the key generation algorithm, and SHA1withDSA as the digital signature algorithm. It is done with JDK 1.3.1.


```

java -cp . JcaKeyPair 512 dsa DSA
java -cp . JcaSign JcaSign.class JcaSign_dsa.sgn SHA1withDSA dsa.pri DSA

KeyFactory Object Info:
Algorithm = DSA
Provider = SUN version 1.2
toString = java.security.KeyFactory@53c015

Private Key Info:
Algorithm = DSA
Saved File = dsa.pri
Length = 201
toString = Sun DSA Private Key
parameters:DSA
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
x: 4bf261ad9e69a12993a48f63b43b81c61f2aebfd

Signature Object Info:
Algorithm = SHA/DSA
Provider = SUN version 1.2

Sign Processing Info:
Number of input bytes = 2940
Number of output bytes = 46

```

The program seems to be working:

- A pair of keys is generated using the DSA algorithm first. The private key is stored in dsa.pri, and the public key in dsa.pub.
- The private key is decoded from the file using the KeyFactory class in PKCS#8 format.
- Then a signature is generated for input file, JcaSign.class, using the SHA1withDSA algorithm, which uses SHA-1 for the message digest generation, and DSA for encryption.
- The signature is stored in JcaSign_dsa.sgn.

Now try it with the RSA key generation algorithm. You should have no problem at all.

```

java -cp . JcaKeyPair 512 rsa RSA
java -cp . JcaSign JcaSign.class JcaSign_rsa.sgn MD2withRSA rsa.pri RSA

```

Signature Verification Program - JcaVerify.java

The following program is a standalone program that reads in an input file, a signature file and a public key file, and verifies that if the signature file matches the input file based on

the specified digital signature algorithm.

```
/**
 * JcaVerify.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
import java.security.spec.*;
class JcaVerify {
    public static void main(String[] a) {
        if (a.length<5) {
            System.out.println("Usage:");
            System.out.println("java JcaVerify input signFile"
                +" signAlgo keyFile keyAlgo");
            return;
        }
        String input = a[0];
        String signFile = a[1];
        String signAlgo = a[2]; // SHA1withDSA, SHA1withRSA,
        String keyFile = a[3];
        String keyAlgo = a[4]; // DSA, RSA
        try {
            PublicKey pubKey = readPublicKey(keyFile, keyAlgo);
            byte[] sign = readSignature(signFile);
            verify(input, signAlgo, sign, pubKey);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
    private static PublicKey readPublicKey(String input,
        String algorithm) throws Exception {
        FileInputStream pubKeyStream = new FileInputStream(input);
        int pubKeyLength = pubKeyStream.available();
        byte[] pubKeyBytes = new byte[pubKeyLength];
        pubKeyStream.read(pubKeyBytes);
        pubKeyStream.close();
        X509EncodedKeySpec pubKeySpec
            = new X509EncodedKeySpec(pubKeyBytes);
        KeyFactory keyFactory = KeyFactory.getInstance(algorithm);
        PublicKey pubKey = keyFactory.generatePublic(pubKeySpec);
        System.out.println();
        System.out.println("Public Key Info: ");
        System.out.println("Algorithm = "+pubKey.getAlgorithm());
        System.out.println("Saved File = "+input);
        System.out.println("Length = "+pubKeyBytes.length);
        System.out.println("toString = "+pubKey.toString());
        return pubKey;
    }
    private static byte[] readSignature(String input)
        throws Exception {
        FileInputStream signStream = new FileInputStream(input);
        int signLength = signStream.available();
        byte[] signBytes = new byte[signLength];
        signStream.read(signBytes);
        signStream.close();
        return signBytes;
    }
    private static boolean verify(String input, String algorithm,
```

```

        byte[] sign, PublicKey pubKey) throws Exception {
    Signature sg = Signature.getInstance(algorithm);
    sg.initVerify(pubKey);
    System.out.println();
    System.out.println("Signature Object Info: ");
    System.out.println("Algorithm = "+sg.getAlgorithm());
    System.out.println("Provider = "+sg.getProvider());
    FileInputStream in = new FileInputStream(input);
    int bufSize = 1024;
    byte[] buffer = new byte[bufSize];
    int n = in.read(buffer,0,bufSize);
    int count = 0;
    while (n!=-1) {
        count += n;
        sg.update(buffer,0,n);
        n = in.read(buffer,0,bufSize);
    }
    in.close();
    boolean ok = sg.verify(sign);
    System.out.println("Verify Processing Info: ");
    System.out.println("Number of input bytes = "+count);
    System.out.println("Verification result = "+ok);
    return ok;
}
}

```

Here is result of my first test to verify a signature generated with the DSA algorithm and the SHA1withDSA algorithm. See the previous section for more information.

```

java -cp . JcaVerify JcaSign.class JcaSign.sgn SHA1withDSA dsa.pub DSA
Public Key Info:
Algorithm = DSA
Saved File = dsa.pub
Length = 243
toString = Sun DSA Public Key
Parameters:DSA
p:
fca682ce 8e12caba 26efccf7 110e526d b078b05e decbcd1e b4a208f3 ae1617ae
01f35b91 a47e6df6 3413c5e1 2ed0899b cd132acd 50d99151 bdc43ee7 37592e17
q:
962edddc 369cba8e bb260ee6 b6a126d9 346e38c5
g:
678471b2 7a9cf44e e91a49c5 147db1a9 aaf244f0 5a434d64 86931d2d 14271b9e
35030b71 fd73da17 9069b32e 2935630e 1c206235 4d0da20a 6c416e50 be794ca4
y:
2cf7c9ba a2dd3ce5 0e2d6384 122951fa a956ebd9 573a5168 ce9c0a83 b8c63348
ce5c5d65 b35c8c81 beb9206b 8cf36587 0b24e605 a0150dbe a40dd57d 64b42b8a
Signature Object Info:
Algorithm = SHA/DSA
Provider = SUN version 1.2
Verify Processing Info:
Number of input bytes = 2940
Verification result = true

```

Yes. The program is working correctly. Here is another on the signature generated with RSA and MD2withRSA algorithms.

```
java -cp . JcaVerify JcaSign.class JcaSign_rsa.sgn MD2withRSA rsa.pub RSA

Public Key Info:
Algorithm = RSA
Saved File = rsa.pub
Length = 94
toString = com.sun.rsajca.JSA_RSAPublicKey@12e78c

Signature Object Info:
Algorithm = MD2withRSA
Provider = SunRsaSign version 1.0
Verify Processing Info:
Number of input bytes = 2940
Verification result = true
```

Conclusions:

- Generating and verification digital signatures are done by encrypting and decrypting the message digest of the input data with private keys and public keys.
- The Signature class requires the help of the KeyPairGenerator class and the KeyFactory class.
- Sun provides several digital signature algorithms: SHA1withDSA, SHA1withRSA, MD2withDSA, and MD2withRSA.
- Of course, you must choose a digital signature algorithm that is compatible with the keys.
- Keys are stored in files in encoded formats. The default encoding for private key is PKCS#8, and the default encoding for public key is X.509.

JCA - Certificates, 'keytool' and 'keystore'

This chapter describes some parts of the JCA (Java Cryptography Architecture) which has been included in JDK since 1.1:

- What is a certificate and a certificate chain?
- What is "keystore"?
- What are the functions offered by "keytool"?
- Examples of using "keytool".

Certificates and Certificate Chains

Certificate: A digitally signed statement from the issuer saying that the public key of the subject has some specific value.

The above definition is copied from the JDK 1.3.1 documentation. It has a couple of important terms:

- "signed statement" - The certificate must be signed by the issuer with a digital signature.
- "issuer" - The person or organization who is issuing this certificate.
- "public key" - The public key of a key pair selected by the subject.
- "subject" - The person or organization who owns the public key.

X.509 Certificate - A certificate written in X.509 standard format. X.509 standard was introduced in 1988. It requires a certificate to have the following information:

- Version - X.509 standard version number.
- Serial Number - A sequence number given to each certificate.
- Signature Algorithm Identifier - Name of the algorithm used to sign this certificate by the issuer
- Issuer Name - Name of the issuer.
- Validity Period - Period during which this certificate is valid.
- Subject Name - Name of the owner of the public key.
- Subject Public Key Information - The public key and its related information.

How can you get a certificate for your own public key?

- Requesting it from a Certificate Authority (CA), like VeriSign, Thawte or Entrust.
- Doing it yourself - using tools like JDK "keytool" to generate a self-signed certificate.

Certificate Chain: A series of certificates that one certificate signs the public key of the issuer of the next certificate. Usually the top certificate (the first certificate) is self-signed, where issuer signed its own public key.

What is "keystore"?

"keystore" - A database used by JDK "keytool" command and KeyStore class to store your own private keys, and public key certificates you received from someone else.

"keystore" supports the following features:

- Two types of entries: key entries for private keys and certificate entries for public key certificates.
- A key entry contains the private key and a certificate chain of the corresponding public key.
- Every entry has a unique alias name.
- Key entries are protected by separate passwords.
- "keystore" may have different implementations from different security package providers. The default implementation from Sun is called JKS.

"keytool" - Key and Certificate Management Tool

"keytool" is command line tool introduced in JDK 1.2 to manage keys and certificates using "keystore". "keytool" replaces the same functions offered by "jvakey" in JDK 1.1. "keytool" offers a number functions through the following major command options:

- "-genkey": Generates a key pair and stores it as a key entry in the keystore.
- "-list": Lists all entries in the keystore.
- "-export": Exports the certificate of the specified key entry or certificate entry out of the keystore to a certificate file.
- "-printcert": Prints summary information of a certificate from a certificate file.
- "-import": Imports the certificate from a certificate file as a certificate entry into the keystore.
- "-keyclone": Creates a new key entry by copying an existing key entry.
- "-selfcert": Replaces the certificate in a key entry with a new self-signed certificate.
- "-delete": Deletes the entry of the specified alias name.

"keytool" Example - Generating Key Pairs and Self-Signed Certificates

In the first example, I want to try the "-genkey" command option using JDK 1.3.1:

```
keytool -genkey -alias my_home -keystore herong.jks
Enter keystore password: HerongJKS
What is your first and last name?
[Unknown]: Herong Yang
What is the name of your organizational unit?
[Unknown]: My Unit
What is the name of your organization?
[Unknown]: My Home
What is the name of your City or Locality?
```

```
[Unknown]: My City
What is the name of your State or Province?
[Unknown]: My State
What is the two-letter country code for this unit?
[Unknown]: US
Is <CN=Herong Yang, OU=My Unit, O=My Home, L=My City, ST=My State,
C=US> correct?
[no]: yes
Enter key password for <my_home>
(RETURN if same as keystore password): My1stKey
```

Based on the documentation, the above example command should do the following for me:

- Create a "keystore" file, herong.jks, in JKS format, with password of "HerongJKS".
- Generate a pair of private key and public key for me using the default implementation of the default security package.
- Generate a certificate chain with a single self-signed certificate of my public key.
- Insert a key entry into the keystore with my private key and the certificate chain.

The following command shows that we do have a key entry in the keystore file:

```
keytool -list -keystore herong.jks -storepass HerongJKS

Keystore type: jks
Keystore provider: SUN

Your keystore contains 1 entry:

my_home, Sat Jun 1 07:15:16 EDT 2002, keyEntry,
Certificate fingerprint
(MD5): BE:D2:AF:4E:A7:44:13:08:16:4C:68:3B:D1:99:79:55
```

"keytool" Example - Exporting and Import Certificates

In the second example, I want to export the certificate stored in the key entry to a certificate file, then import it back into the keystore as certificate entry:

```
keytool -export -alias my_home -file my_home.crt -keystore herong.jks
-storepass HerongJKS
Certificate stored in file <my_home.crt>

keytool -printcert -file my_home.crt
Owner: CN=Herong Yang, OU=My Unit, O=My Home, L=My City, ST=My Sta...
Issuer: CN=Herong Yang, OU=My Unit, O=My Home, L=My City, ST=My St...
Serial number: 407928a4
Valid from: Sat Jun 1 07:14:44 EDT 2002 until: Sat Aug 31 07:14:44...
Certificate fingerprints:
    MD5: BE:D2:AF:4E:A7:44:13:08:16:4C:68:3B:D1:99:79:55
    SHA1: AE:67:0C:C5:21:5C:F6:6F:45:33:9E:FB:8E:50:EA:32:32:D1:92:BB

keytool -import -alias my_home.crt -file my_home.crt
-keystore herong.jks -storepass HerongJKS
Certificate already exists in keystore under alias <my_home>
Do you still want to add it? [no]: yes
Certificate was added to keystore
```

```
keytool -list -keystore herong.jks -storepass HerongJKS

Keystore type: jks
Keystore provider: SUN

Your keystore contains 2 entries:

my_home.crt, Sat Jun 1 12:25:46 EDT 2004, trustedCertEntry,
Certificate fingerprint (MD5): BE:D2:AF:4E:A7:44:13:08:16:4C:68:3B...
my_home, Sat Jun 1 07:15:16 EDT 2002, keyEntry,
Certificate fingerprint (MD5): BE:D2:AF:4E:A7:44:13:08:16:4C:68:3B...
```

Looking good so far:

- The "-export" command option exports the self-signed certificate of my public key into a file, my_home.crt.
- The "-printcert" command option prints out summary information of a certificate stored in a file in X.509 format. As you can see from the print out, I am the issuer and the owner of this certificate.
- The "-import" command option imports the certificate from the certificate file back into the keystore under different alias, my_home.crt.

Certificates can also be exported in a printable format: based on RFC 1421 specification, using the BASE64 encoding algorithm.

```
keytool -export -alias my_home.crt -file my_home.rfc -rfc
-keystore herong.jks -storepass HerongJKS
Certificate stored in file <my_home.rfc>

type my_home.rfc
-----BEGIN CERTIFICATE-----
MIIDDTCCAssCBEB5KKQwCwYHKOZIZjgEAWUAMGwxCzAJBgNVBAYTAlVMTREwDwYDVQ...
dGF0ZTEQMA4GA1UEBxMHTXkgQ2l0eTEQMA4GA1UEChMHTXkgSG9tZTEQMA4GA1UEC...
dDEUMBIGA1UEAxMLSGVyb25nIFlhbmcwHhcNMDQwNDExMTExNDQ0WhcNMDQwNzEwMT...
MQswCQYDVQQGEwJVUzERMA8GA1UECBMITXkgU3RhdGUxEDA0BgNVBACTB015IENpdH...
BA0TB015IEhvbWUxEDA0BgNVBAsTB015IFVuaXQxZDASBgNVBAMTC0hlcm9uZyBZYW...
ASwGBYqGSM44BAEwggEfAoGBAP1/U4EddRIpUt9KnC7s50f2EbdSP09EAMMeP4C2US...
WT2NWPq/xfw6MPbLm1Vs14E7gB00b/JmYLdrnVClpJ+f6AR7ECLCT7up1/63xhv401...
+4P208UewwI1VBNAFpEy9nXzrith1yrv8iIDGZ3RSAHHAhUAL2BQjxUjC8yykrmCou...
gYEA9+GghdabPd7LvKtcNrHxXmUr7v60uqC+VdMCz0HgmdRWVe0utRZT+ZxBxCBGL...
Fh03zwkyjMim4TwEotUfI0o4K0uHiuzpnWRbqN/C/ohNWLx+2J6ASQ7zKTxvqhRkI...
BpKLZl6Ae1U1ZAFM0/7PSSoDgYUAAoGBAjdQsMLIf1nh4T/HZvVeltsrTGED118CkG...
ygy530LwrSK+6ptJpXP8tPMn9YFVJ3eigJrMTaZvGyd40WRiYM6Woyj3T4H73LEKLD...
QeNYOAm8cp3l9ZQkNnmIA1P6CRR43EeAmdTUlK8y6RWTsr0iJMdDMAsGByqGSM44BA...
AhQ4zAU0PWe1wdiwe9XDsVPcKS1xwIUWtdok6RIeeCMRIytKwcT0o7/qpM=
-----END CERTIFICATE-----
```

"keytool" Example - Cloning Certificates with New Identities

In the third example, I want to create a new key entry with the same key pair of an existing key entry, but change the identity information:

```
keytool -keyclone -alias my_home -dest my_copy -keypass My1stKey
-new My2ndKey -keystore herong.jks -storepass HerongJKS
```



```
keytool -selfcert -alias my_copy -keypass My2ndKey
-dname "cn=Herong Yang, ou=My Unit 2, o=My Organization 2, c=US"
-keystore herong.jks -storepass HerongJKS

keytool -export -alias my_copy -file my_copy.crt -keystore herong.jks
-storepass HerongJKS
Certificate stored in file <my_copy.crt>

keytool -printcert -file my_copy.crt
Owner: CN=Herong Yang, OU=My Unit 2, O=My Organization 2, C=US
Issuer: CN=Herong Yang, OU=My Unit 2, O=My Organization 2, C=US
Serial number: 40798b4f
Valid from: Sat Jun 1 14:15:43 EDT 2002 until: Sat Aug 31 14:15:43...
Certificate fingerprints:
    MD5: 4A:E4:D9:BC:E9:8C:50:27:6C:00:59:76:D1:14:05:79
    SHA1: FA:F5:30:78:22:3B:52:28:0D:41:24:0B:CA:CC:6F:D4:0E:...
```

Conclusion:

- A key entry in keystore contains a private key and a certificate of the public key.
- Certificates can be exported into certificate files out of keystore.
- Certificates can be imported from certificate back into keystore.
- There seems be to no way to export private keys.
- There seems be to no way to generate a certificate of a given public key.

JCA - KeyStore and Related Classes

This chapter describes some parts of the JCA (Java Cryptography Architecture) which has been included in JDK since 1.1:

- The Certificate class.
- The CertificateFactory class.
- The KeyStore class.
- Example programs

Certificate Class

`java.security.cert.Certificate` is an abstract class representing common features of different implementations of different certificate types. Its main methods are:

- `getEncoded()` - Returns a byte array of the encoded form of this certificate. The encoding format used depends on the certificate type. For "X.509" type of certificates, the encoding format will be "ASN.1 DER".
- `getPublicKey()` - Returns the public key of signed in this certificate.
- `getType()` - Returns the type of this certificate.
- `verify()` - Verifies this certificate with the specified public key. I am not sure why this method has no return value.

There is a deprecated interface, `java.security.Certificate`, in JDK with the same name as `Certificate`, which could confuse the compiler. To help the compiler, you may need to use the full class name, `java.security.cert.Certificate`, in your code.

CertificateFactory Class

`java.security.cert.CertificateFactory` is a class to read in different types of certificates from certificate files.

- `getIntance()` - Returns a `CertificateFactory` object of the specified certificate type using the implementation of the specified security package provider. If not specified, the default provider will be used.
- `generateCertificate()` - Read in a certificate from the specified input stream, and returns it as a `Certificate` object. It can read certificate in both binary (DER encoded) and printable (RFC standard) formats. Note that only the first certificate in the input stream will be processed.
- `generateCertificates()` - Read in multiple certificates from the specified input stream, and returns them as a `Certificate` array. It can read certificate in both binary (DER encoded) and printable (RFC standard) formats.
- `getType()` - Returns the type of this certificate factory.
- `getProvider()` - Returns the security package provider.

JcaCertificateTest.java - Sample Program

In the following sample program, I want to show you how to read in a certificate from a certificate file, and write it out to a certificate file in the default encoding format.

```
/**
 * JcaCertificateTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
import java.security.cert.*;
class JcaCertificateTest {
    public static void main(String[] a) {
        if (a.length<2) {
            System.out.println("Usage:");
            System.out.println("java JcaCertificateFactoryTest input"
                +" output");
            return;
        }
        String input = a[0];
        String output = a[1];
        try {
            test(input,output);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
    private static void test(String input, String output)
        throws Exception {
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        System.out.println();
        System.out.println("CertificateFactory Object Info: ");
        System.out.println("Type = "+cf.getType());
        System.out.println("Provider = "+cf.getProvider());
        System.out.println("toString = "+cf.toString());

        FileInputStream fis = new FileInputStream(input);
        java.security.cert.Certificate cert = cf.generateCertificate(fis);
        fis.close();
        System.out.println();
        System.out.println("Certificate Object Info: ");
        System.out.println("Type = "+cert.getType());
        System.out.println("toString = "+cert.toString());

        PublicKey pubKey = cert.getPublicKey();
        System.out.println();
        System.out.println("PublicKey Object Info: ");
        System.out.println("Algorithm = "+pubKey.getAlgorithm());
        System.out.println("Format = "+pubKey.getFormat());
        System.out.println("toString = "+pubKey.toString());

        FileOutputStream fos = new FileOutputStream(output);
        byte[] certBytes = cert.getEncoded();
        fos.write(certBytes);
        fos.close();
    }
}
```

Here is the result of my first test. It is done with JDK 1.3.1.

```
java -cp . JcaCertificateTest my_home.crt temp.crt

CertificateFactory Object Info:
Type = X.509
Provider = SUN version 1.2
toString = java.security.cert.CertificateFactory@7f5ea7

Certificate Object Info:
Type = X.509
toString = [
[
  Version: V1
  Subject: CN=Herong Yang, OU=My Unit, O=My Home, L=My City, ST=My...
  Signature Algorithm: SHA1withDSA, OID = 1.2.840.10040.4.3

  Key: Sun DSA Public Key
    Parameters: DSA
      p: fd7f5381 1d751229 52df4a9c 2eece4e7 f611b752 3cef44...
        455d4022 51fb593d 8d58fabf c5f5ba30 f6cb9b55 6cd7813b 801d346f...
        6b9950a5 a49f9fe8 047b1022 c24fbba9 d7feb7c6 1bf83b57 e7c6a8a6...
        83f6d3c5 1ec30235 54135a16 9132f675 f3ae2b61 d72aeff2 2203199d...
      q: 9760508f 15230bcc b292b982 a2eb840b f0581cf5
      g: f7e1a085 d69b3dde cbbcab5c 36b857b9 7994afbb fa3aea...
        5159578e bad4594f e6710710 8180b449 167123e8 4c281613 b7cf0932...
        3c167a8b 547c8d28 e0a3ae1e 2bb3a675 916ea37f 0bfa2135 62f1fb62...
        cca4f1be a8519089 a883dfe1 5ae59f06 928b665e 807b5525 64014c3b...

    y:
      9750b0c9 487f59e1 e13fc766 f55e96db 2b4c6103 d75f0290 6424893b...
      ca0cb9dc e2f0ad22 beea9b49 a573fcb4 f327f581 552777a2 809acc4d...
      78d16462 60ce96a3 28f74f81 fbdcb10a 2c3c9a29 33cf0a40 f541e358...
      9de5f594 24367988 0353fa09 1478dc47 8099d4d4 94af32e9 1593b2b3...

  Validity: [From: Sat Jun 1 07:14:44 EDT 2002,
              To: Sat Aug 31 07:14:44 EDT 2002]
  Issuer: CN=Herong Yang, OU=My Unit, O=My Home, L=My City, ST=My ...
  SerialNumber: [ 407928a4 ]

]
  Algorithm: [SHA1withDSA]
  Signature:
0000: 30 2C 02 14 38 CC 05 0E    3D 67 B5 C1 D8 B0 C9 EF    0,..8...=...
0010: 57 0E C5 4F 70 A4 B5 C7    02 14 59 37 68 93 A4 48    W..Op.....
0020: 79 E0 8C 44 8C AD 2B 07    13 3A 8E FF AA 93          y..D..+.....

]

PublicKey Object Info:
Algorithm = DSA
Format = X.509
toString = Sun DSA Public Key
  Parameters: DSA
    p: fd7f5381 1d751229 52df4a9c 2eece4e7 f611b752 3cef44...
      455d4022 51fb593d 8d58fabf c5f5ba30 f6cb9b55 6cd7813b 801d346f...
      6b9950a5 a49f9fe8 047b1022 c24fbba9 d7feb7c6 1bf83b57 e7c6a8a6...
      83f6d3c5 1ec30235 54135a16 9132f675 f3ae2b61 d72aeff2 2203199d...
    q: 9760508f 15230bcc b292b982 a2eb840b f0581cf5
    g: f7e1a085 d69b3dde cbbcab5c 36b857b9 7994afbb fa3aea...
```

```
5159578e bad4594f e6710710 8180b449 167123e8 4c281613 b7cf0932...
3c167a8b 547c8d28 e0a3ae1e 2bb3a675 916ea37f 0bfa2135 62f1fb62...
cca4f1be a8519089 a883dfe1 5ae59f06 928b665e 807b5525 64014c3b...
```

y:

```
9750b0c9 487f59e1 e13fc766 f55e96db 2b4c6103 d75f0290 6424893b...
ca0cb9dc e2f0ad22 beea9b49 a573fcb4 f327f581 552777a2 809acc4d...
78d16462 60ce96a3 28f74f81 fbdcb10a 2c3c9a29 33cf0a40 f541e358...
9de5f594 24367988 0353fa09 1478dc47 8099d4d4 94af32e9 1593b2b3...
```

The result looks very good:

- The input certificate file was created by "keytool", see the previous chapter for details. It was written in a binary format (DER).

The program can also be used to read in certificate files written in printable format (RFC). Assuming my_home.rfc is a certificate file created by "keytool" in RFC format, the following command will read in the certificate in my_home.rfc without any problem:

```
java -cp . JcaCertificateTest my_home.rfc temp.crt
```

KeyStore Class

java.security.KeyStore is a class representing a key store database in memory. It offers the following major methods:

- `getIntance()` - Returns a KeyStore object of the specified key store type of the implementation of the specified security package provider. If not specified, the default provider will be used.
- `load()` - Deletes all entries and loads all entries of a key store from the specified input stream with the specified password.
- `store()` - Stores all entries to the output stream with the specified password.
- `size()` - Returns the number of entries.
- `aliases()` - Returns alias names of all entries as Enumeration object.
- `deleteEntry()` - Deletes the entry of the specified alias name.
- `getCertificate()` - Returns the certificate of the entry of the specified alias name. If the specified entry is a key entry, it returns the first certificate of the certificate chain.
- `getCertificateChain()` - Returns the certificate chain as a Certificate array of the entry of the specified alias name. If the specified entry is a certificate entry, it returns null.
- `getKey()` - Returns the key of the entry of the specified alias name with the specified password. If the specified entry is a certificate entry, it returns null.
- `setCertificateEntry()` - Inserts a new certificate entry with the specified alias name and the specified certificate.
- `setKeyEntry()` - Inserts a new key entry with the specified alias name, the specified certificate chain, and the specified password.
- `isKeyEntry()` - Returns true if the entry with the specified alias name is a key entry.
- `isCertificateEntry()` - Returns true if the entry with the specified alias name is a certificate entry.

- `getType()` - Returns the type of this KeyStore.
- `getProvider()` - Returns the security package provider of this KeyStore.

JcaKeyStoreTest.java - Sample Program

The following sample program shows you how to load a keystore database from a file into a KeyStore object, extracting a certificate from the keystore into a certificate file, then importing the certificate back into the keystore.

```
/**
 * JcaKeyStoreTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.util.*;
import java.security.*;
import java.security.cert.*;
class JcaKeyStoreTest {
    public static void main(String[] a) {
        if (a.length<3) {
            System.out.println("Usage:");
            System.out.println("java JcaKeyStoreTest store sPass alias");
            return;
        }
        String store = a[0];
        String sPass = a[1];
        String alias = a[2];
        try {
            test(store,sPass,alias);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
    private static void test(String store, String sPass, String alias)
        throws Exception {
        KeyStore ks = KeyStore.getInstance("JKS");
        System.out.println();
        System.out.println("KeyStore Object Info: ");
        System.out.println("Type = "+ks.getType());
        System.out.println("Provider = "+ks.getProvider());
        System.out.println("toString = "+ks.toString());

        FileInputStream fis = new FileInputStream(store);
        ks.load(fis,sPass.toCharArray());
        fis.close();
        System.out.println();
        System.out.println("KeyStore Content: ");
        System.out.println("Size = "+ks.size());
        Enumeration e = ks.aliases();
        while (e.hasMoreElements()) {
            String name = (String) e.nextElement();
            System.out.print("    "+name+": ");
            if (ks.isKeyEntry(name)) System.out.println(" Key entry");
            else System.out.println(" Certificate entry");
        }

        java.security.cert.Certificate cert = ks.getCertificate(alias);
    }
}
```

```

        System.out.println();
        System.out.println("Certificate Object Info: ");
        System.out.println("Type = "+cert.getType());
        System.out.println("toString = "+cert.toString());

        FileOutputStream fos = new FileOutputStream(alias+".crt");
        byte[] certBytes = cert.getEncoded();
        fos.write(certBytes);
        fos.close();

        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        System.out.println();
        System.out.println("CertificateFactory Object Info: ");
        System.out.println("Type = "+cf.getType());
        System.out.println("Provider = "+cf.getProvider());
        System.out.println("toString = "+cf.toString());

        fis = new FileInputStream(alias+".crt");
        cert = cf.generateCertificate(fis);
        ks.setCertificateEntry(alias+ks.size(), cert);
        fis.close();

        fos = new FileOutputStream(store);
        ks.store(fos, sPass.toCharArray());
        fos.close();
    }
}

```

Here is the result of my first test. It uses the key store file generated from the "keytool" command. See the previous chapter for details.

```

java -cp . JcaKeyStoreTest herong.jks HerongJKS my_home

KeyStore Object Info:
Type = JKS
Provider = SUN version 1.2
toString = java.security.KeyStore@6ec612

KeyStore Content:
Size = 4
  my_home_2: Certificate entry
  my_home: Key entry
  his_home: Key entry
  my_copy: Key entry

Certificate Object Info:
Type = X.509
toString = [
  [
    Version: V1
    Subject: CN=Herong Yang, OU=My Unit, O=My Home, L=My City, ST=My...
    Signature Algorithm: SHA1withDSA, OID = 1.2.840.10040.4.3
    Key: Sun DSA Public Key
    Parameters: DSA
  ...
  Issuer: CN=Herong Yang, OU=My Unit, O=My Home, L=My City, ST=My ...
  SerialNumber: [ 407928a4 ]
  ]
  Algorithm: [SHA1withDSA]
  Signature:
0000: 30 2C 02 14 38 CC 05 0E 3D 67 B5 C1 D8 B0 C9 EF 0,...8...=...

```

```
0010: 57 0E C5 4F 70 A4 B5 C7 02 14 59 37 68 93 A4 48 W..Op.....
0020: 79 E0 8C 44 8C AD 2B 07 13 3A 8E FF AA 93 y..D..+.....
]
```

CertificateFactory Object Info:

Type = X.509

Provider = SUN version 1.2

toString = java.security.cert.CertificateFactory@3ab50a

Note that the extracted certificate is imported back into the key store a new certificate entry. You can see the new entry, my_home4, if you run the program again:

```
java -cp . JcaKeyStoreTest herong.jks HerongJKS my_home
```

KeyStore Object Info:

Type = JKS

Provider = SUN version 1.2

toString = java.security.KeyStore@6ec612

KeyStore Content:

Size = 5

my_home_2: Certificate entry

my_home4: Certificate entry

my_home: Key entry

his_home: Key entry

my_copy: Key entry

Certificate Object Info:

...

Conclusion:

- KeyStore class offers about the same functionalities of the "keytool" command.
- There seems be to no way to generate a certificate chain.
- There seems be to no way to generate a certificate of a given public key.

JCE - Cipher - Encryption and Decryption

This chapter describes some parts of the JCE (Java Cryptography Extension) which has been introduced in JDK since 1.1:

- What is a secret key?
- Classes in JDK to generate a secret key.
- Classes in JDK to write and read a secret key to and from a file.
- A sample program to generate, write, and read a secret key.

What Is a Secret Key?

A secret key is the key used in a symmetric encryption algorithm, where the same key is used both the encryption process and the decryption process.

Known symmetric encryption algorithms:

- Blowfish - The block cipher designed by Bruce Schneier.
- DES - The Digital Encryption Standard as described in FIPS PUB 46-2.
- DESede - Triple DES Encryption (DES-EDE).

The SecretKey Interface

`javax.crypto.SecretKey` is an interface providing a grouping point for various secret keys. It extends `java.security.Key`, and inherits 3 methods:

`getAlgorithm()` - Returns the algorithm name used to generate the key.

`getEncoded()` - Returns the key as a byte array in its primary encoding format.

`getFormat()` - Returns the name of the primary encoding format of this key.

The KeyGenerator Class

`javax.crypto.KeyGenerator` is an abstract class providing a link to implementation classes of secret key generation algorithms provided by various security package providers. Major methods in the `KeyGenerator` class:

`getInstance()` - Returns a `KeyGenerator` object of the specified algorithm from the implementation of the specified provider. If provider is not specified, the default implementation is used. This is a static method.

`init()` - Initializes this key generator with the specified key size.

`generateKey()` - Generates a key and returns it as a `SecretKey` object.

getAlgorithm() - Returns the algorithm name of this generator.

getProvider() - Returns the provider as a Provider object of this generator.

Converting Secret Keys to and from Byte Arrays

Converting a secret key to a byte array is supported by the SecretKey interface with the getEncoded() method. All secret key implementation classes use RAW the encoding format.

Converting a secret key from a byte array is not so easy. Two options are available:

- Using the generic SecretKeySpec class to convert a byte array directly to a SecretKey object directly.
- Using algorithm specific KeySpec implementations to convert a byte array to a KeySpec object first, then using SecretKeyFactory to convert a KeySpec object to a SecretKey object.

javax.crypto.spec.SecretKeySpec is a class represents a secret key in a generic fashion. It offers the following constructor and methods:

SecretKeySpec() - Convert a secret key from the specified byte array according to the specified algorithm and constructs a SecretKeySpec object based on the secret key.

getAlgorithm() - Returns the algorithm name used to generate the key.

getEncoded() - Returns the key as a byte array in its primary encoding format.

getFormat() - Returns the name of the primary encoding format of this key.

java.security.spec.KeySpec is an interface providing a grouping point for algorithm specific key specification implementations. There are two implementations for secret key algorithms:

javax.crypto.spec.DESKeySpec is a KeySpec implementation for DES algorithm. It offers DESKeySpec() to construct a KeySpec object from the specific byte array.

javax.crypto.spec.DESEDEKeySpec is a KeySpec implementation for DESede algorithm. It offers DESEDEKeySpec() to construct a KeySpec object from the specific byte array.

javax.crypto.spec.SecretKeyFactory is a class as a conversion tool between SecretKey objects and algorithm specific KeySpec objects. It offers the following methods:

getInstance() - Returns a SecretKeyFactory object of the specified algorithm and the specified security package provider. If not specified, the default security package provider will be used.

generateSecret() - Generates a SecretKey object from the specified KeySpec object, and returns it.

getKeySpec() - Converts a SecretKey object to a KeySpec object, and returns it.

getAlgorithm() - Returns the algorithm name of this object.

getProvider() - Returns the provider as a provider object of this object.

JceSecretKeyTest.java - Sample Program

The following sample program shows you how to generate a secret key, write it a file, and read it back.

```
/**
 * JceSecretKeyTest.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.spec.*;
import javax.crypto.*;
import javax.crypto.spec.*;
class JceSecretKeyTest {
    public static void main(String[] a) {
        if (a.length<3) {
            System.out.println("Usage:");
            System.out.println("java JceSecretKeyTest keySize output"
                +" algorithm");
            return;
        }
        int keySize = Integer.parseInt(a[0]);
        String output = a[1];
        String algorithm = a[2]; // Blowfish, DES, DESede, HmacMD5
        try {
            writeKey(keySize,output,algorithm);
            readKey(output,algorithm);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }
    private static void writeKey(int keySize, String output,
        String algorithm) throws Exception {
        KeyGenerator kg = KeyGenerator.getInstance(algorithm);
        kg.init(keySize);
        System.out.println();
        System.out.println("KeyGenerator Object Info: ");
        System.out.println("Algorithm = "+kg.getAlgorithm());
        System.out.println("Provider = "+kg.getProvider());
        System.out.println("Key Size = "+keySize);
        System.out.println("toString = "+kg.toString());

        SecretKey ky = kg.generateKey();
        String fl = output+".key";
        FileOutputStream fos = new FileOutputStream(fl);
        byte[] kb = ky.getEncoded();
```

```

        fos.write(kb);
        fos.close();
        System.out.println();
        System.out.println("SecretKey Object Info: ");
        System.out.println("Algorithm = "+ky.getAlgorithm());
        System.out.println("Saved File = "+fl);
        System.out.println("Size = "+kb.length);
        System.out.println("Format = "+ky.getFormat());
        System.out.println("toString = "+ky.toString());
    }
    private static void readKey(String input, String algorithm)
        throws Exception {
        String fl = input+".key";
        FileInputStream fis = new FileInputStream(fl);
        int kl = fis.available();
        byte[] kb = new byte[kl];
        fis.read(kb);
        fis.close();
        KeySpec ks = null;
        SecretKey ky = null;
        SecretKeyFactory kf = null;
        if (algorithm.equalsIgnoreCase("DES")) {
            ks = new DESKeySpec(kb);
            kf = SecretKeyFactory.getInstance("DES");
            ky = kf.generateSecret(ks);
        } else if (algorithm.equalsIgnoreCase("DESEde")) {
            ks = new DESEdeKeySpec(kb);
            kf = SecretKeyFactory.getInstance("DESEde");
            ky = kf.generateSecret(ks);
        } else {
            ks = new SecretKeySpec(kb, algorithm);
            ky = new SecretKeySpec(kb, algorithm);
        }

        System.out.println();
        System.out.println("KeySpec Object Info: ");
        System.out.println("Saved File = "+fl);
        System.out.println("Length = "+kb.length);
        System.out.println("toString = "+ks.toString());

        System.out.println();
        System.out.println("SecretKey Object Info: ");
        System.out.println("Algorithm = "+ky.getAlgorithm());
        System.out.println("toString = "+ky.toString());
    }
}

```

Here is the result of my first test. It is done with JDK 1.4.1.

```

java -cp . JceSecretKeyTest 56 key1 Blowfish

KeyGenerator Object Info:
Algorithm = Blowfish
Provider = SunJCE version 1.42
Key Size = 56
toString = javax.crypto.KeyGenerator@a8c488

SecretKey Object Info:
Algorithm = Blowfish
Saved File = key1.key
Size = 7

```

```
Format = RAW
toString = javax.crypto.spec.SecretKeySpec@2685020e

KeySpec Object Info:
Saved File = key1.key
Length = 7
toString = javax.crypto.spec.SecretKeySpec@2685020e

SecretKey Object Info:
Algorithm = Blowfish
toString = javax.crypto.spec.SecretKeySpec@2685020e
```

The program seems to be working:

- Since I am not specifying the provider name, the implementation of the Blowfish algorithm provided in the default security package was selected. Of course, Sun is the provider of the default security package.
- The Blowfish key is only 7 bytes when "encoded" in RAW format.
- When importing the blowfish key back from the 7 raw bytes, SecretKeyScep class is used instead of SecretKeyFactory class.

In the second test, I wants to try DES algorithm:

```
java -cp . JceSecretKeyTest 56 key2 DES

KeyGenerator Object Info:
Algorithm = DES
Provider = SunJCE version 1.42
Key Size = 56
toString = javax.crypto.KeyGenerator@998b08

SecretKey Object Info:
Algorithm = DES
Saved File = key2.key
Size = 8
Format = RAW
toString = com.sun.crypto.provider.DESKey@fffe7985

KeySpec Object Info:
Saved File = key2.key
Length = 8
toString = javax.crypto.spec.DESKeySpec@106082

SecretKey Object Info:
Algorithm = DES
toString = com.sun.crypto.provider.DESKey@fffe7985
```

Of course, you can continue testing with DESede and HmacMD5.

JCE - Cipher for Encryption and Decryption

This chapter describes some parts of the JCE (Java Cryptography Extension) which has been introduced in JDK since 1.1:

- The Cipher class.
- How to use cipher with secret keys for encryption and decryption.
- How to use cipher with private and public key pairs for encryption and decryption.

The Cipher Class

`javax.crypto.Cipher` is a class that provides functionality of a cryptographic cipher for encryption and decryption. It has the following major methods.

`getInstance()` - Returns a Cipher object of the specified transformation (algorithm plus options) from the implementation of the specified provider. If provider is not specified, the default implementation is used. This is a static method.

`init()` - Initializes this cipher for the specified operation mode with the specified key or public key certificate. Two important operation modes are `Cipher.ENCRYPT_MODE` and `Cipher.DECRYPT_MODE`.

`update()` - Feeds additional input data to this cipher and generates partial output data.

`doFinal()` - Feeds the last part of the input data to this cipher and generates the last part of the output data.

`getBlockSize()` - Returns the block size of this cipher.

`getAlgorithm()` - Returns the algorithm name of this cipher.

`getProvider()` - Returns the provider as a `Provider` object of this cipher.

JceSecretCipher.java - Cipher with Secret Key

The following sample program shows you how to do encryption and decryption with a secret key:

```
/**
 * JceSecretCipher.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.spec.*;
import javax.crypto.*;
import javax.crypto.spec.*;
class JceSecretCipher {
    public static void main(String[] a) {
```

```

        if (a.length<5) {
            System.out.println("Usage:");
            System.out.println("java JceSecretCipher algorithm mode keyFile"
                +" input output");
            return;
        }
        String algorithm = a[0];
        String mode = a[1];
        String keyFile = a[2];
        String input = a[3];
        String output = a[4];
        try {
            SecretKey ky = readKey(keyFile,algorithm);
            secretCipher(algorithm, mode, ky, input, output);
        } catch (Exception e) {
            System.out.println("Exception: "+e);
            return;
        }
    }

    private static SecretKey readKey(String input, String algorithm)
        throws Exception {
        String fl = input;
        FileInputStream fis = new FileInputStream(fl);
        int kl = fis.available();
        byte[] kb = new byte[kl];
        fis.read(kb);
        fis.close();
        KeySpec ks = null;
        SecretKey ky = null;
        SecretKeyFactory kf = null;
        if (algorithm.equalsIgnoreCase("DES")) {
            ks = new DESKeySpec(kb);
            kf = SecretKeyFactory.getInstance("DES");
            ky = kf.generateSecret(ks);
        } else if (algorithm.equalsIgnoreCase("DESede")) {
            ks = new DESedeKeySpec(kb);
            kf = SecretKeyFactory.getInstance("DESede");
            ky = kf.generateSecret(ks);
        } else {
            ks = new SecretKeySpec(kb,algorithm);
            ky = new SecretKeySpec(kb,algorithm);
        }
        System.out.println();
        System.out.println("KeySpec Object Info: ");
        System.out.println("Saved File = "+fl);
        System.out.println("Length = "+kb.length);
        System.out.println("toString = "+ks.toString());
        System.out.println();
        System.out.println("SecretKey Object Info: ");
        System.out.println("Algorithm = "+ky.getAlgorithm());
        System.out.println("toString = "+ky.toString());
        return ky;
    }

    private static void secretCipher(String algorithm, String mode,
        SecretKey ky, String input, String output) throws Exception {
        Cipher cf = Cipher.getInstance(algorithm);
        if (mode.equalsIgnoreCase("encrypt"))
            cf.init(Cipher.ENCRYPT_MODE,ky);
        else if (mode.equalsIgnoreCase("decrypt"))
            cf.init(Cipher.DECRYPT_MODE,ky);
        else
            throw new Exception("Invalid mode: "+mode);
    }

```

```

        System.out.println();
        System.out.println("Cipher Object Info: ");
        System.out.println("Block Size = "+cf.getBlockSize());
        System.out.println("Algorithm = "+cf.getAlgorithm());
        System.out.println("Provider = "+cf.getProvider());
        System.out.println("toString = "+cf.toString());

        FileInputStream fis = new FileInputStream(input);
        FileOutputStream fos = new FileOutputStream(output);
        int bufSize = 1024;
        byte[] buf = new byte[bufSize];
        int n = fis.read(buf, 0, bufSize);
        int fisSize = 0;
        int fosSize = 0;
        while (n!=-1) {
            fisSize += n;
            byte[] out = cf.update(buf, 0, n);
            fosSize += out.length;
            fos.write(out);
            n = fis.read(buf, 0, bufSize);
        }
        byte[] out = cf.doFinal();
        fosSize += out.length;
        fos.write(out);
        fis.close();
        fos.close();
        System.out.println();
        System.out.println("Cipher Process Info: ");
        System.out.println("Input Size = "+fisSize);
        System.out.println("Output Size = "+fosSize);
    }
}

```

Here is the result of my first test. It is done with JDK 1.4.1.

```

java -cp . JceSecretCipher Blowfish encrypt bfish.key
        JceSecretCipher.java jce.cph

KeySpec Object Info:
Saved File = bfish.key
Length = 7
toString = javax.crypto.spec.SecretKeySpec@2685020e

SecretKey Object Info:
Algorithm = Blowfish
toString = javax.crypto.spec.SecretKeySpec@2685020e

Cipher Object Info:
Block Size = 8
Algorithm = Blowfish
Provider = SunJCE version 1.4
toString = javax.crypto.Cipher@106082

Cipher Process Info:
Input Size = 3684
Output Size = 3688

java -cp . JceSecretCipher Blowfish decrypt bfish.key jce.cph jce.clr

KeySpec Object Info:
Saved File = bfish.key

```



```
Length = 7
toString = javax.crypto.spec.SecretKeySpec@2685020e

SecretKey Object Info:
Algorithm = Blowfish
toString = javax.crypto.spec.SecretKeySpec@2685020e

Cipher Object Info:
Block Size = 8
Algorithm = Blowfish
Provider = SunJCE version 1.4
toString = javax.crypto.Cipher@106082

Cipher Process Info:
Input Size = 3688
Output Size = 3684

comp JceSecretCipher.java jce.clr
Comparing JceSecretCipher.java and jce.clr...
Files compare OK
```

Note that:

- bfish.key is the key file generated by KeyGenerator with the Blowfish algorithm.
- The last command confirms that the encryption process and the decryption process work correctly.
- The block size of Blowfish encryption algorithm is 8 bytes.

You can do more tests with DES and HmacMD5 keys.

JcePublicCipher.java - Cipher with Private and Public Key Pair

The following sample program shows you how to do encryption and decryption with a private and public key pair:

```
/**
 * JcePublicCipher.java
 * Copyright (c) 2002 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
import java.security.spec.*;
import javax.crypto.*;
import javax.crypto.spec.*;
class JcePublicCipher {
    public static void main(String[] a) {
        if (a.length<5) {
            System.out.println("Usage:");
            System.out.println("java JceSecretCipher algorithm mode keyFile"
                +" input output");
            return;
        }
        String algorithm = a[0];
        String mode = a[1];
        String keyFile = a[2];
        String input = a[3];
        String output = a[4];
```

```

    try {
        Key ky = readKey(algorithm, mode, keyFile);
        publicCipher(algorithm, mode, ky, input, output);
    } catch (Exception e) {
        System.out.println("Exception: "+e);
        return;
    }
}

private static Key readKey(String algorithm, String mode,
    String input) throws Exception {
    KeyFactory keyFactory = KeyFactory.getInstance(algorithm);
    System.out.println();
    System.out.println("KeyFactory Object Info: ");
    System.out.println("Algorithm = "+keyFactory.getAlgorithm());
    System.out.println("Provider = "+keyFactory.getProvider());
    System.out.println("toString = "+keyFactory.toString());

    FileInputStream fis = new FileInputStream(input);
    int kl = fis.available();
    byte[] kb = new byte[kl];
    fis.read(kb);
    fis.close();
    Key ky = null;
    if (mode.equalsIgnoreCase("encrypt")) {
        X509EncodedKeySpec pubKeySpec
            = new X509EncodedKeySpec(kb);
        ky = keyFactory.generatePublic(pubKeySpec);
    } else if (mode.equalsIgnoreCase("decrypt")) {
        PKCS8EncodedKeySpec priKeySpec
            = new PKCS8EncodedKeySpec(kb);
        ky = keyFactory.generatePrivate(priKeySpec);
    } else
        throw new Exception("Invalid mode: "+mode);
    System.out.println();
    System.out.println("Key Object Info: ");
    System.out.println("Algorithm = "+ky.getAlgorithm());
    System.out.println("Saved File = "+input);
    System.out.println("Length = "+kl);
    System.out.println("Format = "+ky.getFormat());
    System.out.println("toString = "+ky.toString());
    return ky;
}

private static void publicCipher(String algorithm, String mode,
    Key ky, String input, String output) throws Exception {
    Cipher cf = Cipher.getInstance(algorithm);
    if (mode.equalsIgnoreCase("encrypt"))
        cf.init(Cipher.ENCRYPT_MODE, ky);
    else if (mode.equalsIgnoreCase("decrypt"))
        cf.init(Cipher.DECRYPT_MODE, ky);
    else
        throw new Exception("Invalid mode: "+mode);
    System.out.println();
    System.out.println("Cipher Object Info: ");
    System.out.println("Block Size = "+cf.getBlockSize());
    System.out.println("Algorithm = "+cf.getAlgorithm());
    System.out.println("Provider = "+cf.getProvider());
    System.out.println("toString = "+cf.toString());

    FileInputStream fis = new FileInputStream(input);
    FileOutputStream fos = new FileOutputStream(output);
    int bufSize = 1024;
    byte[] buf = new byte[bufSize];

```

```
int n = fis.read(buf,0,bufSize);
int fisSize = 0;
int fosSize = 0;
while (n!=-1) {
    fisSize += n;
    byte[] out = cf.update(buf,0,n);
    fosSize += out.length;
    fos.write(out);
    n = fis.read(buf,0,bufSize);
}
byte[] out = cf.doFinal();
fosSize += out.length;
fos.write(out);
fis.close();
fos.close();
System.out.println();
System.out.println("Cipher Process Info: ");
System.out.println("Input Size = "+fisSize);
System.out.println("Output Size = "+fosSize);
}
```

Unfortunately, I can not test this sample program, because there is no default implementation of DSA or RSA algorithm.

Installing J2SE 1.5.0 on Windows 2000

Download

- Go to java.sun.com/j2se/1.5.0/download.jsp.
- Click the "Download JDK" hyper link.
- Follow the instructions on the Web pages to download `jdk-1_5_0-windows-i586.exe` to a working directory on your hard disk. This file is about 45 MB.
- Double click on the file name: `jdk-1_5_0-windows-i586.exe` in the working directory in the File Explorer.
- Follow the instruction on the installation window to finish the installation. Remember to specify the target directory to be: `\j2sdk1.5.0`.
- Open a command window to try the java command. If you are getting the following output, your installation was ok: returns with message:

```
\j2sdk1.5.0\bin\java -version
java version "1.5.0"
Java(TM) 2 Runtime Environment, Standard Edition (build 1.5.0-b64)
Java HotSpot(TM) Client VM (build 1.5.0-b64, mixed mode)
```

First Java Program

Use NotePad to enter the following Java program into a file called `Hello.java`:

```
class Hello {
    public static void main(String[] a) {
        System.out.println("Hello world!");
    }
}
```

Then compile this program in a command window with the `javac` command:

```
\j2sdk1.5.0\bin\javac Hello.java
```

To execute the program, use the `java` command:

```
\j2sdk1.5.0\bin\java -cp . Hello
Hello world!
```

Congratulations, you have successfully entered, compiled and executed your first Java program.

SSL (Secure Socket Layer)

Sample programs listed in this chapter have been tested with JDK 1.5.0.

What is SSL?

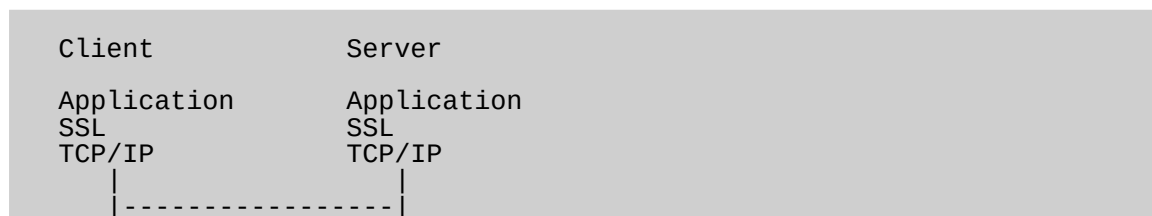
SSL (Secure Socket Layer) is a communication protocol created by Netscape in 1994 to ensure secure transactions between web servers and browsers. A good description about SSL was given by Franck Martin in "SSL Certificates HOWTO" at <http://www.tldp.org/HOWTO/SSL-Certificates-HOWTO>:

- A browser requests a secure page (usually https://).
- The web server sends its public key with its certificate.
- The browser checks that the certificate was issued by a trusted party (usually a trusted root certificate authority), that the certificate is still valid and that the certificate is related to the site contacted.
- The browser then uses the public key, to encrypt a random symmetric encryption key and sends it to the server with the encrypted URL required as well as other encrypted http data.
- The web server decrypts the symmetric encryption key using its private key and uses the symmetric key to decrypt the URL and http data.
- The web server sends back the requested html document and http data encrypted with the symmetric key.
- The browser decrypts the http data and html document using the symmetric key and displays the information.

The latest version of SSL is version 3.0 which was defined in 1996. A new protocol called Transport Layer Security (TLS) has been developed to replace SSL.

SSL Specification Overview

SSL protocol operates between the TCP/IP layer and the application layer in the communication layer model. See the following diagram:



The objective of SSL protocol is to offer to the application the following security properties:

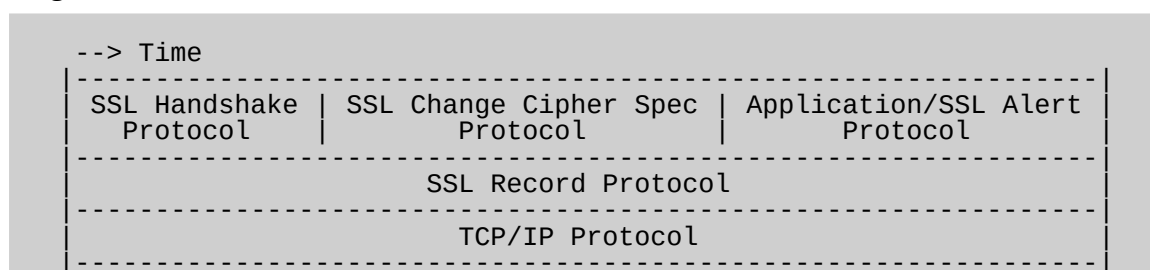
- **Privacy** - Application data can be encrypted with symmetric cryptography technologies.

- **Authenticity** - Remote host can be authenticated with certificate technologies.
- **Integrity** - Application data's integrity can be checked with message digest technologies.

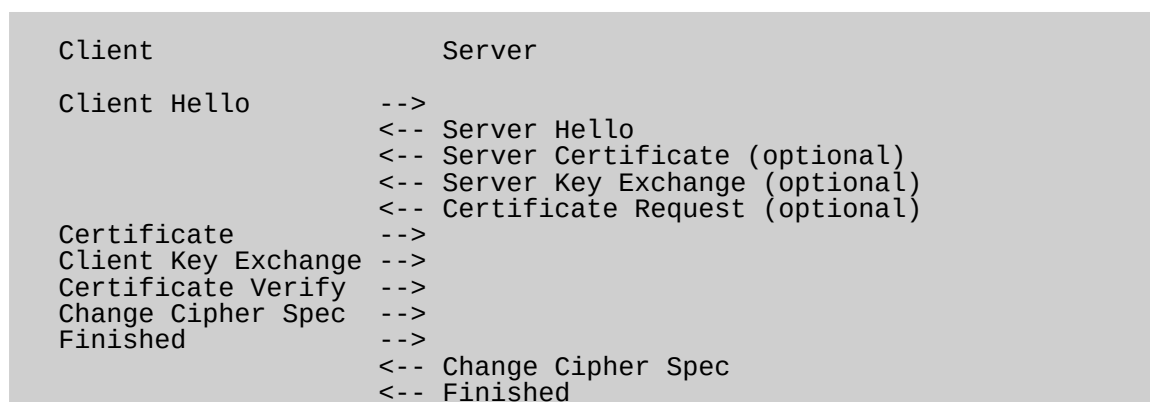
SSL protocol is actually composed of 4 sub-protocols:

- **Record Protocol** - Operates between the TCP/IP layer and application layer to apply fragmentation, compression, encryption, and message digest operations.
- **Handshake Protocol** - Operates on top of the record protocol layer before any real application data transmission to authenticate remote host, exchange encryption settings and initializing the record protocol layer.
- **Change Cipher Spec Protocol** - Operates on top of the record protocol layer to inform remote host to change security settings in the record protocol layer.
- **Alert Protocol** - Operates on top of the record protocol layer to send alerts to the remote host.

SSL's sub-protocols work together with application protocols as shown in the following diagram:



As you can see, the handshake protocol is very important for establishing the SSL. The following diagram shows you what are the messages used in the handshake protocol and in what sequence they are used:



For details of the SSL specifications, see "The SSL Protocol, Version 3.0" at <http://wp.netscape.com/eng/ssl3/draft302.txt>.

How Can SSL Ensure Privacy?

SSL allows the server and the client to gain privacy with the following approach:

- During the handshake process, the server sends its public key to the client.
- The client then selects a secret key, encrypts it with server's public key, and sends it to the server.
- The server decrypts the secret key with its private key. Both ends are now ready to use the secret key.
- When application data is transmitted, both ends will encrypt data with the secret key.

JSSE - Java Implementation of SSL and TLS

JSSE (Java Secure Socket Extension) provides a Java implementation of SSL and TLS protocols through the following major classes and interfaces:

- `javax.net.ssl.SSLServerSocket` - Representing the server end of a secure communication.
- `javax.net.ssl.SSLSocket` - Representing the client end of a secure communication.
- `javax.net.ssl.SSLServerSocketFactory` - Used to create `SSLServerSocket` objects.
- `javax.net.ssl.SSLSocketFactory` - Used to create `SSLSocket` objects.
- `javax.net.ssl.SSLContext` - Representing a secure communication context. Once initialized, it can be used as a factory to create `SSLServerSocketFactory` objects and `SSLSocketFactory` objects.
- `javax.net.ssl.KeyManager` - Responsible for managing the key material used to authenticate the local `SSLSocket`.
- `javax.net.ssl.KeyManagerFactory` - Used to create `KeyManager` objects based on keys from a given `KeyStore` object.

Default SSL Socket Factories - `SslSocketTest.java`

The following sample program shows you how to create default SSL socket factories:

```
/**
 * SslSocketTest.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
import java.security.*;
import javax.net.*;
import javax.net.ssl.*;
public class SslSocketTest {
    public static void main(String[] args) {
        PrintStream out = System.out;
        out.println("\nDefault SSL socket factory:");
        try {
            // Generating the default SSLServerSocketFactory
            SSLServerSocketFactory ssf = (SSLServerSocketFactory)
                SSLServerSocketFactory.getDefault();
            System.out.println("\nSSLServerSocketFactory class: "
```

```

        +ssf.getClass());
String[] dcsList = ssf.getDefaultCipherSuites();
out.println("    Default cipher suites:");
for (int i=0; i<dcsList.length; i++) {
    out.println("        "+dcsList[i]);
}

// Generating SSLServerSocket
SSLServerSocket ss
    = (SSLServerSocket) ssf.createServerSocket();
System.out.println("\nSSLServerSocket class: "
    +ss.getClass());
System.out.println("    String: "+ss.toString());

// Generating the default SSLSocketFactory
SSLSocketFactory sf =
    (SSLSocketFactory) SSLSocketFactory.getDefault();
out.println("\nSSLSocketFactory class: "
    +sf.getClass());
dcsList = sf.getDefaultCipherSuites();
out.println("    Default cipher suites:");
for (int i=0; i<dcsList.length; i++) {
    out.println("        "+dcsList[i]);
}

// Generating SSLSocket
SSLSocket s
    = (SSLSocket) sf.createSocket();
System.out.println("\nSSLSocket class: "+s.getClass());
System.out.println("    String: "+s.toString());
} catch (Exception e) {
    System.err.println(e.toString());
}
}
}
}

```

If you run this program with JDK 1.5.0, you will get:

Default SSL socket factory:

```

SSLServerSocketFactory class:
class com.sun.net.ssl.internal.ssl.SSLServerSocketFactoryImpl
Default cipher suites:
SSL_RSA_WITH_RC4_128_MD5
SSL_RSA_WITH_RC4_128_SHA
TLS_RSA_WITH_AES_128_CBC_SHA
TLS_DHE_RSA_WITH_AES_128_CBC_SHA
TLS_DHE_DSS_WITH_AES_128_CBC_SHA
SSL_RSA_WITH_3DES_EDE_CBC_SHA
SSL_DHE_RSA_WITH_3DES_EDE_CBC_SHA
SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA
SSL_RSA_WITH_DES_CBC_SHA
SSL_DHE_RSA_WITH_DES_CBC_SHA
SSL_DHE_DSS_WITH_DES_CBC_SHA
SSL_RSA_EXPORT_WITH_RC4_40_MD5
SSL_RSA_EXPORT_WITH_DES40_CBC_SHA
SSL_DHE_RSA_EXPORT_WITH_DES40_CBC_SHA
SSL_DHE_DSS_EXPORT_WITH_DES40_CBC_SHA

SSLServerSocket class:
class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl

```



```
String: [SSL: ServerSocket[unbound]]

SSLSocketFactory class:
class com.sun.net.ssl.internal.ssl.SSLSocketFactoryImpl
Default cipher suites:
SSL_RSA_WITH_RC4_128_MD5
SSL_RSA_WITH_RC4_128_SHA
TLS_RSA_WITH_AES_128_CBC_SHA
TLS_DHE_RSA_WITH_AES_128_CBC_SHA
TLS_DHE_DSS_WITH_AES_128_CBC_SHA
SSL_RSA_WITH_3DES_EDE_CBC_SHA
SSL_DHE_RSA_WITH_3DES_EDE_CBC_SHA
SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA
SSL_RSA_WITH_DES_CBC_SHA
SSL_DHE_RSA_WITH_DES_CBC_SHA
SSL_DHE_DSS_WITH_DES_CBC_SHA
SSL_RSA_EXPORT_WITH_RC4_40_MD5
SSL_RSA_EXPORT_WITH_DES40_CBC_SHA
SSL_DHE_RSA_EXPORT_WITH_DES40_CBC_SHA
SSL_DHE_DSS_EXPORT_WITH_DES40_CBC_SHA

SSLSocket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
String: a3bcc1[SSL_NULL_WITH_NULL_NULL: Socket[unconnected]]
```

Note that JSSE does support a number of cipher suites for the SSL record protocol.

SSL Socket Factories with SSLContext - SslContextTest.java

SSL sockets created with the default SSL socket factories will not have any certificates associated to authenticate them. In order to associate certificates with SSL sockets, we need to use the SSLContext class to create SSL socket factories. Here is a sample program, SslContextTest.java:

```
/**
 * SslContextTest.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
import java.security.*;
import javax.net.*;
import javax.net.ssl.*;

public class SslContextTest {
    public static void main(String[] args) {
        PrintStream out = System.out;
        out.println("\nTesting socket factory with SSLContext:");
        try {
            // SSLContext protocols: TLS, SSL, SSLv3
            SSLContext sc = SSLContext.getInstance("SSLv3");
            System.out.println("\nSSLContext class: "+sc.getClass());
            System.out.println("    Protocol: "+sc.getProtocol());
            System.out.println("    Provider: "+sc.getProvider());

            // SSLContext algorithms: SunX509
            KeyManagerFactory kmf
                = KeyManagerFactory.getInstance("SunX509");
            System.out.println("\nKeyManagerFactory class: "
                +kmf.getClass());
            System.out.println("    Algorithm: "+kmf.getAlgorithm());
```

```

        System.out.println("    Provider: "+kmf.getProvider());

        // KeyStore types: JKS
        String ksName = "herong.jks";
        char ksPass[] = "HerongJKS".toCharArray();
        char ctPass[] = "My1stKey".toCharArray();
        KeyStore ks = KeyStore.getInstance("JKS");
        ks.load(new FileInputStream(ksName), ksPass);
        System.out.println("\nKeyStore class: "+ks.getClass());
        System.out.println("    Type: "+ks.getType());
        System.out.println("    Provider: "+ks.getProvider());
        System.out.println("    Size: "+ks.size());

        // Generating KeyManager list
        kmf.init(ks, ctPass);
        KeyManager[] kmList = kmf.getKeyManagers();
        System.out.println("\nKeyManager class: "
            +kmList[0].getClass());
        System.out.println("    # of key manager: " +kmList.length);

        // Generating SSLServerSocketFactory
        sc.init(kmList, null, null);
        SSLServerSocketFactory ssf = sc.getServerSocketFactory();
        System.out.println("\nSSLServerSocketFactory class: "
            +ssf.getClass());

        // Generating SSLServerSocket
        SSLServerSocket ss
            = (SSLServerSocket) ssf.createServerSocket();
        System.out.println("\nSSLServerSocket class: "
            +ss.getClass());
        System.out.println("    String: "+ss.toString());

        // Generating SSLSocketFactory
        sc.init(kmList, null, null);
        SSLSocketFactory sf = sc.getSocketFactory();
        System.out.println("\nSSLSocketFactory class: "
            +sf.getClass());

        // Generating SSLSocket
        SSLSocket s
            = (SSLSocket) sf.createSocket();
        System.out.println("\nSSLSocket class: "+s.getClass());
        System.out.println("    String: "+s.toString());
    } catch (Exception e) {
        System.err.println(e.toString());
    }
}
}

```

Of course, to run this program, you need to have the key store file, herong.jks, ready. It contains a self-signed pair of private and public keys. Read my notes on "JCA - Certificates, 'keytool' and 'keystore'", if you want to use "keytool" to create such a key store file.

If you run this program with JDK 1.5.0, you will get:

Testing socket factory with SSLContext:

SSLContext class: class javax.net.ssl.SSLContext

```
Protocol: SSLv3
Provider: SunJSSE version 1.5

KeyManagerFactory class: class javax.net.ssl.KeyManagerFactory
Algorithm: SunX509
Provider: SunJSSE version 1.5

KeyStore class: class java.security.KeyStore
Type: JKS
Provider: SUN version 1.5
Size: 1

KeyManager class:
class com.sun.net.ssl.internal.ssl.SunX509KeyManagerImpl
# of key manager: 1

SSLServerSocketFactory class:
class com.sun.net.ssl.internal.ssl.SSLServerSocketFactoryImpl

SSLServerSocket class:
class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl
String: [SSL: ServerSocket[unbound]]

SSLSocketFactory class:
class com.sun.net.ssl.internal.ssl.SSLSocketFactoryImpl

SSLSocket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
String: 1891d8f[SSL_NULL_WITH_NULL_NULL: Socket[unconnected]]
```

SSL - Socket Communication

Sample programs listed in this chapter have been tested with JDK 1.5.0.

SSL Server Socket Example - SslReverseEchoer.java

The following sample program, SslReverseEchoer.java, shows you how to create a SSL server socket with a self-signed pair of keys:

```
/**
 * SslReverseEchoer.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
import java.security.*;
import javax.net.ssl.*;

public class SslReverseEchoer {
    public static void main(String[] args) {
        String ksName = "herong.jks";
        char ksPass[] = "HerongJKS".toCharArray();
        char ctPass[] = "My1stKey".toCharArray();
        try {
            KeyStore ks = KeyStore.getInstance("JKS");
            ks.load(new FileInputStream(ksName), ksPass);
            KeyManagerFactory kmf =
                KeyManagerFactory.getInstance("SunX509");
            kmf.init(ks, ctPass);
            SSLContext sc = SSLContext.getInstance("TLS");
            sc.init(kmf.getKeyManagers(), null, null);
            SSLServerSocketFactory ssf = sc.getServerSocketFactory();
            SSLServerSocket s
                = (SSLServerSocket) ssf.createServerSocket(8888);
            printServerSocketInfo(s);
            SSLSocket c = (SSLSocket) s.accept();
            printSocketInfo(c);
            BufferedWriter w = new BufferedWriter(new OutputStreamWriter(
                c.getOutputStream()));
            BufferedReader r = new BufferedReader(new InputStreamReader(
                c.getInputStream()));
            String m = "Welcome to SSL Reverse Echo Server."+
                " Please type in some words.";
            w.write(m,0,m.length());
            w.newLine();
            w.flush();
            while ((m=r.readLine())!= null) {
                if (m.equals(".")) break;
                char[] a = m.toCharArray();
                int n = a.length;
                for (int i=0; i<n/2; i++) {
                    char t = a[i];
                    a[i] = a[n-1-i];
                    a[n-i-1] = t;
                }
                w.write(a,0,n);
                w.newLine();
                w.flush();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        }
        w.close();
        r.close();
        c.close();
        s.close();
    } catch (Exception e) {
        System.err.println(e.toString());
    }
}

private static void printSocketInfo(SSLSocket s) {
    System.out.println("Socket class: "+s.getClass());
    System.out.println("    Remote address = "
        +s.getInetAddress().toString());
    System.out.println("    Remote port = "+s.getPort());
    System.out.println("    Local socket address = "
        +s.getLocalSocketAddress().toString());
    System.out.println("    Local address = "
        +s.getLocalAddress().toString());
    System.out.println("    Local port = "+s.getLocalPort());
    System.out.println("    Need client authentication = "
        +s.getNeedClientAuth());
    SSLSession ss = s.getSession();
    System.out.println("    Cipher suite = "+ss.getCipherSuite());
    System.out.println("    Protocol = "+ss.getProtocol());
}

private static void printServerSocketInfo(SSLServerSocket s) {
    System.out.println("Server socket class: "+s.getClass());
    System.out.println("    Socker address = "
        +s.getInetAddress().toString());
    System.out.println("    Socker port = "
        +s.getLocalPort());
    System.out.println("    Need client authentication = "
        +s.getNeedClientAuth());
    System.out.println("    Want client authentication = "
        +s.getWantClientAuth());
    System.out.println("    Use client mode = "
        +s.getUseClientMode());
}
}

```

Of course, to run this program, you need to have the key store file, herong.jks, ready. It contains a self-signed pair of private and public keys. Read my notes on "JCA - Certificates, 'keytool' and 'keystore'", if you want to use "keytool" to create such a key store file.

If you run this program with JDK 1.5.0, you will get:

```

Server socket class:
class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl
Socker address = 0.0.0.0/0.0.0.0
Socker port = 8888
Need client authentication = false
Want client authentication = false
Use client mode = false

```

Now my SSL socket server is ready and listening for new connections.

SSL Client Socket Example - SslSocketClient.java

Since my SSL socket server does not require client authentication, we can create a SSL socket client with the default SSL socket factory. Here is my sample program, SslSocketClient.java, which can be used to communicate with SslReverseEchoer.java:

```
/**
 * SslSocketClient.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
import javax.net.ssl.*;

public class SslSocketClient {
    public static void main(String[] args) {
        BufferedReader in = new BufferedReader(
            new InputStreamReader(System.in));
        PrintStream out = System.out;
        SSLSocketFactory f =
            (SSLSocketFactory) SSLSocketFactory.getDefault();
        try {
            SSLSocket c =
                (SSLSocket) f.createSocket("localhost", 8888);
            printSocketInfo(c);
            c.startHandshake();
            BufferedWriter w = new BufferedWriter(
                new OutputStreamWriter(c.getOutputStream()));
            BufferedReader r = new BufferedReader(
                new InputStreamReader(c.getInputStream()));
            String m = null;
            while ((m=r.readLine())!= null) {
                out.println(m);
                m = in.readLine();
                w.write(m,0,m.length());
                w.newLine();
                w.flush();
            }
            w.close();
            r.close();
            c.close();
        } catch (IOException e) {
            System.err.println(e.toString());
        }
    }

    private static void printSocketInfo(SSLSocket s) {
        System.out.println("Socket class: "+s.getClass());
        System.out.println("  Remote address = "
            +s.getInetAddress().toString());
        System.out.println("  Remote port = "+s.getPort());
        System.out.println("  Local socket address = "
            +s.getLocalSocketAddress().toString());
        System.out.println("  Local address = "
            +s.getLocalAddress().toString());
        System.out.println("  Local port = "+s.getLocalPort());
        System.out.println("  Need client authentication = "
            +s.getNeedClientAuth());
        SSLSession ss = s.getSession();
        System.out.println("  Cipher suite = "+ss.getCipherSuite());
        System.out.println("  Protocol = "+ss.getProtocol());
    }
}
```

Now run this program in a separate window, you will get:

```
Socket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
  Remote address = localhost/127.0.0.1
  Remote port = 8888
  Local socket address = /127.0.0.1:2183
  Local address = /127.0.0.1
  Local port = 2183
  Need client authentication = false
  Cipher suite = SSL_NULL_WITH_NULL_NULL
  Protocol = NONE
javax.net.ssl.SSLException: Connection has been shutdown:
  javax.net.ssl.SSLHandshakeException:
    sun.security.validator.ValidatorException:
      PKIX path building failed:
        sun.security.provider.certpath.SunCertPathBuilderException:
          unable to find valid certification path to requested target
```

Obviously, there is a problem. The TCP/IP layer connection was ok, but the SSL handshake process failed. The error message says that server certificate received has no valid path from any existing certificate authority.

One way to resolve this problem is to pre-install the server's public key and define it as a trusted:

- On the server side, export my public key out as a certificate.
- On the client side, import the server's public key into a key store file.
- Run the SSL client program with the key store file as trusted. This can be done by using "-Djavax.net.ssl.trustStore=myKeyStore.jks" as java option.

Here is what I did on the server side:

```
>\jdk\bin\keytool -export -keystore herong.jks -alias my_home
-file my_home.crt

Enter keystore password: HerongJKS
Certificate stored in file <my_home.crt>

>"send my_home.crt to the client machine..."
```

Here is what I did on the client side:

```
>"receive my_home.crt from the server machine..."

>\jdk\bin\keytool -import -keystore public.jks -alias herong_home
-file my_home.crt

Enter keystore password: PublicJKS
Owner: CN=Herong Yang, OU=My unit, O=My home, L=My city, ST=My sta...
Issuer: CN=Herong Yang, OU=My unit, O=My home, L=My city, ST=My st...
Serial number: 42266fba
Valid from: Sat Jan 01 21:00:26 EST 2005 until: Tue May 31 22:00:2...
Certificate fingerprints:
  MD5: 55:2C:5B:49:47:CB:61:40:FD:1A:3C:B7:16:6C:8E:7E
  SHA1: EC:B6:77:A1:21:9F:7E:AE:AF:B0:7C:AC:C5:B7:24:B7:45:...
```

```
Trust this certificate? [no]: y
Certificate was added to keystore
```

Now run SslReverseEchoer.java again. Then run SslSocketClient with public.jks:

```
>\jdk\bin\java -cp . "-Djavax.net.ssl.trustStore=public.jks"
SslSocketClient

Socket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
Remote address = localhost/127.0.0.1
Remote port = 8888
Local socket address = /127.0.0.1:2187
Local address = /127.0.0.1
Local port = 2187
Need client authentication = false
Cipher suite = TLS_DHE_DSS_WITH_AES_128_CBC_SHA
Protocol = TLSv1
Welcome to SSL Reverse Echo Server. Please type in some words.
Hello world!
!dlrow olleH
It works!
!skrow tI
.
```

Congratulations! We have successfully used JSSE to create a SSL socket communication.

Note that:

- Even I specified SSL when constructing the SSLContext object, the final protocol used in the communication is TLSv1. I don't know why.
- Client authentication is not used.

JSSE Debug Option

If you want to know what is really going on at the SSL layer, you could use the JSSE Debug options, "-Djavax.net.debug=options". Here is how I used it on the client side:

```
>\jdk\bin\java -cp . "-Djavax.net.ssl.trustStore=public.jks"
"-Djavax.net.debug=help" SslSocketClient

all          turn on all debugging
ssl          turn on ssl debugging

The following can be used with ssl:
    record    enable per-record tracing
    handshake print each handshake message
.....

(Run SslReverseEchoer.java in another window)

>\jdk\bin\java -cp . "-Djavax.net.ssl.trustStore=public.jks"
"-Djavax.net.debug=ssl:record" SslSocketClient

setting up default SSLSocketFactory
.....
init truststore
adding as trusted cert:
```



```

Subject: CN=Herong Yang, OU=My unit, O=My home, L=My ci
Issuer: CN=Herong Yang, OU=My unit, O=My home, L=My ci
Algorithm: DSA; Serial number: 0x42266fba
.....
init context
trigger seeding of SecureRandom
.....
%% No cached client session
*** ClientHello, TLSv1
RandomCookie: GMT: 1111734670 bytes = { 64, 255, 55, 15,
Session ID: {}
Cipher Suites: [SSL_RSA_WITH_RC4_128_MD5, SSL_RSA_WITH_RC
Compression Methods: { 0 }
***
main, WRITE: TLSv1 Handshake, length = 73
main, WRITE: SSLv2 client hello message, length = 98
main, READ: TLSv1 Handshake, length = 1187
*** ServerHello, TLSv1
RandomCookie: GMT: 1111734670 bytes = { 120, 194, 143, 2
Session ID: {66, 68, 186, 142, 195, 126, 97, 92, 127, 59
Cipher Suite: TLS_DHE_DSS_WITH_AES_128_CBC_SHA
Compression Method: 0
***
%% Created: [Session-1, TLS_DHE_DSS_WITH_AES_128_CBC_SHA
** TLS_DHE_DSS_WITH_AES_128_CBC_SHA
*** Certificate chain
chain [0] = [
[
Version: V1
Subject: CN=Herong Yang, OU=My unit, O=My home, L=My ci
Signature Algorithm: SHA1withDSA, OID = 1.2.840.10040.4
.....
]
.....
]
***
Found trusted certificate:
[
[
Version: V1
Subject: CN=Herong Yang, OU=My unit, O=My home, L=My ci
Signature Algorithm: SHA1withDSA, OID = 1.2.840.10040.4
.....
]
.....
]
*** Diffie-Hellman ServerKeyExchange
DH Modulus: { 244, 136, 253, 88, 78, 73, 219, 205, 32, 1
DH Base: { 2 }
Server DH Public Key: { 100, 97, 85, 119, 180, 34, 56, 2
Anonymous
*** ServerHelloDone
*** ClientDiffieHellmanPublic
DH Public key: { 40, 239, 235, 116, 118, 207, 63, 85, 24
main, WRITE: TLSv1 Handshake, length = 134
SESSION KEYGEN:
PreMaster Secret:
0000: 69 65 81 69 31 1E 9E 4D    34 9C 49 54 6E 0A A5 FB
0010: B9 C0 21 F7 4F 84 D5 75    69 86 F2 10 B5 F6 8D 20
.....
CONNECTION KEYGEN:
Client Nonce:

```

```

0000: 42 44 BA 8E 40 FF 37 0F 7B 46 96 D1 E5 6A 99 FA
0010: A5 7E 85 2F 57 A9 E4 17 8A 2C 74 54 60 6A B4 10
Server Nonce:
0000: 42 44 BA 8E 78 C2 8F E0 60 82 32 C9 C4 EE 68 F3
0010: 6C B7 6A AB B0 F9 E8 DD 66 3F A1 3C 5E 96 97 32
Master Secret:
0000: 49 B5 9D BC 38 95 E6 34 EE 10 89 7E 3E 53 77 F9
0010: BD 93 83 F7 FC D1 F1 6F B1 95 83 B5 97 63 1F 2B
0020: 0D A6 05 50 DA B7 21 28 EB B9 D5 6B A4 2D F0 02
Client MAC write Secret:
0000: 34 57 AE 02 2B CA 00 04 0D D5 49 27 37 DB E0 76
0010: 92 38 DD 1E
Server MAC write Secret:
0000: 2D 58 DD 43 59 4C CB AC F5 C7 B2 ED 5F B1 16 03
0010: E7 A0 57 D2
Client write key:
0000: 65 25 6A 56 FB AE B2 37 B0 BD FE 82 BE 45 F4 5F
Server write key:
0000: 44 23 24 A7 BE CD FB 3C CF D6 50 EC 43 C1 C2 E1
Client write IV:
0000: 73 F3 98 2D F3 9A 6D 72 2F 59 4E 58 80 3D 17 F1
Server write IV:
0000: 68 10 0A C0 FE 36 88 CD 92 E3 14 DC 87 9C 51 93
main, WRITE: TLSv1 Change Cipher Spec, length = 1
*** Finished
verify_data: { 148, 191, 195, 227, 118, 168, 181, 3, 60,
***
main, WRITE: TLSv1 Handshake, length = 48
main, READ: TLSv1 Change Cipher Spec, length = 1
main, READ: TLSv1 Handshake, length = 48
*** Finished
verify_data: { 33, 227, 154, 8, 58, 178, 246, 217, 219,
***
%% Cached client session: [Session-1, TLS_DHE_DSS_WITH_AE
Cipher suite = TLS_DHE_DSS_WITH_AES_128_CBC_SHA
Protocol = TLSv1
%% Client cached [Session-1, TLS_DHE_DSS_WITH_AES_128_CBC
%% Try resuming [Session-1, TLS_DHE_DSS_WITH_AES_128_CBC_
*** ClientHello, TLSv1
RandomCookie: GMT: 1111734671 bytes = { 180, 13, 13, 192
Session ID: {66, 68, 186, 142, 195, 126, 97, 92, 127, 59
Cipher Suites: [SSL_RSA_WITH_RC4_128_MD5, SSL_RSA_WITH_RC
Compression Methods: { 0 }
***
main, WRITE: TLSv1 Handshake, length = 128
main, READ: TLSv1 Application Data, length = 96
Welcome to SSL Reverse Echo Server. Please type in some w
Hello World!
main, WRITE: TLSv1 Application Data, length = 48
main, READ: TLSv1 Handshake, length = 96
*** ServerHello, TLSv1
RandomCookie: GMT: 1111734671 bytes = { 172, 192, 94, 22
Session ID: {66, 68, 186, 142, 195, 126, 97, 92, 127, 59
Cipher Suite: TLS_DHE_DSS_WITH_AES_128_CBC_SHA
Compression Method: 0
***
CONNECTION KEYGEN:
Client Nonce:
0000: 42 44 BA 8F B4 0D 0D C0 2A 41 EC F5 A7 FA 79 34
0010: CD 5E 62 E2 04 13 68 84 D0 62 98 1E C0 1F 15 AA
Server Nonce:
0000: 42 44 BA 8F AC C0 5E E0 74 61 C1 34 E5 14 88 2F

```

```

0010: 6D 16 0F E2 EE 27 A5 D1 FA 52 BB 8B A5 21 A7 4A
Master Secret:
0000: 49 B5 9D BC 38 95 E6 34 EE 10 89 7E 3E 53 77 F9
0010: BD 93 83 F7 FC D1 F1 6F B1 95 83 B5 97 63 1F 2B
0020: 0D A6 05 50 DA B7 21 28 EB B9 D5 6B A4 2D F0 02
Client MAC write Secret:
0000: 7F F0 DC C8 FE E4 9D 57 6E 5D E1 C4 D3 D5 9A 3E
0010: 9A 30 48 90
Server MAC write Secret:
0000: E6 F6 DC A1 87 D0 F2 93 0B E8 7C AE 9D BC 98 42
0010: 6A 22 0D 12
Client write key:
0000: AA 64 2F E3 54 E8 2D 86 61 39 F9 B8 C3 C9 73 79
Server write key:
0000: D8 54 5C 02 56 DE B8 1E ED 28 AC FD 5A 01 8A BD
Client write IV:
0000: C9 EE F0 EB 24 41 1B 06 D7 D3 1A 7B DC CD 7C 59
Server write IV:
0000: 16 35 61 8A 34 F2 D4 76 6A 9A 13 FE 17 3E 74 41
%% Server resumed [Session-1, TLS_DHE_DSS_WITH_AES_128_CB
main, READ: TLSv1 Change Cipher Spec, length = 32
main, READ: TLSv1 Handshake, length = 48
*** Finished
verify_data: { 56, 41, 153, 87, 50, 152, 70, 168, 237, 1
***
main, WRITE: TLSv1 Change Cipher Spec, length = 32
*** Finished
verify_data: { 254, 26, 149, 188, 239, 40, 18, 232, 72,
***
main, WRITE: TLSv1 Handshake, length = 48
main, READ: TLSv1 Application Data, length = 48
!dlrow olleH
main, WRITE: TLSv1 Application Data, length = 32
main, READ: TLSv1 Alert, length = 32
main, RECV TLSv1 ALERT: warning, close_notify
main, called closeInternal(false)
main, SEND TLSv1 ALERT: warning, description = close_not
main, WRITE: TLSv1 Alert, length = 32
main, called close()
main, called closeInternal(true)
main, called close()
main, called closeInternal(true)
main, called close()
main, called closeInternal(true)

```

Exercise : Revise both SslReverseEchoer.java and SslSocketClient.java so that client authentication is required.

SSL - HTTPS

Sample programs listed in this chapter have been tested with JDK 1.5.0.

What is HTTPS?

HTTPS is HTTP over SSL. If you know how to program SSL at socket level, you should have no problem to program HTTPS.

Note that Web browsers assume HTTPS servers are running at port 443 by default.

My First HTTPS Server - HttpsHello.java

To test how HTTPS works, I wrote a simple HTTPS server:

```
/**
 * HttpsHello.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
import javax.net.ssl.*;

public class HttpsHello {
    public static void main(String[] args) {
        String ksName = "herong.jks";
        char ksPass[] = "HerongJKS".toCharArray();
        char ctPass[] = "My1stKey".toCharArray();
        try {
            KeyStore ks = KeyStore.getInstance("JKS");
            ks.load(new FileInputStream(ksName), ksPass);
            KeyManagerFactory kmf =
                KeyManagerFactory.getInstance("SunX509");
            kmf.init(ks, ctPass);
            SSLContext sc = SSLContext.getInstance("TLS");
            sc.init(kmf.getKeyManagers(), null, null);
            SSLServerSocketFactory ssf = sc.getServerSocketFactory();
            SSLServerSocket s
                = (SSLServerSocket) ssf.createServerSocket(8888);
            System.out.println("Server started:");
            printServerSocketInfo(s);
            // Listening to the port
            SSLSocket c = (SSLSocket) s.accept();
            printSocketInfo(c);
            BufferedWriter w = new BufferedWriter(
                new OutputStreamWriter(c.getOutputStream()));
            BufferedReader r = new BufferedReader(
                new InputStreamReader(c.getInputStream()));
            String m = r.readLine();
            w.write("HTTP/1.0 200 OK");
            w.newLine();
            w.write("Content-Type: text/html");
            w.newLine();
            w.newLine();
            w.write("<html><body>Hello world!</body></html>");
            w.newLine();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        w.flush();
        w.close();
        r.close();
        c.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
private static void printSocketInfo(SSLSocket s) {
    System.out.println("Socket class: "+s.getClass());
    System.out.println("    Remote address = "
        +s.getInetAddress().toString());
    System.out.println("    Remote port = "+s.getPort());
    System.out.println("    Local socket address = "
        +s.getLocalSocketAddress().toString());
    System.out.println("    Local address = "
        +s.getLocalAddress().toString());
    System.out.println("    Local port = "+s.getLocalPort());
    System.out.println("    Need client authentication = "
        +s.getNeedClientAuth());
    SSLSession ss = s.getSession();
    System.out.println("    Cipher suite = "+ss.getCipherSuite());
    System.out.println("    Protocol = "+ss.getProtocol());
}
private static void printServerSocketInfo(SSLServerSocket s) {
    System.out.println("Server socket class: "+s.getClass());
    System.out.println("    Socker address = "
        +s.getInetAddress().toString());
    System.out.println("    Socker port = "
        +s.getLocalPort());
    System.out.println("    Need client authentication = "
        +s.getNeedClientAuth());
    System.out.println("    Want client authentication = "
        +s.getWantClientAuth());
    System.out.println("    Use client mode = "
        +s.getUseClientMode());
}
}
}

```

If you run this program in a command window, you will get:

```

Server started:
Server socket class:
class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl
Socker address = 0.0.0.0/0.0.0.0
Socker port = 8888
Need client authentication = false
Want client authentication = false
Use client mode = false

```

To test HttpsHello.java, I wrote a simple HTTPS client program, HttpsClient.java:

```

/**
 * HttpsClient.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
import javax.net.ssl.*;
public class HttpsClient {
    public static void main(String[] args) {

```

```

    PrintStream out = System.out;

    // Getting the default SSL socket factory
    SSLSocketFactory f =
        (SSLSocketFactory) SSLSocketFactory.getDefault();
    out.println("The default SSL socket factory class: "
        +f.getClass());
    try {
        // Getting the default SSL socket factory
        SSLSocket c =
            (SSLSocket) f.createSocket("localhost", 8888);
        printSocketInfo(c);
        c.startHandshake();
        BufferedWriter w = new BufferedWriter(new OutputStreamWriter(
            c.getOutputStream()));
        BufferedReader r = new BufferedReader(new InputStreamReader(
            c.getInputStream()));
        w.write("GET / HTTP/1.0");
        w.newLine();
        w.newLine(); // end of HTTP request
        w.flush();
        String m = null;
        while ((m=r.readLine())!= null) {
            out.println(m);
        }
        w.close();
        r.close();
        c.close();
    } catch (IOException e) {
        System.err.println(e.toString());
    }
}

private static void printSocketInfo(SSLSocket s) {
    System.out.println("Socket class: "+s.getClass());
    System.out.println("    Remote address = "
        +s.getInetAddress().toString());
    System.out.println("    Remote port = "+s.getPort());
    System.out.println("    Local socket address = "
        +s.getLocalSocketAddress().toString());
    System.out.println("    Local address = "
        +s.getLocalAddress().toString());
    System.out.println("    Local port = "+s.getLocalPort());
    System.out.println("    Need client authentication = "
        +s.getNeedClientAuth());
    SSLSession ss = s.getSession();
    System.out.println("    Cipher suite = "+ss.getCipherSuite());
    System.out.println("    Protocol = "+ss.getProtocol());
}
}

```

Of course, we have to run `HttpsClient.java` with the server's certificate in a trusted key store file:

```

>\jdk\bin\java -cp . "-Djavax.net.ssl.trustStore=public.jks"
HttpsClient

The default SSL socket factory class:
    class com.sun.net.ssl.internal.ssl.SSLSocketFactoryImpl
Socket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
    Remote address = localhost/127.0.0.1
    Remote port = 8888

```

```
Local socket address = /127.0.0.1:2408
Local address = /127.0.0.1
Local port = 2408
Need client authentication = false
Cipher suite = TLS_DHE_DSS_WITH_AES_128_CBC_SHA
Protocol = TLSv1
HTTP/1.0 200 OK
Content-Type: text/html

<html><body>Hello world!</body></html>
```

The result matches the expectation perfectly.

Connecting HttpsHello.java with Internet Explorer

Now let's use Internet Explorer (IE) to connect to HttpsHello.java. First run HttpsHello.java again.

```
>\jdk\bin\java -cp . HttpsHello

Server started:
Server socket class:
  class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl
  Socker address = 0.0.0.0/0.0.0.0
  Socker port = 8888
  Need client authentication = false
  Want client authentication = false
  Use client mode = false
```

Then run IE with https://localhost:8080, the browser will give a security alert saying that you are about to enter a secure web page. Click "OK" on the alert dialog box, you will see that the HttpsHello program prints more message, and ends with an exception:

```
java.net.SocketException: Connection closed by remote host
    at com.sun.net.ssl.internal.ssl.SSLSocketImpl.checkWrite(U...
    at com.sun.net.ssl.internal.ssl.AppOutputStream.write(Unkn...
    at sun.nio.cs.StreamEncoder$CharsetSE.writeBytes(Unknown S...
    at sun.nio.cs.StreamEncoder$CharsetSE.implFlushBuffer(Unkn...
    at sun.nio.cs.StreamEncoder$CharsetSE.implFlush(Unknown So...
    at sun.nio.cs.StreamEncoder.flush(Unknown Source)
    at java.io.OutputStreamWriter.flush(Unknown Source)
    at java.io.BufferedWriter.flush(Unknown Source)
    at HttpsHello.main(HttpsHello.java:42)
```

At the same time, IE displays another security alert saying there is a problem with the security certificate. The issuer is not a trusted company and the certificate name does not match the server name. If you click "Yes" to continue to view the page, you will get a page-not-available error message. Because HttpsHello has already ended.

So why my HttpsHello is getting this "Connection closed by remote host" exception? The code logic in HttpsHello seems to be correct. Accoring the output message, the accept() is returning a good SSLSocket. The code reads the first line from the HTTP request, then writes back a HTTP response. This is exactly how we would code for non-SSL HTTP communication.

After testing `HttpsHello.java` with a network debugger for a number of times, I finally found why `HttpsHello.java` is not working with IE. The trouble is caused by the self-signed certificate.

When IE reaches a HTTPS server and gets a certificate that failed to pass the validation rules, it will abandon the connection, and display a security alert to the user. If the user wants to ignore the validation error and continue with the HTTP request, IE will make another connection to the server.

As you can see, `HttpsHello.java` is not designed to handle this abandoned connection. `HttpsHello.java` is not aware the fact that the client has already closed the connection, and still tries to write the HTTP response back to the connection. Of course, it will get an exception.

One way to resolve this problem is to check the data returned from `r.readLine()`. If there is no data, we know that the connection is not a good HTTP connection. In this case we could ignore this connection, and go to listen to the next connection.

Another way to resolve this problem is to help IE to pass the validation of the server's certificate. Apparently, IE validates certificate with 3 rules:

1. The certificate is issued by a trusted authority.
2. The certificate is not expired.
3. The certificate name matches the HTTPS server name.

Make IE happy, I generated another key for "localhost", and restarted `HttpsHello.java`:

```
>\jdk\bin\keytool -genkey -alias my_host -keystore herong.jks
Enter keystore password: HerongJKS
What is your first and last name?
  [Unknown]: localhost
What is the name of your organizational unit?
  [Unknown]: My Unit
What is the name of your organization?
  [Unknown]: My Home
What is the name of your City or Locality?
  [Unknown]: My City
What is the name of your State or Province?
  [Unknown]: My State
What is the two-letter country code for this unit?
  [Unknown]: US
Is <CN=localhost, OU=My Unit, O=My Home, L=My City, ST=My State,
C=US> correct?
  [no]: yes
Enter key password for <my_host>
  (RETURN if same as keystore password): My1stKey
>\jdk\bin\java -cp . HttpsHello
```



```
Server started:
Server socket class:
  class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl
  Socker address = 0.0.0.0/0.0.0.0
  Socker port = 8888
  Need client authentication = false
  Want client authentication = false
  Use client mode = false
```

Then run IE with `https://localhost:8080`, the browser will give a security alert saying that you are about to enter a secure web page. Click "OK" on the alert dialog box, IE will give new another alert saying that the server's certificate is not issued by a trusted authority.

Don't click OK to ignore this alert. Click "View Certificate" instead. On the certificate dialog box, click "Install Certificate", and follow the instruction to finish installing my "localhost" certificate into IE. Remember to click "Yes" to add it to the Root Store.

Now restart `HttpsHello.java` and run IE with `https://localhost:8888`. If you see "Hello world!" on the IE window after the first security alert, you know that my "localhost" certificate is working correctly.

To keep IE clean, we should remove "localhost" certificate. Run IE, and go to "Tools", "Internet Options", "Content", "Certificates", and "Trusted Root Certificate Authorities". You should be able to find "localhost" in the certificate list. Highlight it and remove it.

A Better HTTPS Server - `HttpsEchoer.java`

Based on the experience from previous examples, I wrote a better HTTPS server, `HttpsEchoer.java`. It will handle the abandoned connections correctly. Of course, it will echo the HTTP request back to the client.

```
/**
 * HttpsEchoer.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.security.*;
import javax.net.ssl.*;
public class HttpsEchoer {
    public static void main(String[] args) {
        String ksName = "herong.jks";
        char ksPass[] = "HerongJKS".toCharArray();
        char ctPass[] = "My1stKey".toCharArray();
        try {
            KeyStore ks = KeyStore.getInstance("JKS");
            ks.load(new FileInputStream(ksName), ksPass);
            KeyManagerFactory kmf =
                KeyManagerFactory.getInstance("SunX509");
            kmf.init(ks, ctPass);
            SSLContext sc = SSLContext.getInstance("TLS");
            sc.init(kmf.getKeyManagers(), null, null);
            SSLServerSocketFactory ssf = sc.getServerSocketFactory();
            SSLServerSocket s
                = (SSLServerSocket) ssf.createServerSocket(8888);
            System.out.println("Server started:");
```

```

        printServerSocketInfo(s);
        // Listening to the port
        int count = 0;
        while (true) {
            SSLSocket c = (SSLSocket) s.accept();
            // Someone is calling this server
            count++;
            System.out.println("Connection #: "+count);
            printSocketInfo(c);
            BufferedWriter w = new BufferedWriter(
                new OutputStreamWriter(c.getOutputStream()));
            BufferedReader r = new BufferedReader(
                new InputStreamReader(c.getInputStream()));
            String m = r.readLine();
            if (m!=null) {
                // We have a real data connection
                w.write("HTTP/1.0 200 OK");
                w.newLine();
                w.write("Content-Type: text/html");
                w.newLine();
                w.newLine();
                w.write("<html><body><pre>");
                w.newLine();
                w.write("Connection #: "+count);
                w.newLine();
                w.newLine();
                w.write(m);
                w.newLine();
                while ((m=r.readLine())!= null) {
                    if (m.length()==0) break; // End of a GET call
                    w.write(m);
                    w.newLine();
                }
                w.write("</pre></body></html>");
                w.newLine();
                w.flush();
            }
            w.close();
            r.close();
            c.close();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private static void printSocketInfo(SSLSocket s) {
    System.out.println("Server socket class: "+s.getClass());
    System.out.println("    Remote address = "
        +s.getInetAddress().toString());
    System.out.println("    Remote port = "
        +s.getPort());
    System.out.println("    Local socket address = "
        +s.getLocalSocketAddress().toString());
    System.out.println("    Local address = "
        +s.getLocalAddress().toString());
    System.out.println("    Local port = "
        +s.getLocalPort());
}

private static void printServerSocketInfo(SSLServerSocket s) {
    System.out.println("Server socket class: "+s.getClass());
    System.out.println("    Socker address = "
        +s.getInetAddress().toString());
}

```

```

        System.out.println("    Socker port = "
            +s.getLocalPort());
        System.out.println("    Need client authentication = "
            +s.getNeedClientAuth());
        System.out.println("    Want client authentication = "
            +s.getWantClientAuth());
        System.out.println("    Use client mode = "
            +s.getUseClientMode());
    }
}

```

Now run HttpsEchoer in a command window, and run IE with `https://localhost:8888/hello.html`. IE will still display two security alerts as before. But HttpsEchoer will handle the IE's requests correctly. You will get the following in the IE window:

```

Connection #: 2

GET /hello.html HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, appli...
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; .NE...
Host: localhost:8888
Connection: Keep-Alive

```

Note that connection count is 2 not 1. This confirms my notes about IE's behavior in the previous section. Connection 1 was closed by IE when it failed validate the certificate and displayed the second security alert.

If you look at the HttpsEchoer window, you will see 2 connections recorded:

```

Server started:
Server socket class:
    class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl
    Socker address = 0.0.0.0/0.0.0.0
    Socker port = 8888
    Need client authentication = false
    Want client authentication = false
    Use client mode = false
Connection #: 1
Server socket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
    Remote address = /127.0.0.1
    Remote port = 2025
    Local socket address = /127.0.0.1:8888
    Local address = /127.0.0.1
    Local port = 8888
    Need client authentication = false
    Cipher suite = SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA
    Protocol = SSLv3
Connection #: 2
Server socket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
    Remote address = /127.0.0.1
    Remote port = 2026
    Local socket address = /127.0.0.1:8888
    Local address = /127.0.0.1
    Local port = 8888
    Need client authentication = false

```

```
Cipher suite = SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA  
Protocol = SSLv3
```

Conclusion

- HTTPS is HTTP running on top of SSL. You can write HTTPS server and client programs like any other SSL socket programs.
- When IE connects to a HTTPS server, and gets an invalid certificate, it will abandon the connection. HTTPS server program needs to be smart to handle this.

SSL - Client Authentication

Sample programs listed in this chapter have been tested with JDK 1.5.0.

What is SSL Client Authentication

As I mentioned earlier in this book, the SSL handshake process is following the following sequence:

Client	Server
Client Hello	-->
	<-- Server Hello
	<-- Server Certificate (optional)
	<-- Server Key Exchange (optional)
	<-- Certificate Request (optional)
Certificate	-->
Client Key Exchange	-->
Certificate Verify	-->
Change Cipher Spec	-->
Finished	-->
	<-- Change Cipher Spec
	<-- Finished

Two identity authentications may happen during this process:

1. Server Authentication - Server program must have access to the server's "full" certificate, which contains the server's identity, public key and private key. During handshake process, the server will extract the server's "public" certificate and send it to the client. On the client side, the client program must be able to authenticate the server's identity by validating the server's "public" certificate. One way to identify the server's identity is to install server's "public" certificate to client program as a trusted certificate. Server authentication seems to be always required.

2. Client Authentication - This is the 100% mirror process of server authentication. However, client authentication is optional for SSL communication.

Preparing Certificates for Client Authentication

JDK "keytool" seems to be a good tool to generate and manage certificates. So I will "keytool" to generate and manage "full" and "public" certificates for both server and client sides. Let's see what I did on the server side first:

```
>keytool -genkey -alias server_full -keypass ServerKey
      -keystore server.jks -storepass ServerJKS

What is your first and last name?
[Unknown]: my.server.com
What is the name of your organizational unit?
[Unknown]: My Unit
```

```
What is the name of your organization?
[Unknown]: My Home
What is the name of your City or Locality?
[Unknown]: My City
What is the name of your State or Province?
[Unknown]: My State
What is the two-letter country code for this unit?
[Unknown]: US
Is CN=my.server.com, OU=My Unit, O=My Home, L=My City, ST=My State...
[no]: yes

>keytool -export -alias server_full -file server_pub.crt
      -keystore server.jks -storepass ServerJKS

Certificate stored in file <server_pub.crt>

>"send server_pub.crt to the client side..."
```

At this moment, the server's "full" certificate is ready and stored in server.jks. The server's "public" certificate is also ready and stored in server_pub.crt. Next, let's see what I did on the client side:

```
>keytool -genkey -alias client_full -keypass ClientKey
      -keystore client.jks -storepass ClientJKS

What is your first and last name?
[Unknown]: my.client.com
What is the name of your organizational unit?
[Unknown]: My Unit
What is the name of your organization?
[Unknown]: My Home
What is the name of your City or Locality?
[Unknown]: My City
What is the name of your State or Province?
[Unknown]: My State
What is the two-letter country code for this unit?
[Unknown]: US
Is CN=my.client.com, OU=My Unit, O=My Home, L=My City, ST=My State...
[no]: yes

>keytool -export -alias client_full -file client_pub.crt
      -keystore client.jks -storepass ClientJKS

Certificate stored in file <client_pub.crt>

>"send client_pub.crt to the server side..."

>"receive server_pub.crt from the server side..."

>keytool -import -alias server_pub -file server_pub.crt
      -keystore client.jks -storepass ClientJKS

Owner: CN=my.server.com, OU=My Unit, O=My Home, L=My City, ST=My S...
Issuer: CN=my.server.com, OU=My Unit, O=My Home, L=My City, ST=My ...
.....
Trust this certificate? [no]: yes
Certificate was added to keystore

>keytool -list -keystore client.jks -storepass ClientJKS
```

```
Keystore type: jks
Keystore provider: SUN

Your keystore contains 2 entries

client_full, Mar 29, 2005, keyEntry,
Certificate fingerprint (MD5): 53:5F:62:00:4A:5F:0E:DC:1A:8F:4B:8E...
server_pub, Mar 29, 2005, trustedCertEntry,
Certificate fingerprint (MD5): 34:71:CD:2F:E8:D9:32:57:34:61:46:4C...
```

At this moment, the client's "full" certificate is ready and stored in client.jks. The client's "public" certificate is also ready and stored in client_pub.crt. The server's "public" certificate is also added client.jks as a trusted certificate. Next, I have to go the server side and add the client's "public" certificate:

```
>keytool -import -alias client_pub -file client_pub.crt
-keystore server.jks -storepass ServerJKS

Owner: CN=my.client.com, OU=My Unit, O=My Home, L=My City, ST=My S...
Issuer: CN=my.client.com, OU=My Unit, O=My Home, L=My City, ST=My ...
.....
Trust this certificate? [no]: yes
Certificate was added to keystore

>keytool -list -keystore server.jks -storepass ServerJKS

Keystore type: jks
Keystore provider: SUN

Your keystore contains 2 entries

server_full, Mar 29, 2005, keyEntry,
Certificate fingerprint (MD5): 34:71:CD:2F:E8:D9:32:57:34:61:46:4C...
client_pub, Mar 29, 2005, trustedCertEntry,
Certificate fingerprint (MD5): 53:5F:62:00:4A:5F:0E:DC:1A:8F:4B:8E...
```

I think I am ready to perform a SSL communication with both server and client authentications. What do you think?

Server Program for Client Authentication - SslReverseEchoerRevised

To perform both server and client authentications, I have revised my sample program SslReverseEchoer.java:

```
/**
 * SslReverseEchoerRevised.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
import java.security.*;
import javax.net.ssl.*;
public class SslReverseEchoerRevised {
    public static void main(String[] args) {
        if (args.length<3) {
            System.out.println("Usage:");
```

```

        System.out.println(
            "    java SslReverseEchoerRevised ksName ksPass ctPass");
        return;
    }
    String ksName = args[0];
    char[] ksPass = args[1].toCharArray();
    char[] ctPass = args[2].toCharArray();
    System.setProperty("javax.net.ssl.trustStore", args[0]);
    System.setProperty("javax.net.ssl.trustStorePassword",
        args[1]);
    try {
        KeyStore ks = KeyStore.getInstance("JKS");
        ks.load(new FileInputStream(ksName), ksPass);
        KeyManagerFactory kmf =
            KeyManagerFactory.getInstance("SunX509");
        kmf.init(ks, ctPass);
        SSLContext sc = SSLContext.getInstance("SSL");
        sc.init(kmf.getKeyManagers(), null, null);
        SSLServerSocketFactory ssf = sc.getServerSocketFactory();
        SSLServerSocket s
            = (SSLServerSocket) ssf.createServerSocket(8888);
        s.setNeedClientAuth(true);
        printServerSocketInfo(s);
        SSLSocket c = (SSLSocket) s.accept();
        printSocketInfo(c);
        BufferedWriter w = new BufferedWriter(new OutputStreamWriter(
            c.getOutputStream()));
        BufferedReader r = new BufferedReader(new InputStreamReader(
            c.getInputStream()));
        String m = "Welcome to SSL Reverse Echo Server."+
            " Please type in some words.";
        w.write(m,0,m.length());
        w.newLine();
        w.flush();
        while ((m=r.readLine())!= null) {
            if (m.equals(".")) break;
            char[] a = m.toCharArray();
            int n = a.length;
            for (int i=0; i<n/2; i++) {
                char t = a[i];
                a[i] = a[n-1-i];
                a[n-i-1] = t;
            }
            w.write(a,0,n);
            w.newLine();
            w.flush();
        }
        w.close();
        r.close();
        c.close();
        s.close();
    } catch (Exception e) {
        System.err.println(e.toString());
    }
}

private static void printSocketInfo(SSLSocket s) {
    System.out.println("Socket class: "+s.getClass());
    System.out.println("    Remote address = "
        +s.getInetAddress().toString());
    System.out.println("    Remote port = "+s.getPort());
    System.out.println("    Local socket address = "
        +s.getLocalSocketAddress().toString());
}

```



```

        System.out.println("    Local address = "
            +s.getLocalAddress().toString());
        System.out.println("    Local port = "+s.getLocalPort());
        System.out.println("    Need client authentication = "
            +s.getNeedClientAuth());
        SSLSession ss = s.getSession();
        try {
            System.out.println("Session class: "+ss.getClass());
            System.out.println("    Cipher suite = "
                +ss.getCipherSuite());
            System.out.println("    Protocol = "+ss.getProtocol());
            System.out.println("    PeerPrincipal = "
                +ss.getPeerPrincipal().getName());
            System.out.println("    LocalPrincipal = "
                +ss.getLocalPrincipal().getName());
        } catch (Exception e) {
            System.err.println(e.toString());
        }
    }
}
private static void printServerSocketInfo(SSLServerSocket s) {
    System.out.println("Server socket class: "+s.getClass());
    System.out.println("    Socker address = "
        +s.getInetAddress().toString());
    System.out.println("    Socker port = "
        +s.getLocalPort());
    System.out.println("    Need client authentication = "
        +s.getNeedClientAuth());
    System.out.println("    Want client authentication = "
        +s.getWantClientAuth());
    System.out.println("    Use client mode = "
        +s.getUseClientMode());
}
}

```

The changes are minor:

- Key store file name, and passwords are moved to command line level.
- The specified key store file is used to supply trusted "public" certificates with system properties.
- More SSL session information is printed.

Client Program for Client Authentication - SslSocketClientRevised

Of course, I also revised SslSocketClient.java:

```

/**
 * SslSocketClientRevised.java
 * Copyright (c) 2005 by Dr. Herong Yang
 */
import java.io.*;
import java.net.*;
import java.security.*;
import javax.net.ssl.*;
public class SslSocketClientRevised {
    public static void main(String[] args) {
        if (args.length<3) {
            System.out.println("Usage:");
            System.out.println(

```

```

        " java SslReverseEchoerRevised ksName ksPass ctPass");
    return;
}
String ksName = args[0];
char[] ksPass = args[1].toCharArray();
char[] ctPass = args[2].toCharArray();
System.setProperty("javax.net.ssl.trustStore", args[0]);
System.setProperty("javax.net.ssl.trustStorePassword",
    args[1]);
BufferedReader in = new BufferedReader(
    new InputStreamReader(System.in));
PrintStream out = System.out;
try {
    KeyStore ks = KeyStore.getInstance("JKS");
    ks.load(new FileInputStream(ksName), ksPass);
    KeyManagerFactory kmf =
    KeyManagerFactory.getInstance("SunX509");
    kmf.init(ks, ctPass);
    SSLContext sc = SSLContext.getInstance("SSL");
    sc.init(kmf.getKeyManagers(), null, null);
    SSLSocketFactory f = sc.getSocketFactory();
    SSLSocket c =
    (SSLSocket) f.createSocket("localhost", 8888);
    printSocketInfo(c);
    c.startHandshake();
    BufferedWriter w = new BufferedWriter(
        new OutputStreamWriter(c.getOutputStream()));
    BufferedReader r = new BufferedReader(
        new InputStreamReader(c.getInputStream()));
    String m = null;
    while ((m=r.readLine())!= null) {
        out.println(m);
        m = in.readLine();
        w.write(m,0,m.length());
        w.newLine();
        w.flush();
    }
    w.close();
    r.close();
    c.close();
} catch (Exception e) {
    System.err.println(e.toString());
}
}
private static void printSocketInfo(SSLSocket s) {
    System.out.println("Socket class: "+s.getClass());
    System.out.println("  Remote address = "
        +s.getInetAddress().toString());
    System.out.println("  Remote port = "+s.getPort());
    System.out.println("  Local socket address = "
        +s.getLocalSocketAddress().toString());
    System.out.println("  Local address = "
        +s.getLocalAddress().toString());
    System.out.println("  Local port = "+s.getLocalPort());
    System.out.println("  Need client authentication = "
        +s.getNeedClientAuth());
    SSLSession ss = s.getSession();
    try {
        System.out.println("Session class: "+ss.getClass());
        System.out.println("  Cipher suite = "
            +ss.getCipherSuite());
        System.out.println("  Protocol = "+ss.getProtocol());
    }
}

```

```

        System.out.println("    PeerPrincipal = "
            +ss.getPeerPrincipal().getName());
        System.out.println("    LocalPrincipal = "
            +ss.getLocalPrincipal().getName());
        System.out.println("    PeerPrincipal = "
            +ss.getPeerPrincipal().getName());
    } catch (Exception e) {
        System.err.println(e.toString());
    }
}
}

```

The changes are minor:

- Key store file name, and passwords are moved to command line level.
- The specified key store file is used to supply trusted "public" certificates with system properties.
- More SSL session information is printed.

Running Client Authentication Programs

With SslReverseEchoerRevised.java and server.jks prepared on the server side, I am ready to start the server program:

```

>java -cp . SslReverseEchoerRevised server.jks ServerJKS ServerKey
Server socket class:
  class com.sun.net.ssl.internal.ssl.SSLServerSocketImpl
  Socker address = 0.0.0.0/0.0.0.0
  Socker port = 8888
  Need client authentication = true
  Want client authentication = false
  Use client mode = false

```

Now switch to the client side, and run the client program:

```

java -cp . SslSocketClientRevised client.jks ClientJKS ClientKey
Socket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
  Remote address = localhost/127.0.0.1
  Remote port = 8888
  Local socket address = /127.0.0.1:1418
  Local address = /127.0.0.1
  Local port = 1418
  Need client authentication = false
Session class: class com.sun.net.ssl.internal.ssl.SSLSessionImpl
  Cipher suite = TLS_DHE_DSS_WITH_AES_128_CBC_SHA
  Protocol = TLSv1
  PeerPrincipal = CN=my.server.com,OU=My Unit,O=My Home,L=My City...
  LocalPrincipal = CN=my.client.com,OU=My Unit,O=My Home,L=My Cit...

Welcome to SSL Reverse Echo Server. Please type in some words.
Client authentication is working!
!gnikrow si noitacitnehtua tneilC
.

```

Looking at the server side again, you will see messages:

```
Socket class: class com.sun.net.ssl.internal.ssl.SSLSocketImpl
  Remote address = /127.0.0.1
  Remote port = 1418
  Local socket address = /127.0.0.1:8888
  Local address = /127.0.0.1
  Local port = 8888
  Need client authentication = true
Session class: class com.sun.net.ssl.internal.ssl.SSLSessionImpl
  Cipher suite = TLS_DHE_DSS_WITH_AES_128_CBC_SHA
  Protocol = TLSv1
  PeerPrincipal = CN=my.client.com,OU=My Unit,O=My Home,L=My City...
  LocalPrincipal = CN=my.server.com,OU=My Unit,O=My Home,L=My Cit...
```

Wonderful! Everything worked I expected. Client program authenticated the server's identity ok, and server program authenticated the client's identity ok too.

Conclusion

- SSL server authentication seems to be required by default.
- SSL client authentication is optional.
- Client authentication is a mirror process of server authentication.
- JSSE uses SSLContext and KeyManager to access the "full" certificate on the local side.
- JSSE uses system properties to supply remote (peer) "public" certificates as trusted, if remote certificates are not issued by recognized certificate authorities.

References

- *Java 2 SDK, Standard Edition, Documentation, Version 1.3.1*, Sun Microsystems, <http://java.sun.com/j2se/1.3/docs/index.html>
- *Java 2 SDK, Standard Edition Documentation, Version 1.4.1*, Sun Microsystems, <http://java.sun.com/j2se/1.4.1/docs/index.html>
- *Java Cryptography Extension (JCE)*, Sun Microsystems, <http://java.sun.com/products/jce/>
- *The WeakHashMap Class*, Jack Shirazi, 07/09/2001, <http://www.onjava.com/pub/a/onjava/2001/07/09/optimization.html>
- *Java Cryptography Architecture API Specification and Reference*, Dec 1999, Sun Microsystems, <http://java.sun.com/j2se/1.3/docs/guide/security/CryptoSpec.html>
- *Java Cryptography Extension (JCE) Reference Guide*, Sun Microsystems, <http://java.sun.com/j2se/1.3/docs/guide/security/JCERefGuide.html>
- *keytool - Key and Certificate Management Tool* Sun Microsystems, java.sun.com/j2se/1.3/docs/tooldocs/win32/keytool.html
- *Java 1.1 Unleashed*, Macmillan Computer Publishing, <http://lasim.univ-lyon1.fr/c.ray/bks/java/htm/toc.htm>
- *Java Programming Tutorialsd*, Richard G Baldwin, <http://www.dickbaldwin.com>
- *SSL Certificates HOWTO*, Franck Martin, <http://www.tldp.org/HOWTO/SSL-Certificates-HOWTO>
- *Secure Internet Programming with Java 2, Standard Edition (J2SE) 1.4*, Qusay H. Mahmoud, <http://java.sun.com/developer/technicalArticles/Security/secureinternet>
- *The SSL Protocol, Version 3.0*, <http://wp.netscape.com/eng/ssl3/draft302.txt>